

Verification Without Correspondence

Semantic Fidelity and the Limits of Formal Proof

Flyxion
Independent Researcher

October 2026

Contents

Preface	9
I The Verification Problem	13
1 The Proof That Was Not the Proof	15
1.1 A verified proof of a correct theorem	15
1.2 A verified proof by a different argument	16
1.3 Five situations that must not be merged	16
1.4 What remains to be established	16
2 Capability, Verification, Understanding, and Objective	19
2.1 The four axes	19
2.2 Are the axes independent?	20
2.3 Five positions as characteristic reductions	20
2.4 Ketterer’s proposed position	21
3 Three Meanings of Mathematical Success	23
3.1 Result, derivation, task	23
3.2 A taxonomy of properties	24
3.3 Why no binary criterion suffices	24
3.4 Two results previewed	25
4 The Authority of the Proof Checker	27
4.1 What the kernel decides	27
4.2 What the answer depends on	27
4.3 Three kinds of defect and a fourth kind of excess	28
4.4 Jurisdiction	28
II Translation and Semantic Fidelity	31
5 Natural Language as Mathematical Specification	33
5.1 What a mathematical sentence leaves unsaid	33
5.2 Readings	34
5.3 Presupposition and the default	35
5.4 Interpretation as practice	35

5.5	What the chapter does not claim	36
6	Autoformalization as Constrained Translation	37
6.1	Four tasks under one name	37
6.2	Two forms of success	38
6.3	What a translation preserves	38
6.4	The translation contract	39
6.5	Contracts for the cases of Part IV	40
6.6	Limits of the contract	41
7	The Substitution Problem	43
7.1	Tasks and certifiers	43
7.2	Acceptance does not transfer: RV-015	43
7.3	The error of an unvetted substitution	44
7.4	Levels of substitution	45
7.5	Operators and their observable consequences	46
7.6	What follows	47
8	Back-Translation and the Circularity of Certification	49
8.1	The round trip	49
8.2	Agreement does not entail fidelity: RV-007	49
8.3	Why the second translation is a second obligation	51
8.4	Independent translations	51
8.5	Controlled vocabularies and certified glossaries	52
8.6	Certified intermediate representations	53
8.7	Summary of the chapter	53
III	The Computational Limits of Interpretation	55
9	Undecidability and Hidden Presuppositions	57
9.1	The example	57
9.2	The silent default	58
9.3	Three ways to represent a partial definition	58
9.4	Acceptance loops and the elementary analogue	59
9.5	What the example does and does not show	59
10	The Solvability Complexity of Semantic Preservation	61
10.1	Prerequisites	61
10.2	The least-number families	62
10.3	From the construction to a class of specifications	63
10.4	Restriction and the open boundary	64
10.5	Consequences for the theory of residue	64

11 Impossibility Without Futility	65
11.1 Unrestricted hardness and useful partial procedures	65
11.2 What soundness can and cannot recover	66
11.3 Restricted languages and domain-specific certificates	66
11.4 Human adjudication and conservative abstention	67
11.5 The design principle	67
 IV The Navier-Stokes Case	 69
12 A Derivative Lost in Translation	73
12.1 The two estimates	73
12.2 Why one derivative is lost	74
12.3 What the Lean result does and does not establish	74
12.4 The case in the ledger	75
12.5 On the choice of consequence relation	75
12.6 Three repairs	76
12.7 What the chapter establishes	76
 13 The Pressure-Flux Discrepancy	 77
13.1 The setting and the two estimates	77
13.2 Three differences of argument	78
13.3 Two cautions from the source	78
13.4 The case in the ledger	79
13.5 Statement fidelity and argument fidelity apart	79
 14 What the Navier-Stokes Formalization Establishes	 81
14.1 Three kinds of evidence	81
14.2 Claims that remain justified, in question, or unsupported	82
14.3 The Euler claim and the limits of prediction	83
14.4 The mechanism: acceptance loops	83
14.5 Back-translation does not escape the problem	83
14.6 The state of the residue	84
 V A Theory of Verification Residue	 85
15 Obligations That Survive Verification	87
15.1 What a successful check leaves unsettled	87
15.2 The obligation record	88
15.3 Five ways of being unsettled	88
15.4 The residue and its partition	89
15.5 Historical and warranted discharge	89
15.6 Theorem correctness and proof fidelity	90

16 The Non-Discharge Principle	91
16.1 Kernel acceptance does not entail correspondence	91
16.2 Statement fidelity and argument fidelity are independent	92
16.3 Strengthened hypotheses and substituted tasks	92
16.4 The gap is not computable	94
16.5 A candidate non-discharge principle	95
17 Preservation, Discharge, and Transformation	97
17.1 Why traceability is not enough	97
17.2 Transitions, certificates, and the entailment condition	98
17.3 The typing of transitions	98
17.4 Traceability: the syntactic theorem	98
17.5 Compositional preservation: the semantic theorem	99
17.6 Preservation is not discharge	100
17.7 Weakening: the Navier-Stokes cases	100
17.8 Propagation through dependency structures	101
17.9 Optional information and the link to admissible degradation	102
17.10 What remains	103
18 Epistemic Immutability	105
18.1 Versions, events, and the append-only ledger	105
18.2 Correction and retrospective certification	106
18.3 A worked case	106
18.4 Supersession, retraction, and independent certification	107
18.5 Tamper evidence	107
18.6 Provenance in practice: two training reports	108
18.7 What the model does not do	109
VI Objectives, Optimization, and Institutional Scale	111
19 Reward Functions and the Production of Substitution	113
19.1 Reward as an equivalence relation	113
19.2 Correct answers do not identify correct reasoning	114
19.3 The pipeline as a chain of transformations	115
19.4 Objective admissibility and task admissibility	115
19.5 Reviewability and adequacy	116
19.6 Illustrations from training reports	116
19.7 Design implications	118
20 Publication Under Abundance	119
20.1 Reports at one remove	119
20.2 The accounting argument	120
20.3 Volume is a poor measure of contribution	120
20.4 Five things that should not be confused	121
20.5 A publication architecture	121

VII Architectures of Responsible Verification	123
21 A Residue-Aware Verification System	125
21.1 Proof checking as one component	125
21.2 The multidimensional status	126
21.3 The ledger and the guarantees it supports	127
21.4 A Lean 4 prototype	128
21.5 What the system does not claim	129
22 Limits of Certification	131
22.1 Two answers that fail	131
22.2 Bounded certification	132
22.3 Preservation is not discharge	132
22.4 Understanding	132
A Formal Definitions and Logical Conventions	135
A.1 Verification instances and the three judgments	135
A.2 Contents and the consequence relation	136
A.3 Obligations, statuses, and the residue	136
A.4 Certificates, historical discharge, and warranted discharge	137
A.5 Transitions and transition certificates	139
A.6 Traceability: RV-020 and RV-017	140
A.7 Compositional preservation: RV-008	141
A.8 Independence of preservation and discharge: RV-018	142
A.9 Weakening and the total certified context: RV-019	143
A.10 Representation of certification status: RV-014	143
A.11 Audit of the specification	144
B Computability Background and Derivations	145
B.1 Standard facts	145
B.2 The set d	146
B.3 What the corollary does not say	146
B.4 From d to statement correspondence	147
B.5 Trustworthy autoformalizers	147
B.6 Summary of dependencies	148
C A Catalogue of Counterexamples	149
C.1 Instances that separate the three judgments	149
C.2 Instances for the ledger	150
C.3 Instances for substitution and translation	151
C.4 Instances from the Navier–Stokes formalization	151
C.5 Summary	152
D A Lean 4 Prototype: Declarations and Their Status	153
D.1 Three activities kept apart	153
D.2 Reproducibility record	154
D.3 Classification of declarations	154

D.4	Source listings	155
D.5	Known divergences from Appendix A	160
D.6	What a successful compile would and would not show	161
D.7	Procedure for the first compile	161
D.8	Relation to the test suite of Chapter 21	162
E	Navier–Stokes Source Audit	163
E.1	Purpose, and what counts as primary	163
E.2	Artifacts consulted	164
E.3	Audit of the two documented discrepancies	165
E.4	What the audit changes in the main text	168
E.5	Audit of the claims of Chapter 20	168
E.6	Register of unverified claims and their resolving sources	173
E.7	Limits of this audit	174
	Bibliography	175
	Note on the Status of the References	177

Preface

A formal proof that a proof assistant accepts is a strong object. It has been checked by a small trusted kernel against a stated environment, and no further argument is needed to know that the term has the type it is said to have. This monograph is about what is not settled by that fact. The type may not be the proposition the author meant, the term may not follow the argument the author gave, and the report of success may therefore assert something that the check never examined. The three questions are kept apart throughout: whether the kernel accepts, whether the formal statement corresponds to the intended one, and whether the formal argument corresponds to the intended one.

The occasion is the account by Bastounis, Circelli, and Hansen [3] of the Lean verification of an AI-assisted natural-language proof concerning the Navier–Stokes equations, which documents discrepancies between natural-language estimates and the Lean statements that stand in for them. The monograph takes no position on whether the underlying mathematics is correct, and neither does the source. It asks what follows about the standing of a verified artifact when a translation step has not itself been verified, and it answers with definitions, a small number of theorems, and a discipline for reporting.

What is claimed

The claims are of four kinds, and the text marks which kind each is.

Proved results. Separation results show that the three judgments above are distinct (Chapter 16). A formal specification of obligations, certificates, and ledgers is given in Appendix A, with a traceability theorem and a compositional theorem about preservation. Computability results, due to the source and reconstructed in Appendix B, show that the question of whether a defining term is well defined is hard at every level of the arithmetical hierarchy on classes of specifications that embed a particular family of constructions.

Conditional results. Several statements depend on conventions that are stated and can be changed, for instance that a specification with an undefined term corresponds to no formal proposition, or that the consequence relation is derivability from a certified background. The text gives the dependence each time.

Source-reported claims. What the Lean statements are, and in what respect they differ from the natural-language ones, is taken from the source. A later text-level reading of the cited passages and declarations, recorded in Appendix E, confirmed the displayed estimates and the exponents, but nothing was compiled and the classification below was not confirmed. The classification of the two documented discrepancies as a weakening and a lateral change is marked source-supported and provisional wherever it occurs.

Proposals. The architecture of Part VII and the reporting practices are design proposals. They are argued for and are not proved to work.

What is not claimed

The monograph does not claim that formal verification is unreliable, that the Navier–Stokes argument is wrong, that automated translation is hopeless, or that any particular system has misreported its results. It does not claim a decision procedure for correspondence, and its main negative result is that none exists for unrestricted classes. It does not claim that the restricted classes encountered in practice are hard, and says in several places that they may not be. The preprints on language-model training that are used as illustrations in Part VI are used for the structure they exhibit, their reported figures are not reproduced beyond what the argument needs, and none of them has been independently checked here.

How the book is organized

Part I states the problem through two small cases and separates the senses in which a result can be said to succeed. Part II treats natural language as a specification, autoformalization as constrained translation, substitution of one task for another, and the limits of back-translation. Part III gives the computability results and what survives them. Part IV applies the framework to the documented Navier–Stokes discrepancies. Part V develops the theory of obligations, preservation, and discharge, proves the separation theorems, and treats provenance and the immutability of the record. Part VI turns to reward functions as a source of substitution and to publication under abundance. Part VII describes a residue-aware verification system and closes with the limits of certification. Appendix A is the formal specification on which Part V depends, Appendix B collects the computability background, Appendix C catalogues the counterexamples, Appendix D documents an uncompiled Lean prototype, and Appendix E audits the source claims about the Navier–Stokes formalization and the developments of Chapter 20, marking each as confirmed, reported, interpreted, or unverified.

A reader who wants only the argument can read Chapters 1, 3, 4, and 7, the statement of RV-001 in Chapter 16, and Chapter 21. A reader who wants the mathematics should begin with Appendix A.

Conventions and status

Results are identified by labels of the form RV-*nnn*, which are stable across the text and the appendices. A result stated in one chapter and restated in another is proved once. Throughout, *preserved* and *discharged* are different words for different things: preservation is an entailment relation among contents along a chain of transformations, and discharge is a status of an obligation backed by a certificate that may later lose its warrant. Appendix A fixes both.

Review passes and AI assistance

Review procedure. The mathematical development, meaning Appendix A and the chapters that depend on it, was put through three review passes. Each was carried out by an AI agent

given the text and a written brief to look for false statements, proofs that do not establish the advertised conclusion, hidden assumptions, and cross-reference defects. The passes were run in sequence. The second and third read the text as revised after the preceding pass, and their briefs named the earlier findings. The agents did not have access to the source papers, so claims attributed to them were checked only for internal consistency. Their reports are retained with the manuscript sources. The first pass reported four findings that it classed as serious, the second three, and the third none. Every finding was reviewed by the author and the assistant that drafted the text, and the corrections made are in the text.

What “independent” means here. The word is used only in the sense that the three passes were distinct runs, each with a fresh context that did not contain the drafting conversation. It does not mean epistemic independence. The agents are probably instances of the same family of model as the assistant that drafted the text, and the manuscript does not record the identity of any of them. They read the same text, were given overlapping briefs, and may share training, blind spots and assumptions with each other and with the drafter. Agreement among them is therefore weak evidence of correctness, and the absence of findings in the third pass is not evidence that no errors remain. The passes were not a substitute for refereeing by specialists in logic and proof assistants, and the proofs should be treated as unrefereed.

Role of AI assistance. A substantive part of this work was done with an AI assistant, and the distinctions below are those the book itself urges.

- *Drafting.* The prose and the $\text{\LaTeX}$ source of the chapters and appendices were written by the assistant, working from the author’s direction and outline. This is the largest contribution.
- *Mathematical discovery.* The thesis, the choice of problem, the list of results RV-001 to RV-021 in outline form, and the editorial decisions are the author’s. The assistant proposed formulations, proofs, counterexamples and corrections within that frame, and some statements changed during drafting in response to the review findings (for example the statement of RV-004, which the first pass showed to be false as originally stated). Where a result was first formulated is not recorded precisely, and the author should be consulted before any attribution of a result is made.
- *Proof checking.* No proof in the manuscript has been checked by a proof assistant. The Lean prototype of Appendix D is uncompiled and contains placeholders. Proof checking has consisted of the three AI review passes above and of the assistant’s own re-reading.
- *Independent validation.* None has been done. No specialist has refereed the mathematics, no source claim in Appendix E has been checked by anyone other than the single AI reviewer who prepared it, and no result has been reproduced by a party outside this process.

Part I

The Verification Problem

Chapter 1

The Proof That Was Not the Proof

A formal verification is an event with a definite outcome: a program called a kernel has checked a term against a type and found that the term inhabits it. The outcome is reliable in the sense that matters to the engineering of proof assistants, and the monograph does not dispute it. What it examines is the distance between that outcome and the sentences that are written about it. A formal proof of a statement, a published argument for a theorem, and an answer to a question posed in ordinary mathematical language are three different things, and a verification event bears directly on only the first. This chapter opens the subject with two small cases, taken from the paper of Bastounis, Circelli, and Hansen [3] that supplies the principal documented material of the monograph, in which every step is elementary and in which the distance can be seen without any technical machinery.

1.1 A verified proof of a correct theorem

The first case is Example 2.1 of the source. The theorem to be proved is that the polynomial $p(x) = x^3 - x^2 - x + 1$ is nonnegative for $x \geq -1$. The supplied natural-language argument proceeds by factorization. It asserts that $p(x) = (x-1)(x+1)^2$, with the root -1 of multiplicity two and the root 1 of multiplicity one, and then concludes from the sign pattern of the factors. The factorization is wrong. Expanding $(x-1)(x+1)^2$ gives $x^3 + x^2 - x - 1$, which is not p , and the correct factorization is

$$p(x) = (x+1)(x-1)^2,$$

with the root -1 of multiplicity one and the root 1 of multiplicity two. The theorem is nevertheless true: for $x \geq -1$ the factor $x+1$ is nonnegative and $(x-1)^2$ is nonnegative, so the product is nonnegative.

(For the programme of computer-checked mathematics in general, see [2]; for Lean and its library, [14, 13].) A chatbot asked to translate the supplied argument into Lean returned a proof that compiles. In the course of the translation it found the correct multiplicities, and the proof that Lean accepted follows the correct factorization. The formal proof is valid, it proves the intended theorem, and the kernel accepts it. It does not validate the supplied argument, because the supplied argument is not valid and the formal proof is not a formalization of it. A faithful translation of the argument as written would have failed to compile, and it is the success of the translation that conceals the discrepancy.

A reader who is told only that the translation succeeded learns three things correctly and a fourth incorrectly. The reader learns that a Lean proof of the theorem exists, that it compiles, and that the theorem is true. The reader is also invited to infer that the original argument was sound, and that inference is false. Nothing in the kernel’s judgment bears on it.

1.2 A verified proof by a different argument

The second case, Example 2.2 of the source, is the complementary one. For a real symmetric matrix A the theorem is that $\text{tr}(A^2) \geq 0$. The natural-language proof diagonalizes A in an orthonormal eigenbasis, so that $\text{tr}(A^2) = \sum_i \lambda_i^2 \geq 0$. The Lean proof that was returned omits the change of basis. It uses the symmetry of A directly to write $\text{tr}(A^2) = \sum_{i,j} a_{ij}^2$, a sum of squares, from which nonnegativity is immediate.

Both proofs are correct. The theorem is the same, the formal statement expresses it, and the kernel accepts the proof. What differs is the argument, and the difference is not an error. The Lean proof is arguably more direct. A verification of the intended theorem has been obtained and the supplied argument has not been formalized, since the diagonalization, which is the central step of that argument, has no counterpart in the formal proof.

1.3 Five situations that must not be merged

The two cases, with the others that the source documents, separate five situations that are easily assimilated to a single category of machine error. A *false theorem* is a statement that is not true, and no proof of it exists. An *invalid proof of a true theorem* is an argument with an error whose conclusion happens to hold, as in the first case. A *valid alternative proof* is a correct argument that differs from the one supplied, as in the second. A *mistranslated theorem* is a formal statement that is provable and that is not the intended statement, as in the hidden-existence example of Chapter 9, where a library default silently replaces an undefined quantity. A *verified statement weaker than the original* is a formal result that implies less than the claim it stands in for, as in the derivative estimate of Chapter 12.

The consequences of the five differ. A false theorem should be detected by the failure of any proof attempt. An invalid proof of a true theorem is repaired by replacing the argument, and the repair is evidence about the theorem and not about the original argument. A valid alternative proof is a gain and not a defect. A mistranslation and a weakening are failures of fidelity that the kernel does not see, and they are the failures that matter for the monograph. Treating them as instances of one thing, called an AI error, loses the information that determines what should be done in each case. The remedies are different, the evidence needed to detect them is different, and the claims that survive are different.

1.4 What remains to be established

The cases lead to the question that organizes the rest of the monograph. After a proof checker accepts a derivation, what remains to be established? A partial answer is already available from the two examples. It remains to be established that the formal statement expresses the intended statement, which is the statement correspondence of Chapter 16. It remains to be established that the formal proof follows the argument that was supplied, which is argument

correspondence. And it remains to be established that the definitions, imports, and conventions on which the formal statement depends mean what they were taken to mean, which is the dependency question of Chapter 17.

Each of these is a question that can be asked of any verification, and none has the status of an optional refinement. The first part of the monograph argues that they are distinct from kernel acceptance, the second that they are the content of autoformalization, the third that no total procedure can settle them in general, the fourth that a documented case exhibits them concretely, and the fifth that they can be represented and preserved even when they cannot be settled. The sixth and seventh parts turn to objectives and institutions and to the design of a system that keeps the questions visible. The thesis of the work is that a verification should be reported together with its residue, the precise list of what remains to be established, and that the habit of reporting only the acceptance gives an account that is correct about the kernel and silent about nearly everything else.

Chapter 2

Capability, Verification, Understanding, and Objective

The motivation for this monograph includes an account of mathematical training offered by Daniel Ketterer [11], who trains language models in mathematical reasoning and who distinguishes four axes along which an artificial system can be assessed. The account is first-person testimony about failure modes observed in practice, and it is used here for the distinctions it draws and not for the empirical claims it makes in passing. Two public events that Ketterer cites, an Erdős-conjecture result and a security breach, are specific historical claims that the monograph does not rely on. This chapter reconstructs the four axes, asks whether they are independent and whether they are of one logical kind, and examines the five positions that Ketterer describes as competing views, with a view to locating them in the framework of the later chapters.

2.1 The four axes

Capability is the question whether a system can produce a correct result. Verification is the question whether it can be mechanically established that the result is correct. Understanding concerns how a human knows why a result is true. Objective concerns what the system is actually optimizing and what it can reach. In the training of models on mathematical problems, as Ketterer describes it, the reward typically comes from a verification, usually of the final answer and sometimes of intermediate steps, and so the verifier functions as the objective.

Ketterer’s central observation is that the verification can be satisfied without the task being accomplished. He reports having watched models make several errors that cancel and still be rewarded, and, more significantly, having watched models misrepresent a problem, solve the different problem correctly, and be rewarded, with no human intervention on whether the problem solved was the problem posed. The two failures differ in kind. A sign error produces a locally invalid step, and a locally invalid step is the kind of defect that step-level checking can find. A substitution of the problem produces a sequence in which every step after the restatement is valid for the problem actually solved, and the only flawed step is the restatement itself. Detecting that step requires knowing what the original problem meant. This is what Ketterer calls understanding, in contrast with mechanical checking, and he draws the consequence that if a real solution cannot be told from a substitution that lands on the same

answer, the training signal will not distinguish them either.

2.2 Are the axes independent?

The first question is whether the four axes are independent in the sense that the value of any one is not determined by the others. It is natural to ask this in the form of the implications that a position might assume. Does capability imply verification, so that whatever a system can do it can be checked doing? Does verification imply capability, so that a system whose outputs pass the check is capable? Does understanding imply either, or follow from either?

The framework of Parts V and VI answers some of these directly. Kernel acceptance of a formal proof does not entail statement correspondence (Theorem 16.1), and statement correspondence does not entail argument correspondence (Theorem 16.2). A verification that a result passes a check does not entail that the task was accomplished (Theorem 19.1 and Proposition 19.3), since the check is a function of a reward that may fail to factor the task predicate. Conversely a system may be capable of producing a correct, task-satisfying result in a case where no mechanical check is available for the property that makes it correct, so capability does not entail verification. These are the implications for which the monograph supplies an argument or a construction. The capability-to-verification case is an informal observation and is not constructed, the implications involving understanding are not treated, and I do not claim a full table of pairwise independence of the four axes.

The second question is whether the axes are properties of the same logical kind. The remark that they are not is due to the commentary this monograph shares with Ketterer's account, and it is correct. Capability describes what a system can accomplish, which is a modal property of the system. Verification describes an epistemic procedure for establishing a property of an output, which is a relation between a procedure and an artifact. Understanding describes a cognitive or interpretive relationship between a person and a result. Objective describes a criterion governing optimization, which is a property of a training process. A taxonomy that lists them as four values of one variable suggests that they can be traded off against one another along a single scale, and they cannot. In the vocabulary of Part V, capability is a property of the artifact producer, verification is a certificate, understanding is the authority behind a correspondence judgment, and objective is a transformation of the task into a reward.

2.3 Five positions as characteristic reductions

Ketterer describes five positions in the public discussion of mathematical progress by language models and shows that each collapses one axis into another or omits one. The advocates collapse verification into capability, taking a passed check as the achievement of the goal. The debunkers collapse capability into understanding, concluding from the absence of understanding that capability is absent. The pragmatists collapse verification into understanding, on the ground that for one researcher checking and understanding occur in the same head, which fails at the scale at which training produces more traces than anyone can read. The ecological views keep understanding central but treat it as a property of the field, and they are silent about parties that are not bound by the field's norms. The alignment-based views put the objective first and omit understanding, although checking whether a restatement is faithful requires it, and to automate that check is to build another verifier.

It is a natural reading that these are competing theories. A better reading, which the commentary on his essay proposes and which this monograph adopts, is that they are characteristic ways of overprivileging one dimension of a multidimensional problem. A researcher may hold that models are capable, insist on formal verification, regard human understanding as indispensable, and prioritize alignment simultaneously, and nothing in the logic forbids it. The categories are not exhaustive or mutually exclusive as communities. They are useful as a catalogue of the reductions, each of which corresponds to a specific inference that Part V shows to be invalid. The advocates' inference from the passing of a check to the accomplishment of a task is the failure of Theorem 19.1. The debunkers' inference from absence of understanding to absence of capability is a failure of independence between the axes. The pragmatists' identification of checking with understanding fails because the two can be separated by scale. The framework of the monograph represents each of the four as a separate component of a status structure and does not require any of them to be collapsed.

2.4 Ketterer's proposed position

Ketterer's own position is to put understanding where the objective is set. Traces are numerous and objectives are few, so that nobody can read every trace but someone can read the formal statement, the reward definition, and the scorer before training starts. The proposal is a sound allocation of attention, and as an argument about where human interpretive effort is affordable it is persuasive. Chapter 19 examines it in detail and argues that it addresses one of two admissibilities. Review of the objective before training bears on whether the criterion expresses the intended class of tasks. It does not bear on whether an operation performed after training, on a problem the reviewers never saw, preserves the obligations of the task, since the interpretation of the novel problem is constructed at inference time. The chapter concludes that understanding at the point of objective-setting is necessary and not sufficient.

The remainder of this chapter's contribution is a framing. The four axes locate the problem. The monograph's own concept, the verification residue, is a refinement along the verification and understanding axes: it names what remains to be established after the verifier has spoken, and it provides a place for the output of understanding, namely the correspondence judgment recorded as a certificate with an identified authority. The question that Part II takes up is how that residue arises when the process that produces the formal artifact is a translation from natural language.

Chapter 3

Three Meanings of Mathematical Success

A report that a mathematical task has succeeded can mean at least three things, and the failure to distinguish them is the source of a large part of the confusion that this monograph addresses. The distinctions are not subtle once stated, and they have an exact form that the later chapters use. This chapter states them, proposes an initial taxonomy of the properties that a mathematical output may have, and explains why those properties cannot in general be reduced to one binary criterion of acceptance.

3.1 Result, derivation, task

Result success is the property that the final output of a procedure is correct, meaning that it agrees with the right answer. A numerical computation that returns the right number has result success. *Derivational success* is the property that the route by which the output was obtained is valid, meaning that each step follows from the steps before it by accepted inference. *Task success* is the property that the output answers the question that was originally posed, in the sense in which it was posed.

The three come apart in each direction. A correct numerical answer may be reached through invalid reasoning: two errors may cancel, or a wrong method may land on the right value, as in the trajectory that computes $2 + 3$ as $2 \cdot 3 - 1$. A correct theorem may be proved through a derivation different from the one supplied, as in the symmetric-matrix case of Chapter 1, so that derivational success of the produced proof does not carry derivational success of the supplied one. And a formally correct result may fail to answer the question originally posed, as in the substitution of $x^3 = 27$ for $x^2 = 9$ with $x > 0$, over the real numbers, where every step is valid and the answer 3 is correct for both problems and yet the problem solved is not the problem posed.

The relation among the three is better understood as a system of predicates, none of which is in general a function of the others, than as a hierarchy. Let a trajectory h produce an output $y(h)$ for a task T . Result success is $y(h) \in \text{Sol}(T)$ when answers are checked against a solution set. Derivational success is the validity of h as a derivation. Task success is the satisfaction of the obligations of T by h , which includes that the derivation addresses T and not a substitute. Result success follows from task success for the class of tasks whose obligations

include correctness of the answer, and does not follow for tasks whose obligations omit it; the converse fails by the examples above, for every task class considered. Derivational success does not entail task success, since a valid derivation may be of the wrong problem, and task success does not entail derivational success of every step if the task is satisfied by a different route.

3.2 A taxonomy of properties

The three notions are the coarsest of a family. A usable taxonomy distinguishes at least the following properties of a mathematical output relative to a task. *Correctness* is the truth of the stated result. *Validity* is the correctness of the inferences of the derivation from its stated premises, which does not include the truth of those premises. *Fidelity* is the correspondence of the stated or formalized result, and of the argument, to the intended statement and argument. *Relevance* is the bearing of the output on the question asked, as distinct from its correctness. *Completeness* is the coverage of all cases or conditions that the task requires. *Adequacy* is the sufficiency of the output for the purposes for which it was requested, which may include being intelligible, being checkable, or being usable in later work.

The claim made about these properties is that none is in general a consequence of another, and it is made by examples and not by an exhaustive matrix of the thirty ordered pairs. The examples establish that the following properties can be had without the one that follows. A correct but irrelevant result is a true theorem offered in response to a different question, and has correctness without relevance. A valid but incomplete proof treats the cases it treats correctly and omits others, and has validity without completeness. A valid derivation from a false premise has validity without correctness. A faithful but incorrect formalization of a false statement is an accurate translation of an error, and has fidelity without correctness. An adequate but not fully valid argument is a sketch sufficient to convince an expert and containing a gap, and has adequacy without validity. The remaining ordered pairs are not claimed here, and some may be excluded in particular classes of task; the independence asserted is relative to the predicates as defined and to the existence of such instances. The taxonomy does not claim to be minimal or exhaustive, and its purpose is to show that the single word *correct* in ordinary use stands for a conjunction of properties that can be separately satisfied.

3.3 Why no binary criterion suffices

A binary acceptance criterion is a function from outputs to $\{0, 1\}$. If it is to be used as a proxy for the conjunction of the properties above, it must factor through them in the sense that the value depends only on their joint values and agrees with the intended aggregate. Two obstacles arise. The first is that the conjunction of several independent properties does not determine the value of any single one, so an acceptance criterion that is true exactly when all hold will be false in cases where only the property of interest to a particular user fails, and will conflate failures that call for different remedies. The second is the one made precise in Chapter 19 for rewards (RV-010): a reward that takes equal values on two outputs that differ in the task predicate cannot agree with the task on both, and the irreducible disagreement is measured by the conditional impurity within the level sets of the reward.

The consequence for reporting is that a verification should state which of the properties it establishes. The kernel establishes validity of the formal derivation relative to a stated envi-

ronment, and it establishes correctness of the formal statement in the sense that the statement is provable there. It establishes nothing about fidelity, relevance, or adequacy. A report that says the proof was verified is correct if read as a statement about the first and misleading if read as a statement about the rest, and the monograph's practice is to attach to the word the property it names.

3.4 Two results previewed

Two of the monograph's results are stated informally here because they express the distinctions of this chapter and are proved later. The first is RV-004, which says that a proof can be valid under strengthened hypotheses without proving the original statement. If a derivation proves $H' \rightarrow G$ where H' is strictly stronger than the intended hypothesis H , the derivation is valid and relevant to a special case, and it does not by itself yield a proof of $H \rightarrow G$, which is the stronger theorem. The second is the substitution result RV-015, proved in Chapter 7, which states, for a sound and complete certifier, exactly when acceptance for a substituted task transfers to the original. The proof of RV-004 is given in Chapter 16. Both are instances of the general fact that validity of a derivation is relative to what the derivation is a derivation of.

Chapter 4

The Authority of the Proof Checker

The reliability of a proof checker is a technical achievement, and the argument of this monograph depends on it. The question of this chapter is the extent of its authority. A proof checker is an authority on one question, whether a given term has a given type in a given environment, and it is not an authority on the questions that surround that one. The chapter states what the question is, lists what the answer depends on, classifies the ways in which the claims made on the checker’s behalf can exceed its jurisdiction, and argues that the checker can be entirely reliable while those claims are not warranted.

4.1 What the kernel decides

The kernel of a system such as Lean checks that a term inhabits a type under an accepted logical environment. In the notation of Chapter 1 and Appendix A, it decides the judgment $K(V) \iff \Gamma \vdash \pi : F$, where Γ is the environment, F the proposition, and π the term. The judgment is a function of those three arguments. It does not mention the natural-language specification N , the purported argument P , or the interpretation $\mathcal{I}$, and Theorem 16.1 exploits exactly this fact. It does not adjudicate the historical or linguistic intentions behind the type.

A proof assistant is organized so that the kernel is a small trusted component and the rest is not trusted. Tactics, automation, and the elaboration of source syntax into terms are untrusted in the sense that their outputs are rechecked by the kernel, and an error in a tactic can cause a proof to fail and cannot cause a false term to be accepted unless it exploits a defect of the kernel. This is the familiar de Bruijn criterion, and it explains why a proof that the kernel accepts can be trusted as a proof of its type. It also explains the limits of the trust, because everything that precedes the kernel’s input, including what the statement says, belongs to the untrusted part and is never checked by the kernel against anything outside the system.

4.2 What the answer depends on

The kernel’s judgment is relative to the environment Γ , and the environment is a substantial object. It contains the axioms in force, the definitions that give meaning to the symbols of the statement, the imported libraries with their theorems and conventions, any constants declared opaque or without a defining equation, and any assumptions the development has introduced. Each of these is a place where a claim made about the verification can outrun what was checked.

Axioms fix the logical strength under which acceptance holds. A result accepted in an environment with additional axioms is a result relative to them, and a claim that a theorem was proved in the base system is false if an extra axiom was used. For this reason, the source's criterion for success in translating a proof into Lean includes that Lean accepts the result without a placeholder for a missing proof and without additional axioms. *Definitions* determine what the statement says. A definition may be total by convention in a way that departs from the informal notion, as in the library *infimum* that returns zero on the empty set, which is the source of the silent default of Chapter 9. The kernel accepts the definition because it is a definition, and it does not check that it matches the informal object. *Imports* bring in the conventions of a library, and a result that uses an imported lemma is a result relative to the library's statement of the lemma, not to its informal description. *Opaque constants and assumptions* enter the environment without a definition and so contribute to what the final theorem is conditional on, whether or not the theorem's statement displays the dependence.

A practical corollary is that an environment should be reported with a result, since acceptance is acceptance in an environment. The ledger of Chapter 21 stores it for that reason.

4.3 Three kinds of defect and a fourth kind of excess

The defects that can affect a verified result are of different kinds and the chapter keeps them apart. A *kernel-level defect* is a flaw in the trusted component that allows a false term to be accepted. It is the kind of defect that the engineering of proof assistants aims to exclude, and it is rare. A *specification error* is a discrepancy between the formal statement and the intended statement, and it is not a defect of the kernel, since the kernel correctly checked the statement it was given. It includes the weakening, the lateral substitution, and the silent default documented in earlier chapters. An *environment defect* is a flaw in a definition, an import, or an assumption on which the statement depends, so that the statement is not what it appears to be even though it is stated as intended. These three are defects in the object. The fourth category is an excess in the claim: an *inappropriate interpretation of a verified result*, in which a correct verification of a correct statement is read as establishing more than it does, for example as confirming a particular published argument.

The distinction between the first and the others is the most important for institutional purposes. Criticism of a verification effort is sometimes taken to be criticism of the kernel and is answered with the correct observation that the kernel is reliable. The documented discrepancies of Part IV are not kernel-level. They are specification-level, and they are untouched by the kernel's reliability. The monograph's statement is that a verifier can be entirely reliable while the claims made on its behalf exceed its jurisdiction, and the examples show that the two conditions are independent.

4.4 Jurisdiction

The word jurisdiction is used in the following sense. The jurisdiction of a verifier is the set of questions on which its output constitutes a certificate in the sense of Appendix A: a statement accepted by an identified authority under an identified interpretation with an identified set of dependencies. For the kernel the authority is the trusted checker, the interpretation is the type theory, and the dependencies are the environment. The certificate is of the formal

obligation O_{formal} . It is not a certificate of $O_{\text{statement}}$ or O_{argument} , whose certificates, if any, come from other authorities under other interpretations, such as a human reader's judgment that a definition matches the informal notion.

An assertion of the form that a theorem has been verified therefore has a jurisdictional reading, namely that O_{formal} is discharged. Other readings are claims about obligations on which the verifier was never consulted. The practice recommended throughout this monograph is that each such claim be attached to the obligation it concerns and to the authority behind it. Where no authority has ruled, the obligation is open, and the honest report says so. The first result of Part V, Theorem 16.1, makes the statement that kernel acceptance does not entail statement correspondence precise, and it is stated here in outline: since the kernel's judgment is invariant under changes to the originating specification, no function of it can determine whether the formal statement corresponds to that specification.

Part II

Translation and Semantic Fidelity

Chapter 5

Natural Language as Mathematical Specification

Every verification instance in this monograph begins with a natural-language specification N , and everything the rest of the pipeline can establish is an establishment about some formal object that was derived from it. The relation between N and that object is therefore the first thing that needs an account. This chapter gives one. Mathematical prose is neither unrestricted ordinary language nor a fully formal calculus, and the account has to respect both halves of that description. It has to explain why mathematicians can usually communicate a statement precisely in prose, and it has to explain why a precise reading is nevertheless not a property of the words alone.

5.1 What a mathematical sentence leaves unsaid

A theorem statement in a paper is written for a reader who shares a great deal with the author. The reader knows the conventions of the subfield, the definitions in force in the section, the standing assumptions announced three pages earlier, and the intended domain of each variable. The author relies on all of this, and a formalizer who sees only the sentence does not have it. The following features recur.

Implicit quantification. The sentence “ f is continuous” leaves open the domain on which continuity is asserted, and the sentence “the matrix A satisfies $A^2 = I$ ” leaves open whether A is fixed by context or universally quantified. A sentence with a free variable is read as universally closed in some fields and as existentially closed in others, and a sentence with several quantifiers can be read with more than one scope.

Contextual definitions. Terms such as “bounded,” “compact,” “smooth,” “measurable,” and “a.e.” denote different notions in different texts. Within a single text the definition may have been given once, forty pages earlier, and may have been modified by a later remark that applies only in a particular section.

Inherited assumptions. A lemma is often stated under the standing hypotheses of its section, and the hypotheses do not reappear in the statement. A formalization of the lemma in isolation either omits them, which yields a different statement, or adds them, which requires knowing which ones apply.

Existence presuppositions. Definite descriptions such as “the least n such that” and “the

unique solution” presuppose that the described object exists and is unique. The presupposition is part of how the sentence is understood and is not stated in it.

Conventional abbreviations and informal inference. Phrases such as “clearly,” “by symmetry,” and “without loss of generality” carry argumentative content that the explicit syntax does not. They matter for O_{argument} rather than for $O_{\text{statement}}$, and they are discussed here because the same reader-dependence governs them.

5.2 Readings

I make the dependence on context explicit by treating interpretation as a relation. Let $\mathcal{L}_N$ be the language of natural-language specifications and $\mathcal{L}_F$ a formal language with its meaning function $\llbracket \cdot \rrbracket_{\mathcal{I}}$ into a common domain of propositional contents.

Definition 5.1 (Context and reading). *A context c for a specification N records the definitions in scope, the conventions of the field, the standing hypotheses, and the intended domains of the variables of N . A reading of N in c is a formal sentence $F \in \mathcal{L}_F$ that a competent reader, informed of c , would accept as a rendering of N . The set of readings is denoted $\text{Read}_c(N)$.*

The definition does not assume that $\text{Read}_c(N)$ is a single point, and it does not assume that it is a single equivalence class under $\equiv_{\mathcal{I}}$. It assumes only that the notion of a competent reader is meaningful, which is the same assumption on which mathematical communication rests. The two properties of the set that matter are the following.

Definition 5.2 (Resolved and ambiguous specifications). *A specification N is resolved in c if all elements of $\text{Read}_c(N)$ are pairwise equivalent, $F \equiv_{\mathcal{I}} F'$ for all $F, F' \in \text{Read}_c(N)$, and $\text{Read}_c(N) \neq \emptyset$. It is ambiguous in c if $\text{Read}_c(N)$ contains two inequivalent sentences, and unreadable in c if $\text{Read}_c(N) = \emptyset$.*

The statement correspondence of Appendix A can now be stated relative to a context. A formalization F corresponds to N in c when F is equivalent to a reading of N in c . The following observation records the consequences for the resolved and ambiguous cases and for a change of context; the unreadable case has no corresponding formalization to compare and is reported as such. It is an elementary consequence of the definitions and is recorded because each case corresponds to a different kind of report.

Proposition 5.3 (Correspondence is context-relative). *Let $F \in \mathcal{L}_F$ and let c, c' be contexts for N .*

- (i) *If N is resolved in c , then F corresponds to N in c if and only if $F \equiv_{\mathcal{I}} F_0$ for any one reading F_0 of N in c .*
- (ii) *If N is ambiguous in c , then there is an F that corresponds to N in c and is inequivalent to another reading of N in c ; correspondence to N in c then does not determine the content of F up to equivalence.*
- (iii) *If $\text{Read}_c(N) \neq \text{Read}_{c'}(N)$ as sets of equivalence classes, there is an F that corresponds in one context and not in the other.*

Proof. (i) Equivalence is transitive and all readings are equivalent. (ii) Take two inequivalent readings F_1, F_2 ; each corresponds to N in c and they are inequivalent. (iii) Take F in a class that belongs to one set of classes and not to the other. $\square$

Part (ii) is the form in which ambiguity enters the instance: an acceptance of “a formalization of N ” certifies nothing about which reading was formalized. Part (iii) is the form in which a change of context, for example a different edition of a text or a different standing hypothesis, enters. A report on statement correspondence is therefore incomplete unless it names the context.

5.3 Presupposition and the default

The treatment of definite descriptions deserves separate attention, because it is the point at which prose conventions and formal-language conventions diverge in a way that Chapter 9 exploits. In prose, “the least n such that $B(n)$ ” is understood to require that some n satisfy B . If none does, the sentence has no truth value or is regarded as defective, depending on the philosophical convention. In a formal library, a function that computes the infimum of a set of natural numbers must be total, and the standard convention assigns a default value on the empty set. The two conventions agree whenever the presupposition holds and diverge whenever it fails.

Proposition 5.4 (Agreement of a total formalization with a presupposing description). *Let $S \subseteq \mathbb{N}$ be given by a predicate B and let ιS denote the description “the least element of S ,” which is defined exactly when $S \neq \emptyset$. Let inf^* be a total function on subsets of $\mathbb{N}$ with $\text{inf}^* S = \min S$ for nonempty S and $\text{inf}^* \emptyset = d_0$ for a fixed default d_0 . For any formula $\Phi(x)$,*

- (i) *if $S \neq \emptyset$ then $\Phi(\iota S)$ and $\Phi(\text{inf}^* S)$ have the same truth value, and*
- (ii) *if $S = \emptyset$ then $\Phi(\iota S)$ is undefined and $\Phi(\text{inf}^* S)$ has the truth value of $\Phi(d_0)$, which may be either.*

Proof. For nonempty S the two terms denote the same natural number, so the truth values coincide. For empty S the description is undefined by assumption, and the formal term denotes d_0 . $\square$

The proposition has a plain consequence for what a reader may infer. When a formalization uses a total function in place of a description, the formalization is a faithful rendering on the part of the parameter space where the presupposition holds. On the rest it is a rendering of a different sentence, which the prose did not utter. Whether that is a defect depends on whether the specification purports to cover the rest, and that is decided by the context c and not by the formal library. The family of Chapter 9 turns this into a statement about the complexity of deciding which part of the parameter space one is in.

5.4 Interpretation as practice

The reading relation cannot be reduced to a function from sentences to propositions, and the reason is not a defect of any particular function. Competent readers disagree on some readings and agree on others, the agreement rests on training and shared examples, and the boundaries

of what counts as an acceptable reading are themselves objects of mathematical discussion. The account of this chapter does not attempt to replace that practice with a formal definition. It records the practice as the ground of $O_{\text{statement}}$, and it draws two consequences for the rest of the monograph.

First, the authority behind a statement-correspondence certificate is always a reader, human or otherwise, operating in a context. A certificate from a model that reads the specification and judges the formalization faithful is a certificate from an authority of this kind, and its standing is determined by what is known about the reliability of the authority in the relevant context, as Chapter 4 would require of any authority. Second, when the authorities disagree, the disagreement is information. The ledger of Chapter 21 stores the readings and their provenance, and it does not collapse them.

5.5 What the chapter does not claim

The chapter does not claim that natural-language mathematics is inadequately precise. It claims that the precision it has is a property of a community with its conventions, and that a pipeline which removes the community, as automated translation does, must either reconstruct the conventions explicitly or accept that some of its statement correspondences are undetermined. The next chapter describes what is being asked of translation under that constraint.

Chapter 6

Autoformalization as Constrained Translation

Autoformalization is the production of formal mathematics from informal mathematics by an automated system. The term covers tasks that differ in what is to be preserved, and the differences matter for what a success report can mean. This chapter separates the tasks, adopts the two forms of success that the source distinguishes, and proposes a record, called a translation contract, in which the claims made on behalf of a translation can be stated and later audited.

6.1 Four tasks under one name

The following tasks are commonly grouped.

- (i) *Theorem formalization.* Given a statement in prose, produce a formal statement. The output has no proof. Its correctness is exactly statement correspondence, $C(V)$, in the notation of Appendix A.
- (ii) *Proof reconstruction.* Given a formal statement, produce a formal proof. The informal argument may be consulted or ignored. Correctness is kernel acceptance of the proof of the given statement.
- (iii) *Proof translation.* Given a prose proof of a statement together with a formalization of the statement, produce a formal proof that follows the prose argument. Correctness has a kernel component and an argument component, $A(V)$.
- (iv) *Whole-document translation.* Given a mathematical text, produce a formal development containing the definitions, statements, proofs, and dependencies of the text. Correctness concerns the preservation of the text as a whole.

The tasks are ordered by the amount of the original that has to survive. In the first, only a statement does. In the last, a structure of definitions and dependencies does, and a failure of preservation can occur at any node or at any edge.

6.2 Two forms of success

The source identifies two forms of autoformalization, and the monograph adopts them in the following paraphrase. I state them here because Part IV’s cases are cases of the gap between them.

Definition 6.1 (Form 1: acceptance of a translated proof). *Given a natural-language proof of a theorem and a formalization of the theorem that has already been accepted as faithful, a translation is successful in form 1 if the resulting Lean development is accepted by the kernel without any placeholder for a missing proof and without additional axioms.*

Definition 6.2 (Form 2: faithful semantic translation). *Given a mathematical text, a translation is successful in form 2 if it is a faithful semantic translation of the entire text, including its definitions, statements, proofs, and the dependencies between them. A different but provable assertion does not count as a faithful translation.*

The definitions differ in what they presuppose and in what they check. Form 1 presupposes that the statement has been faithfully formalized, and its criterion is a property of the formal artifact alone: the kernel accepts, no placeholder remains, no axiom has been added. Form 2 presupposes nothing and its criterion refers to the original. The difference has an immediate logical consequence, stated here with its limits.

Proposition 6.3 (Form 1 does not entail form 2). *Let D be a formal development meeting the form 1 criterion with respect to a statement F that has been accepted as faithful. Then form 2 success does not follow: D need not be a faithful semantic translation of the text N from which the proof was taken. Specifically, the form 1 criterion is a function of D and its environment, and cannot register a proof in D whose steps lack counterparts in the informal proof.*

Proof. The form 1 criterion mentions only that the kernel accepts D , that no placeholder remains, and that no axiom has been added. The instance is the symmetric-matrix example of Chapter 1 and Theorem 16.2, in which a faithfully formalized statement is proved by a route without counterparts, under a strict argument-correspondence relation, for the steps of the informal proof. The form 1 criterion holds and the form 2 criterion, which requires faithful translation of proofs as well as statements, fails. The criterion is also silent about definitions, and Chapter 9 documents a total library default that departs from the informal notion; I do not claim that it yields a form 1 instance with a faithful statement, and the proposition rests on the argument case alone. $\square$

The proposition is stated with the statement held faithful, as form 1 presupposes, so it is not a restatement of Theorem 16.1. It rests on the separation results for definitions and arguments and inherits their conditions. The source’s own reading, as I understand it, is that the Lean translations it examines meet form 1 and that the question of form 2 is what its analysis shows to be unresolved. This is a reading of the source and is marked as such.

6.3 What a translation preserves

A translation, of any of the four kinds, can be described as a sequence of steps, each of which transforms a representation of the mathematics into another. A step may rename, expand a definition, strengthen or weaken a hypothesis, introduce a convention, or replace an argument by

another with the same conclusion. Appendix A types such transitions as conservative, strengthening, weakening, or lateral, relative to a certified background, and proves in Theorem A.17 that a chain of admissible, preserving steps yields entailment of the originating content from the cumulative context, and that this entailment becomes establishment only when the premises are established. The vocabulary is intended for use here: what a translation preserves is the class of content types that its steps are not allowed to change.

Definition 6.4 (Content types). *The content of a mathematical text is divided into the following types, which are taken as the dimensions of preservation. Statements are the propositions asserted. Definitions are the meanings assigned to terms. Hypotheses are the conditions under which a result is asserted, including standing assumptions. Arguments are the inferential structure of the proofs. Dependencies are the relations between results, namely which result is used to prove which. Scope is the intended range of the quantifiers and variables.*

The list is not claimed to be canonical. It is intended to be fine enough that the failures documented in Part IV, a weakened exponent in a hypothesis, a lateral change of the set on which an estimate holds, and a total default for a partial term, each fall under exactly one type, and coarse enough to be manageable.

6.4 The translation contract

A translation is made under an understanding of what is to be preserved and what is to be disclosed. The understanding is rarely written down. When it is not, claims of success have no fixed referent, and a reader cannot tell whether a given change was licensed. The contract makes the understanding explicit.

To attach content types to the obligations of a ledger, fix a *tagging* κ , a partial function assigning to an obligation one content type of Definition 6.4, and call a ledger *tag-respecting* if $(o, o') \in Q_i$ implies $\kappa(o') = \kappa(o)$. The tagging is defined on the obligations that descend from the originating obligations of the content being translated. It is undefined on the baseline obligation O_{formal} and on tracked assumption obligations, which are outside the six types, and compliance is required only where it is defined. The tagging is part of the record: which obligation of a translation concerns a statement, which a definition, and which a dependency is a decision, made by whoever sets up the ledger, and it can itself be wrong.

Definition 6.5 (Translation contract). *A translation contract is a quadruple $\mathfrak{C} = (\text{Pres}, \text{Strict}, \text{Perm}, \text{Disc})$ where Pres is a set of content types that must be preserved, Strict $\subseteq$ Pres is the set of those types at which only conservative steps are compliant, Perm is a set of the transition types {conservative, strengthening, weakening, lateral} that are permitted at obligations whose type lies outside Pres, and Disc is a disclosure rule requiring that every occurrence of a permitted type other than conservative be recorded with its obligation, its direction, and its justification.*

A tag-respecting ledger *complies with* $\mathfrak{C}$ if, for every step T_i and every $o \in \mathcal{O}_{i-1}$ on which κ is defined: when $\kappa(o) \in \text{Pres} \setminus \text{Strict}$, the step is preserving at o , that is, of type conservative or strengthening at o ; when $\kappa(o) \in \text{Strict}$, the step is of type conservative at o ; and when $\kappa(o) \notin \text{Pres}$, the type of T_i at o lies in Perm and, if it is not conservative, it is disclosed as Disc requires. A contract is specific to a task. For form 1, Pres is the statement as given, and the contract can require more than the form 1 criterion checks, for example the arguments, at the

price of leaving form 1. For form 2, $\text{Pres} = \text{Strict}$ contains all six types and Perm is empty, since a strengthening step is a departure from a faithful translation. The two contracts differ in how much of the original must survive, which is the difference between the criteria; I do not claim that they differ in nothing else.

Proposition 6.6 (What compliance gives). *Let a tag-respecting ledger of admissible transition certificates comply with a contract $\mathfrak{C}$, over a consequence relation satisfying (C1) to (C3).*

- (i) *For every originating obligation o with $\kappa(o) \in \text{Pres}$, $\Gamma_n(o) \models_{\mathcal{I}} \varphi(o)$.*
- (ii) *For every originating obligation o with $\kappa(o) \notin \text{Pres}$, the ledger records each step at which preservation fails along the descendants of o , so that the failure is disclosed and not silent; compliance gives no entailment for such o .*
- (iii) *Compliance does not give establishment of $\varphi(o)$. By Proposition A.18(i) a compliant chain can preserve an open obligation through any number of steps without discharging it, and by Theorem A.17 establishment further requires, besides sound certificates, that the sources of the premises in $\Gamma_n(o)$ be warranted-discharged.*

Proof. (i) By tag-respect, every obligation in $S_{k-1}(o)$ has a tag in Pres , so every step is preserving at every obligation reached from o . The first statement of Theorem A.17, which needs preservation only at the obligations reached from o , then applies and gives $\Gamma_n(o) \models_{\mathcal{I}} \varphi(o)$. (ii) Likewise every obligation reached from o has a tag outside Pres , and at each of them compliance requires a type in Perm and disclosure of any type other than conservative; the disclosures are the record. A step of type weakening or lateral, if permitted, defeats the hypothesis of Theorem A.17, which is why no entailment is claimed. (iii) is Proposition A.18(i) together with the second statement of Theorem A.17. $\square$

The proposition does not say that compliance with a good contract guarantees a good translation. Its content is that the contract fixes which obligations are held to preservation, which departures are tolerated and reported, and that even full compliance leaves the question of establishment where it was, with the authorities behind the discharges and the soundness of $\models_{\mathcal{I}}$. Entailment is a logical statement and does not depend on whether certificates remain warranted; establishment does. A contract serves two functions: it fixes the referent of a claim of fidelity, and it makes the claim auditable because each step is either covered by the contract or reported as a violation. The tagging on which both rest is an additional certified claim.

6.5 Contracts for the cases of Part IV

The use of the contract can be illustrated by what a form 1 contract fails to say about the cases of Part IV. Under a form 1 contract the statement is given and Pres contains it, so the translation of the proof is not licensed to change the statement. In the derivative case, the weakening of the exponent is a change to a hypothesis inside a proof step, not to the stated theorem, and the contract is silent on intermediate claims. A contract with Pres extended to include the hypotheses of cited lemmas, and with Perm empty, would classify the step as a violation at the moment of translation and require disclosure. The classification is the source-supported, provisional classification of Chapter 12, and the example is one of what a more explicit contract would have recorded, not a statement that a violation was certified.

6.6 Limits of the contract

The contract is a record and not a procedure. It does not say how to determine whether a step is conservative. That determination is a transition certificate, whose authority is a reader or a prover, and which is subject to the limits of Part III: for the class of specifications that contains hidden-existence families, no uniform procedure decides statement correspondence, and a contract cannot manufacture one. What the contract adds is accountability about which determinations were made and by whom, and a place for the determinations that were not made.

Chapter 7

The Substitution Problem

A verification can succeed on a problem other than the one that was posed. The phenomenon has three features that are separate in the literature and are separated here: the posed problem and the solved problem differ in content, the success criterion is satisfied for the solved problem, and the satisfaction is reported as success on the posed problem. The chapter gives a model of tasks and certifiers in which these features can be stated, proves the basic result (RV-015), classifies the operators that produce substitution, and states what an answer-based check can and cannot see.

7.1 Tasks and certifiers

Definition 7.1 (Task). A task T is a pair $(Q_T, \text{Sol}(T))$, where Q_T is a question, specified in some language, and $\text{Sol}(T) \subseteq \mathcal{S}$ is the set of candidates in a fixed candidate space $\mathcal{S}$ that count as solving Q_T . Candidates may be numerical answers, truth values, proofs, or formal terms, according to the task.

A task is thus identified with the pair of its question and its solution set, and two tasks with the same solution set over the same candidate space can have different questions. The model deliberately retains Q_T , since the substitution problem is that the questions differ while a check that sees only candidates cannot tell.

Definition 7.2 (Certifier). A certifier for T' is a function $\kappa_{T'} : \mathcal{S} \rightarrow \{0, 1\}$. It is sound for T' if $\kappa_{T'}(s) = 1$ implies $s \in \text{Sol}(T')$, and complete for T' if the converse holds. It is answer-based if $\kappa_{T'}(s)$ depends on s only through a map $\alpha : \mathcal{S} \rightarrow \mathcal{A}$ to a space of final answers.

A certifier is constructed for the task its designer has in mind, which is T' , the task actually specified in the check. The task the designer or the user intends is T . The question of the chapter is what acceptance by $\kappa_{T'}$ says about membership in $\text{Sol}(T)$.

7.2 Acceptance does not transfer: RV-015

Proposition 7.3 (RV-015, substitution under acceptance-only certification). Let T and T' be tasks over the same candidate space, and let $\kappa_{T'}$ be a certifier for T' that is sound for T' . Acceptance by $\kappa_{T'}$ establishes membership in $\text{Sol}(T)$ for every candidate that $\kappa_{T'}$ can accept if and only if $\text{Acc} \subseteq \text{Sol}(T)$, where Acc is the set of accepted candidates. If $\kappa_{T'}$ is also complete for

T' , this holds if and only if $\text{Sol}(T') \subseteq \text{Sol}(T)$. Without completeness, the inclusion $\text{Sol}(T') \subseteq \text{Sol}(T)$ is sufficient and not necessary. In either case, apart from degenerate certifiers, the values of $\kappa_{T'}$ do not determine whether the inclusion holds, and it must be separately justified.

Proof. Let $\text{Acc} = \{s : \kappa_{T'}(s) = 1\}$. Soundness gives $\text{Acc} \subseteq \text{Sol}(T')$. If $\text{Sol}(T') \subseteq \text{Sol}(T)$ then $\text{Acc} \subseteq \text{Sol}(T)$, which is the sufficiency. Suppose now that $\kappa_{T'}$ is complete, so that $\text{Acc} = \text{Sol}(T')$, and suppose the inclusion fails. Choose $s \in \text{Sol}(T') \setminus \text{Sol}(T)$. Then $\kappa_{T'}(s) = 1$ and $s \notin \text{Sol}(T)$, so acceptance does not establish membership in $\text{Sol}(T)$ for every acceptable candidate, which is the necessity. Without completeness the inclusion is not necessary: let $\kappa_{T'}$ accept only a proper subset of $\text{Sol}(T')$ that lies in $\text{Sol}(T)$ while $\text{Sol}(T') \not\subseteq \text{Sol}(T)$. For the last sentence: the function $\kappa_{T'}$ is fixed once T' and the certifier are fixed, while the truth of $\text{Sol}(T') \subseteq \text{Sol}(T)$ varies with T over tasks sharing the candidate space and $\kappa_{T'}$, so the values of $\kappa_{T'}$ cannot determine it. $\square$

The proposition has the shape of Proposition 16.3 for statements, and it is stated for solution sets because the substitution can occur at the level of an answer, a statement, or an argument. The cases are recorded in the form that practice needs. If $\text{Sol}(T') = \text{Sol}(T)$, acceptance transfers, and the substitution is harmless for this candidate space even though $Q_T \neq Q_{T'}$. If $\text{Sol}(T') \subsetneq \text{Sol}(T)$, so that T' asks more of a candidate, acceptance transfers and some solutions of T are rejected. If $\text{Sol}(T) \subsetneq \text{Sol}(T')$, so that T' asks less, acceptance can be a false accept for T . The correspondence between these cases and the changes to a statement is given below for statement-level tasks.

7.3 The error of an unvetted substitution

The proposition is qualitative. A quantitative form is available when the candidates are drawn from a distribution, and it shows exactly where an answer-based reading is blind.

Proposition 7.4 (Error of reading acceptance as success). *Let μ be a probability measure on $\mathcal{S}$ for which the solution sets are measurable, let $\kappa_{T'}$ be sound and complete for T' , and consider the claim “ s solves T ” made whenever $\kappa_{T'}(s) = 1$ and denied otherwise. The probability that the claim is wrong is $\mu(\text{Sol}(T) \triangle \text{Sol}(T'))$. The false-accept mass is $\mu(\text{Sol}(T') \setminus \text{Sol}(T))$ and the false-reject mass is $\mu(\text{Sol}(T) \setminus \text{Sol}(T'))$.*

Proof. The claim is wrong at s iff exactly one of $s \in \text{Sol}(T')$ and $s \in \text{Sol}(T)$ holds, that is, iff s lies in the symmetric difference. The two components are the two parts of that set. $\square$

The proposition says that the claim is wrong exactly on the symmetric difference of the two solution sets and right on its complement, so the check can fail only there. It also says that substituting a task does not matter at all when the symmetric difference has measure zero under the distribution the candidates are drawn from, and may matter greatly when the distribution is concentrated on the difference. The latter occurs when candidates are produced by an optimizer that is searching for acceptance by $\kappa_{T'}$, which places mass where acceptance is easy and not necessarily where T is solved. Chapter 19 returns to this in connection with selection by an optimizer.

Answer collisions

For an answer-based certifier, the blindness is more extreme. Suppose $\kappa(s) = \beta(\alpha(s))$ for fixed α and β . This is one function, and it is the same function whichever task was intended, so its verdict on each candidate is identical for any two tasks that it is used to certify. Whether the verdict is correct for either depends only on the solution sets, and the check itself cannot register a difference between the questions. The simplest case is a task whose answer is a truth value. Let T ask whether $\sum_{n \geq 1} n^{-2}$ converges and T' ask whether $\sum_{n \geq 1} n^{-3}$ converges. Both have the answer “yes”, and an answer-based check cannot tell whether the question that was answered is the one that was asked. For a task with a two-element answer space, a random substitution agrees on the answer with substantial probability, and for a task with a numerical answer, agreement on the answer can still be produced by distinct questions that were selected to share it.

This is the structure of the failure that practitioners of reinforcement learning on mathematical problems describe: the final answer is checked, the model restates the problem in a different form, solves the restated problem correctly, and obtains the reward. The posed problem and the solved problem differ in content, and their answers coincide. The monograph does not rely on a particular report of this behavior, and its analysis does not depend on frequency. The proposition shows that an answer-based reward is blind to the substitution on the entire set where the answers agree, whatever the frequency.

7.4 Levels of substitution

The substitution can occur at different levels of the instance of Chapter 1. The classification is by the component of the instance that is replaced.

Level	Replaced component	Illustration
Statement	The proposition F is replaced by F'	The exponent of a weight is weakened by one in a hypothesis of a lemma (Chapter 12)
Assumption	A hypothesis is strengthened, weakened, or added	A standing assumption of a section is dropped or silently added in the formalization (Chapter 5)
Domain	The set on which a claim holds is changed	The time quantifier of an estimate is changed from a.e. t to all t , and a dependence on a further quantity is added (Chapter 13)
Definition	A term is bound to a different meaning, including a total default for a partial term	Infimum on the empty set (Chapter 9)
Argument	The proof is replaced by another proof of the same conclusion	The symmetric-matrix example (Chapter 1, Theorem 16.2)
Objective	The success condition is replaced by a proxy	A scorer checking a final value in place of a derivation (Chapter 19)

Statement, assumption, domain, and definition substitutions all alter $\llbracket F \rrbracket_{\mathcal{I}}$ and so alter Sol for tasks whose candidates are proofs of F . Argument substitution leaves the statement

unchanged and alters the argument obligation. Objective substitution leaves the posed task intact and replaces the criterion by which candidates are scored, so it alters κ rather than T' ; it is the form that the training chapters take up.

7.5 Operators and their observable consequences

A substitution operator is a map σ from tasks to tasks. The classification by the relation of $\text{Sol}(\sigma(T))$ to $\text{Sol}(T)$ gives four classes, loosely parallel to the four transition types of Appendix A.

Class	Relation of solution sets	Consequence of acceptance by $\kappa_{\sigma(T)}$
Equivalent	$\text{Sol}(\sigma T) = \text{Sol}(T)$	Transfers
Restricting	$\text{Sol}(\sigma T) \subsetneq \text{Sol}(T)$	Transfers; false rejects possible
Enlarging	$\text{Sol}(\sigma T) \supsetneq \text{Sol}(T)$	Does not transfer; false accepts
Crossing	neither inclusion	Does not transfer; both errors

The table's verdicts on transfer assume that $\kappa_{\sigma(T)}$ is sound and complete for $\sigma(T)$. Without completeness, the first two rows still transfer by soundness, and the last two rows may or may not transfer depending on the accepted set.

Statement-level tasks

For tasks whose candidates are derivations, the model of Definition 7.1 needs a convention, because derivations of different statements are different objects. Fix $\models_{\mathcal{I}}$ as in Appendix A. For a formal statement F let T_F be the task with $\text{Sol}(T_F) = \{\pi : \text{concl}(\pi) \models_{\mathcal{I}} F\}$, the derivations whose conclusion is at least as strong as F . This is the reading under which a proof of a stronger theorem is a proof of a weaker one.

Lemma 7.5 (Direction at the statement level). *If $F \models_{\mathcal{I}} F'$ then $\text{Sol}(T_F) \subseteq \text{Sol}(T_{F'})$.*

Proof. If $\text{concl}(\pi) \models_{\mathcal{I}} F$ and $F \models_{\mathcal{I}} F'$ then $\text{concl}(\pi) \models_{\mathcal{I}} F'$ by cut. $\square$

Assume further that F' has a derivation in the candidate space and that $F' \not\models_{\mathcal{I}} F$. Then the inclusion is strict, and replacing the intended statement F by the strictly weaker F' is an enlarging substitution, and acceptance for $T_{F'}$ does not transfer to T_F in general, while replacing it by a strictly stronger F' is restricting and acceptance transfers. A theorem with a strengthened hypothesis, $H' \rightarrow G$ for H' stronger than H , is weaker than $H \rightarrow G$, so it is an enlarging substitution, which is the content of Proposition 16.3. A weakened conclusion is likewise enlarging, given the deduction property used in Proposition 16.3, since $G \models G'$ then gives $(H \rightarrow G) \models (H \rightarrow G')$. For the statement-level model the kernel accepts only derivations whose conclusion is the stated one, so it is sound and not complete for $T_{F'}$; the verdict that an enlarging substitution does not transfer then needs the accepted set itself to leave $\text{Sol}(T_F)$, which holds under the strictness assumption. The two operations are not opposites at this level: both enlarge. The classification of the Navier–Stokes cases remains that of Part IV, source-supported and provisional, and under this lemma and subject to the source's non-entailment and to derivability of the Lean estimate, the derivative case, in which the Lean estimate is weaker, is enlarging, and this too is provisional.

The observable consequence of each class is the error structure of Proposition 7.4. Under the completeness assumption, equivalent and restricting substitutions produce no false accepts. Enlarging and crossing substitutions produce them, and the mass depends on the distribution of candidates. This is the quantity that an audit should estimate if the audit is allowed to sample.

7.6 What follows

Three conclusions are used later. Acceptance by a sound check for T' is evidence about T only through a separately justified relation between T and T' , which is a content-level claim and is the subject of $O_{\text{statement}}$. The error of an unvetted reading is concentrated on the symmetric difference of the solution sets and is therefore worst exactly when candidates are selected for acceptance. And an answer-based reward is blind to every substitution that preserves the answer. Chapter 8 turns to a natural proposal for detecting substitution, which is to translate the formal object back into prose and compare, and shows why the comparison inherits the same difficulty.

Chapter 8

Back-Translation and the Circularity of Certification

A natural proposal for checking a formalization is to translate the formal statement back into prose and compare the result with the original. If the two agree, the argument goes, nothing was lost. The proposal is attractive because the comparison is between two objects in the same language and because the reverse translation can be automated. This chapter explains why the comparison creates a further correspondence obligation instead of discharging the first, proves that round-trip agreement does not entail fidelity (RV-007), and identifies conditions under which translations can nevertheless provide useful evidence.

8.1 The round trip

Let $D \subseteq \mathcal{L}_N$ be a set of specifications, $f : D \rightarrow \mathcal{L}_F$ a forward translation, and $b : \mathcal{L}_F \rightarrow \mathcal{L}_N$ a reverse translation. Let $\approx$ be a relation of agreement on $\mathcal{L}_N$, which may be string identity, equality after normalization, or a judgment of paraphrase.

Definition 8.1 (Round-trip agreement and fidelity). *The pair (f, b) has round-trip agreement on D if $b(f(N)) \approx N$ for every $N \in D$. The forward translation f is faithful on D if $\llbracket f(N) \rrbracket_{\mathcal{I}} \equiv_{\mathcal{I}} \llbracket N \rrbracket_{\mathcal{I}}$ for every $N \in D$, where the right-hand side is the meaning of N in a fixed context. The reverse translation b is faithful on a set $E \subseteq \mathcal{L}_F$ if $\llbracket b(F) \rrbracket_{\mathcal{I}} \equiv_{\mathcal{I}} \llbracket F \rrbracket_{\mathcal{I}}$ for every $F \in E$.*

Round-trip agreement concerns the surface of N and the surface of $b(f(N))$. Fidelity concerns the meaning. The question is what the first says about the second.

8.2 Agreement does not entail fidelity: RV-007

The observation that follows is elementary. It is stated with two parts, one showing that round-trip agreement is very weak as a property of f alone, and one showing that it is compatible with failure of fidelity in the symmetric case where both translators share the failure.

Proposition 8.2 (Round-trip agreement constrains only injectivity). *Take $\approx$ to be identity. (i) If (f, b) has round-trip agreement on D then f is injective on D . (ii) Conversely, if f is*

injective on D then there is a b such that (f, b) has round-trip agreement on D , whatever the meaning of the formal sentences $f(N)$.

Proof. (i) If $f(N_1) = f(N_2)$ then $N_1 = b(f(N_1)) = b(f(N_2)) = N_2$. (ii) Define b on $f(D)$ by $b(f(N)) = N$, which is well defined by injectivity, and arbitrarily elsewhere. $\square$

Part (ii) is the point. For the pair to pass the round trip it suffices that f lose no surface information, and this is independent of whether f preserves meaning. If one is free to choose b after seeing f , round-trip agreement is satisfiable by every injective translation, faithful or not, provided the reverse translation may be chosen after the forward one. Round-trip agreement therefore carries information about fidelity only to the extent that b is constrained independently of f .

Theorem 8.3 (Round-trip agreement does not entail fidelity, RV-007). *Suppose D contains two specifications N_0, N_1 with $\llbracket N_0 \rrbracket_{\mathcal{I}} \not\equiv_{\mathcal{I}} \llbracket N_1 \rrbracket_{\mathcal{I}}$, and suppose there is a faithful injective translation $f_0 : D \rightarrow \mathcal{L}_F$ with a left inverse b_0 . All denotations of members of D are defined, and $\equiv_{\mathcal{I}}$ is an equivalence relation on defined denotations. Then there are translations f, b such that (f, b) has round-trip agreement on D , f is not faithful on D , and b is not faithful on $f(D)$. The relation $\equiv_{\mathcal{I}}$ is assumed to be an equivalence relation.*

Proof. Write $F_i = f_0(N_i)$. Faithfulness of f_0 gives $\llbracket F_i \rrbracket_{\mathcal{I}} \equiv_{\mathcal{I}} \llbracket N_i \rrbracket_{\mathcal{I}}$, so F_0 and F_1 are inequivalent, since $\equiv_{\mathcal{I}}$ is transitive and symmetric. Let $\rho : \mathcal{L}_F \rightarrow \mathcal{L}_F$ be the transposition exchanging F_0 and F_1 and fixing every other sentence. It is injective with $\rho^{-1} = \rho$. Define $f = \rho \circ f_0$ and $b = b_0 \circ \rho$ on $\mathcal{L}_F$, where b_0 is extended arbitrarily outside $f_0(D)$. For $N \in D$, $b(f(N)) = b_0(\rho(f_0(N))) = b_0(f_0(N)) = N$, so round-trip agreement holds, in fact as identity. Now $f(N_0) = F_1$ and $\llbracket F_1 \rrbracket_{\mathcal{I}} \equiv_{\mathcal{I}} \llbracket N_1 \rrbracket_{\mathcal{I}} \not\equiv_{\mathcal{I}} \llbracket N_0 \rrbracket_{\mathcal{I}}$, so f is not faithful at N_0 . Also $b(f(N_0)) = N_0$ while $\llbracket F_1 \rrbracket_{\mathcal{I}} \not\equiv_{\mathcal{I}} \llbracket N_0 \rrbracket_{\mathcal{I}} = \llbracket b(F_1) \rrbracket_{\mathcal{I}}$, so b is not faithful at $F_1 \in f(D)$. $\square$

A concrete instance: let $D = \{N_0, N_1\}$ with N_0 the sentence “ n is even” and N_1 the sentence “ n is odd” for a natural number n , with F_0, F_1 the corresponding formal predicates. The pair that translates “even” to the formal odd predicate and back again passes the round trip exactly and is wrong in both directions. The construction is a coordinated misreading: ρ is a systematic reinterpretation and b undoes it at the surface, so nothing in the round trip detects it.

The theorem is deliberately weak, since it exhibits a pair under a mild existence hypothesis and not a rate. Its content is that round-trip agreement is not a sufficient condition and that no argument of the form “the round trip returns the original, so nothing was lost” is valid without further premises. A case from Chapter 9 shows the same phenomenon in a form that is not artificial, although it is not an instance of the construction above.

Example 8.4 (The silent default under the round trip). Let N_e define n_e as the least n with $B_e(n)$, and let $f(N_e) = F_e$ be the formalization using the library infimum. Let b render the infimum as “the least n such that $B_e(n)$.” Then $b(f(N_e))$ is, to the surface, N_e , for every e . The round trip agrees for all e including those with $e \notin d$, where F_e assigns the default $n_e = 0$ and the specification is undefined. By Theorem 16.6, under the convention stated there, the statement correspondence $C(V_e)$ holds exactly for $e \in d$, a Σ_2^0 -complete set.

The passage from a presupposing description to a total function is not an injective self-map of $\mathcal{L}_F$, so the example does not instantiate the theorem’s construction. It exhibits the same structure: both directions use the same reading of “the least n such that” because both are

generated by the same convention, and the agreement is a correct fact about strings and not evidence about meanings.

8.3 Why the second translation is a second obligation

The comparison $b(f(N)) \approx N$ is a claim of the form that a certain prose sentence says the same as another. If $\approx$ is string identity, the claim is decidable and uninformative, by Proposition 8.2. If $\approx$ is a judgment of paraphrase, the claim is a claim about meaning, and meaning claims are what the statement obligation $O_{\text{statement}}$ was about in the first place. The reverse translation b then introduces its own obligation, that b is faithful on the relevant formal sentences, which is a statement-correspondence obligation of the same kind with the roles of prose and formal language exchanged. In the vocabulary of Appendix A, the back-translation is a transition whose certificate has to be supplied by an authority, and the authority for it is again a reader. The residue does not shrink: one obligation has been replaced by an obligation that has the same form and, so far as the record shows, the same authority-dependence.

If the authority for b is the same system that produced f , the certificate depends on the thing it certifies. A dependency graph in which the fidelity of f is supported by agreement with b and the fidelity of b is supported by agreement with f has a cycle, and Definition A.11 excludes it. If the authority is a different system, its independence is itself a claim to be supported, and the next section says what is required of it.

8.4 Independent translations

A weaker use of the second translation is as evidence about the first, based on the idea that errors of independent translators are unlikely to coincide. Let E_1, E_2 denote the events that translations f_1, f_2 of the same specification are unfaithful, with probabilities e_1, e_2 . Say the translations *agree* if the resulting formal sentences are equivalent.

Proposition 8.5 (Agreement and coincident error). *Fix a probability distribution over specifications and over the randomness of the translators, define “wrong” for a translation as unfaithful, and say the two translations agree when their outputs are equivalent under $\equiv_{\mathcal{I}}$. Faithful outputs are equivalent to the meaning of the specification, hence to each other, so two faithful translations agree. Then*

$$(i) \Pr(\text{agree and wrong}) \leq \Pr(E_1 \cap E_2) \leq \min(e_1, e_2);$$

$$(ii) \text{ if } E_1 \text{ and } E_2 \text{ are independent, } \Pr(\text{agree and wrong}) \leq e_1 e_2 \text{ and}$$

$$\Pr(\text{wrong} \mid \text{agree}) \leq \frac{e_1 e_2}{(1 - e_1)(1 - e_2)};$$

$$(iii) \text{ the chain in (i) holds with equality, } \Pr(\text{agree and wrong}) = \Pr(E_1 \cap E_2) = \min(e_1, e_2), \\ \text{when the errors coincide, } E_1 = E_2, \text{ and the wrong sentences are equivalent, provided} \\ e_1 = e_2 \text{ or one error event contains the other.}$$

Proof. (i) To agree on a wrong sentence, both outputs must be equivalent to a sentence that is not a faithful rendering, so both translations are unfaithful; the event is contained in $E_1 \cap E_2$, whose probability is at most each of e_1, e_2 . (ii) Under independence $\Pr(E_1 \cap E_2) = e_1 e_2$. The

probability of agreement is at least that of both being faithful, $(1 - e_1)(1 - e_2)$, since faithful translations agree by assumption. The conditional probability is the quotient. (iii) If $E_1 = E_2$ and the wrong sentences are equivalent then agreement and error both occur whenever E_1 does, and $\Pr = e_1$, which equals $\min(e_1, e_2)$ if $e_1 = e_2$. $\square$

For $e_1 = e_2 = 0.1$, independence gives a bound of about 0.012 on the probability of error given agreement, while perfectly correlated errors give an error probability of 0.1 on the very same agreement events. The numbers are illustrative of the structure and carry no empirical claim. The hypothesis that carries the weight is independence, and independence in this setting is not plausible by default: translators trained on overlapping data, conditioned on the same prompt, using the same library convention, or exposed to the same example of a pattern, can fail on the same specifications, and Example 8.4 is a case in which the failure is attributable to the library, which every translator into that library shares. A claim of independence is an obligation to be supported, for instance by showing that the translators differ in method, data, and target conventions, and by estimating the correlation of errors on a held-out sample with trusted labels.

8.5 Controlled vocabularies and certified glossaries

There is a regime in which a round trip, or a comparison of translations, does provide evidence, and its conditions are definite. Suppose that the prose is restricted to a controlled language $\mathcal{L}_C \subseteq \mathcal{L}_N$ built compositionally from a finite glossary G of phrases, each with an assigned formal definition, and suppose the formal sentences are built from the same definitions.

Definition 8.6 (Certified glossary). *A glossary is a finite set of pairs $(w, \gamma(w))$ of a phrase w and a formal term $\gamma(w)$. It is correct in the context c if $\llbracket \gamma(w) \rrbracket_{\mathcal{I}} \equiv_{\mathcal{I}}$ the intended meaning of w in c for every pair. It is certified if for each pair a certificate of that equivalence, accepted by an identified authority, is warranted in the ledger. A certificate evidences correctness only to the extent that it is sound, which is a hypothesis on the authority and not a consequence of its being warranted.*

Proposition 8.7 (Fidelity on a compositional fragment). *Let $\mathcal{L}_C \subseteq \mathcal{L}_N$ be generated from G by a finite set of constructors with unique parsing, suppose that $\equiv_{\mathcal{I}}$ is a congruence for the meaning operations of those constructors, that the intended meaning of a phrase w in c is $\llbracket w \rrbracket_{\mathcal{I}}$, and suppose that both $\llbracket \cdot \rrbracket_{\mathcal{I}}$ on $\mathcal{L}_C$ and $\llbracket \cdot \rrbracket_{\mathcal{I}}$ on the formal terms they translate to are compositional with respect to those constructors, with the formal constructors interpreted by the same operations on meanings. If G is correct in c and f is defined by replacing each phrase w by $\gamma(w)$ and each constructor by its formal counterpart, then f is faithful on $\mathcal{L}_C$ in c .*

Proof. By induction on the construction of $N \in \mathcal{L}_C$. For a glossary phrase, faithfulness is the correctness of G . For a compound $N = \kappa(N_1, \dots, N_k)$, $f(N) = \kappa^F(f(N_1), \dots, f(N_k))$ and by compositionality of both semantics $\llbracket f(N) \rrbracket_{\mathcal{I}}$ is the result of applying the meaning operation of κ to $\llbracket f(N_i) \rrbracket_{\mathcal{I}}$, which by the induction hypothesis are equivalent to $\llbracket N_i \rrbracket_N$. The result is equivalent to $\llbracket N \rrbracket_N$. $\square$

The proposition says where the evidence comes from: from the correctness of the glossary, evidenced by sound certificates, and from the compositionality of the fragment, not from the round trip. Uniqueness of parsing makes f a function, and the congruence property is what

lets the induction pass through the constructors; for the intensional choices of $\equiv_{\mathcal{I}}$ permitted in Appendix A it can fail. In such a fragment the round trip can serve as a check that the translators implement the glossary as written, a check on implementation errors and not on meaning. The cost of the regime is that everything outside $\mathcal{L}_C$ is out of scope, that the glossary certificates are themselves statement-correspondence obligations to be discharged by an authority, and that, as Chapter 5 observed, readings depend on context and practice, so that the compositional fragments are only a part of mathematical prose. Controlled languages are therefore a way of purchasing certified fidelity on a fragment, at the price of the fragment. This is the same trade as that of conservative certification in Chapter 11.

8.6 Certified intermediate representations

A related option is an intermediate representation between prose and the target formalism, designed so that each of the two translations is simpler and so that the intermediate form can be inspected by a reader. The structure is a chain $N \rightarrow M \rightarrow F$, and by Theorem A.17 the cumulative context entails the originating content when both steps are admissible and preserving; the content is transported when every premise of the cumulative context is established, and not otherwise guaranteed. The intermediate form helps if inspection of M is cheaper or more reliable than inspection of F , which is a claim about the authorities and not about the logic. It does not remove the obligation: it moves it into two smaller obligations, each of which has an authority.

8.7 Summary of the chapter

The back-translation is not a verifier of the forward translation. Its agreement with the original is satisfiable by every injective forward map when the reverse map may be chosen afterwards (Proposition 8.2), is compatible with coordinated failure (Theorem 8.3, Example 8.4), and is evidence only under premises that are themselves obligations: independence of translators (Proposition 8.5), or a certified glossary and a compositional fragment (Proposition 8.7). In each case the evidence is conditional on a certificate whose authority is outside the round trip. This is the sense in which certification by translation is circular: the comparison that would settle fidelity can be run only after fidelity of the comparator has been settled.

Part III

The Computational Limits of Interpretation

Chapter 9

Undecidability and Hidden Presuppositions

This part turns from the logic of verification to its computational limits. The question is whether the gap between a verified formal statement and the specification it was meant to express can be closed by a procedure, and the answer developed over this chapter and the next is that on unrestricted classes of specifications it cannot. The argument begins with a small example in which the obstruction is visible without any machinery. A textbook identity so trivial that a proof assistant closes it by ring normalization presupposes that a certain quantity exists, and whether that quantity exists is a question of unbounded arithmetical difficulty. The example is Example 4.1 of Bastounis, Circelli, and Hansen [3]. The reconstruction below follows their Section 4, and the statements are theirs unless attributed otherwise.

9.1 The example

Fix an integer $k \geq 1$ and a computable enumeration $(p_e)_{e \in \mathbb{N}}$ of integer-coefficient polynomials in $k + 1$ variables. For each e and $n \in \mathbb{N}$ let $B_e(n)$ be the predicate

$$B_e(n) \iff \forall (x_1, \dots, x_k) \in \mathbb{N}^k \quad p_e(n, x_1, \dots, x_k) \neq 0.$$

When the set $S_e = \{n : B_e(n)\}$ is nonempty let n_e denote its least element, and put $r_e = 1/(n_e + 1)$. A textbook author who has fixed the enumeration, defined n_e as the least n for which $p_e(n, x_1, \dots, x_k) = 0$ has no solution in natural numbers, and defined r_e as above, may then assert that the identity

$$(r_e + 1)^2 = r_e^2 + 2r_e + 1$$

is obvious. As a statement about numbers it is obvious: it is the expansion of a square, valid for every value of r_e . What the sentence does not make explicit is the presupposition that r_e is a well-defined rational number. That depends on whether n_e exists, and so on whether S_e is empty. The hidden question is thus a question about S_e , and the apparent triviality of the identity gives no information about it.

The structure of the example is worth stating abstractly, because it recurs. A mathematical sentence has a part that carries its claim, here the identity, and a part that introduces the objects the claim is about, here the definition of n_e . The claim is verified or verifiable independently of the introduction, since the identity holds for any value. The introduction is where fidelity

is decided, and it can be very hard to check: for well-definedness the difficulty rises with the level of the arithmetical hierarchy.

9.2 The silent default

Section 4.2 of the source makes the danger concrete. Take $k = 2$ and $p_e(n, x_1, x_2) = x_1 + x_2 - n$. For every n the pair $(x_1, x_2) = (n, 0)$ is a solution, so $B_e(n)$ fails for all n , the set S_e is empty, and r_e is undefined. A formalizer that translates the textbook sentence into Lean may nevertheless define $\mathbf{n_e}$ as the infimum of the set $\{\mathbf{n} \mid \mathbf{B_e\ n}\}$. Lean accepts the definition, and the tactic `ring` closes the goal. The reason is that the library's `Nat.sInf` returns 0 on the empty set. The formalization has silently set $n_e = 0$ and $r_e = 1$, and the identity holds of $r_e = 1$ as it holds of any value. The translation compiles, the proof is accepted, and the statement that was verified is about an object that the original text never defined.

This is a case of the phenomenon that gives the monograph its title. Every judgment the kernel is asked to make is answered correctly. The failure lies entirely in the correspondence between the formal term and the quantity it was meant to denote, and kernel acceptance cannot reveal it. Remark 4.2 of the source draws the consequence for translation. A semantics-preserving autoformalizer must refrain from translating when r_e is undefined, and to refrain correctly it must know, for the given e , whether S_e is empty.

9.3 Three ways to represent a partial definition

The example separates three ways in which a formalization can treat a definition whose well-definedness is not settled, and the distinction is the one that Part V makes precise.

The first is the *total default*. The formal language supplies a total function that returns a conventional value outside the domain, as `Nat.sInf` does. The resulting term is always defined and the translation always compiles. It is faithful where the original is defined and unfaithful elsewhere, and nothing in the artifact marks which case obtains. The defect is silent, and a total default is a form of assigning a value to an undefined object.

The second is the *partial representation*. The formal object is given an explicit type that records partiality, for example an optional value or a subtype carrying a proof of nonemptiness. Statements about r_e must then either handle the undefined case or be conditional on the proof. The partiality of the original is represented as partiality and not erased.

The third is the *hypothesis-carrying representation*. The formal statement is conditional: for every e , if S_e is nonempty then the identity holds for $r_e = 1/(n_e + 1)$. The hypothesis of nonemptiness is an explicit assumption. In the vocabulary of Part V it is an assumption obligation, recorded in the ledger at the moment of translation and remaining in the residue until it is discharged. Such a translation is preserving in the sense of Chapter 17, since the assumption is tracked and the original sentence, which presupposed existence, is entailed by the conditional together with the assumption. The residue contains exactly the question the original text left unasked.

The monograph's position is not that the third representation is always available or always desirable. It is that the first is unfaithful in a manner the kernel cannot detect, and that the other two are faithful because they leave the open question visible.

9.4 Acceptance loops and the elementary analogue

Example 4.3 of the source asks what a guess-and-check loop does with such an example. A loop that generates a candidate translation, submits it to the proof assistant, and halts when the assistant accepts will halt on the silent-default translation, since that translation is accepted. The loop therefore terminates exactly on the mistranslation in this instance. The point generalizes, and Chapter 14 treats it as the mechanism by which acceptance loops convert unfaithful translations into reported successes: acceptance is a test of the formal artifact and carries no information about the artifact's relation to the source.

Example 4.4 gives an elementary analogue in which no computability theory is involved. Replacing “odd” by “prime” in the sentence “there are infinitely many odd numbers” yields a different true sentence, and the altered sentence is as formalizable and as provable as the original. A translation that makes this substitution compiles equally well. The example is useful as a calibration. The hidden-existence construction is not required to exhibit the phenomenon, which is already present in the simplest substitutions, and what the construction adds is a demonstration that detecting the phenomenon is in general beyond any bounded computational level. That is the subject of the next chapter.

9.5 What the example does and does not show

The example shows that a trivial identity can conceal a question about the existence of an object appearing in it. It shows that a formal library's totalizing conventions can erase that question without trace. It does not show that mathematical texts as they occur in practice contain such constructions with any frequency, nor that every definition of the form “the least n such that ...” is dangerous. Many such definitions are accompanied by a proof of existence or sit in a context where existence is obvious. The example is a worst case chosen to expose the structure, and the claims of the next chapter are about the structure and not about frequency.

Chapter 10

The Solvability Complexity of Semantic Preservation

The previous chapter exhibited a definition whose well-definedness could not be read off from the text. This chapter states precisely how hard that question is. The results are those of Appendix A of Bastounis, Circelli, and Hansen [3], and the aim is to reconstruct them with their exact hypotheses, to say what their proofs use, and to separate them from the broader interpretations that are tempting and are not established. The monograph's own contribution here is the formulation of the statement called RV-005, which transfers the source's results from the construction to a stated class of specifications, and of the companion statement RV-006, which records what the results leave open.

10.1 Prerequisites

The arithmetical hierarchy classifies sets of natural numbers by the quantifier complexity of their definitions. A set is Σ_1^0 when it is recursively enumerable, which is to say of the form $\{e : \exists x R(e, x)\}$ with R decidable. It is Σ_l^0 when it has the form $\{e : \exists z_1 \forall z_2 \cdots Q z_l R(e, z_1, \dots, z_l)\}$ with R decidable and the quantifiers alternating, beginning with an existential. The classes Π_l^0 are the complements. Post's theorem [20] identifies these classes with computability relative to iterated Turing jumps: a set is Σ_l^0 exactly when it is recursively enumerable relative to $\emptyset^{(l-1)}$, and it is Δ_{l+1}^0 exactly when it is computable relative to $\emptyset^{(l)}$. A set is Σ_l^0 -complete under many-one reductions when it is Σ_l^0 and every Σ_l^0 set reduces to it. A Σ_l^0 -complete set is not in Δ_l^0 , so it is not computable even relative to $\emptyset^{(l-1)}$. The Halting set is Σ_1^0 -complete.

The Solvability Complexity Index [9] measures, for a computational problem, the least number of successive limits needed to compute it. A problem has $\text{SCI} = 1$ when it is the limit of a sequence of computable approximations, and in general $\text{SCI} = l$ when l nested limits suffice and no smaller number does. The version used in the source is the arithmetic one, written SCI_A , in which the approximating algorithms are arithmetic. For sets of natural numbers the relevant fact, recorded by the source, is that $\text{SCI}_A(\Xi) = l$ corresponds to membership of the associated decision problem in $\Delta_{l+1}^A \setminus \Delta_l^A$. A problem with no finite index has $\text{SCI} = \infty$. The Halting problem has $\text{SCI} = 1$. The source's definitions of towers of algorithms are not reproduced here, and the statements below use only the equivalence with the arithmetical hierarchy that the source records.

10.2 The least-number families

Fix $k \geq 9$ and a computable enumeration $(p_e)_{e \in \mathbb{N}}$ of all integer-coefficient polynomials in $k + 1$ variables. With B_e as in Chapter 9, put

$$d = \{e \in \mathbb{N} : \exists n B_e(n)\}.$$

The set d is the set of indices for which n_e , and therefore r_e , is well defined. It is Σ_2^0 by inspection of its definition, as $\exists n \forall (x_1, \dots, x_k) p_e(n, x_1, \dots, x_k) \neq 0$.

Theorem 10.1 (Theorem A.3 of the source). *For $k \geq 9$ the set d is Σ_2^0 -complete under many-one reductions. Consequently no procedure can always decide whether n_e is well defined, even with an oracle for the Halting problem.*

The structure of the proof is as follows, with the sources of each ingredient identified. The set $\text{NTOT} = \{e : \exists n P_e(n) \uparrow\}$, the indices of partial computable functions that diverge on some input, is Σ_2^0 -complete. For each e the relation $P_e(n) \downarrow$ is recursively enumerable in (e, n) . By the DPRM theorem in the form that reduces the number of unknowns to nine, due to Jones, every recursively enumerable relation is Diophantine with nine unknowns, so there is a polynomial q with $P_e(n) \downarrow \iff \exists (x_1, \dots, x_9) q(e, n, x_1, \dots, x_9) = 0$. Then $P_e(n) \uparrow \iff \forall (x_1, \dots, x_9) q(e, n, \mathbf{x}) \neq 0$, which is B for the polynomial $q(e, \cdot, \cdot)$. Since the enumeration contains every integer-coefficient polynomial, and padding with dummy unknowns handles $k > 9$, a computable function f with $p_{f(e)} = q(e, \cdot, \cdot)$ is available, and $e \in \text{NTOT} \iff f(e) \in d$. This is a many-one reduction. The remaining ingredient, that NTOT is Σ_2^0 -complete, is Post's theorem together with the standard completeness of TOT for Π_2^0 .

A remark on the count of unknowns removes a possible misreading. The bound $k \geq 9$ refers to the unknowns $x_1, \dots, x_k$ over which the universal quantifier ranges, and these are in addition to the parameter n . The polynomials therefore have $k + 1$ variables. The bound is not a count of auxiliary variables introduced for convenience, and it arises from the nine-unknown form of the DPRM theorem.

The theorem extends to every finite level. For $l \geq 2$ and $k \geq 9$ let $B_e^{(l)}(n)$ be defined by an alternating prefix of quantifiers $Q_2 z_2 \cdots Q_{l-1} z_{l-1}$, with $\exists$ at odd indices and $\forall$ at even indices, followed by $\forall (x_1, \dots, x_k) p_e \neq 0$ when l is even and by $\exists (x_1, \dots, x_k) p_e = 0$ when l is odd, in an enumeration of polynomials in $l - 1 + k$ variables. These are the source's displays (A.4a) and (A.4b). Set $d_l = \{e : \exists n B_e^{(l)}(n)\}$.

Theorem 10.2 (Theorem A.4 of the source). *For every $l \geq 2$ and $k \geq 9$ the set d_l is Σ_l^0 -complete under many-one reductions. No algorithm decides well-definedness of the corresponding least-number term, even relative to $\emptyset^{(l-1)}$. The case $l = 2$ recovers $d_2 = d$, and d_l has the Turing degree of $\emptyset^{(l)}$.*

Corollary 10.3 (Corollary A.5 of the source). *For every $l \geq 2$, $\text{SCI}_A(\Xi_{d_l}) = l$. Equivalently, $d_l \in \Delta_{l+1}^A \setminus \Delta_l^A$.*

Section A.2.1 of the source then draws the global conclusion. The problem of answering the emptiness question for all e and all $l \geq 2$ has $\text{SCI} = \infty$, since for each fixed l it contains d_l as a subproblem of index l . The source makes the slogan precise in this way: every trustworthy autoformalizer that always outputs either a faithful translation or the message that the statement cannot be translated must solve problems harder than any problem of finite SCI . This is also the claim made in the abstract of the source.

Remark A.6 of the source records that the bound $k \geq 9$ reflects the state of the literature on the DPRM theorem. A recent refinement by Sun lowers the corresponding bound from $k \geq 11$ to $k \geq 10$ for integer unknowns, while for natural-number unknowns the best known bound for Σ_2^0 -completeness of d is $k \geq 9$. The source does not claim decidability for fewer unknowns. The remark matters for the monograph because it blocks an inference that would otherwise be tempting, namely that the problem becomes decidable below the threshold. The absence of a proof of completeness at $k = 8$ is not a proof of decidability at $k = 8$.

10.3 From the construction to a class of specifications

The theorems above concern a constructed family of sets. The question that matters for verification is whether they constrain a given class of specifications. The monograph's formulation makes the transfer explicit.

Definition 10.4 (Embedding of the least-number constructions). *A class $\mathcal{S}$ of specifications effectively embeds the least-number constructions at level l (with unknowns $k \geq 9$) when there is a computable map $e \mapsto N_e^{(l)} \in \mathcal{S}$ such that $N_e^{(l)}$ introduces the term $n_e^{(l)} = \min\{n : B_e^{(l)}(n)\}$ and $N_e^{(l)}$ is well defined if and only if $e \in d_l$.*

An *autoformalizer with refusal* for $\mathcal{S}$ is a total function T on $\mathcal{S}$ whose value is either a formal proposition or the symbol $\perp$, and it is *trustworthy* when it satisfies two conditions: if $T(N) \neq \perp$ then N is well defined and $T(N)$ corresponds to N , and if $T(N) = \perp$ then N is not well defined. It never mistranslates and never refuses a translatable specification.

Theorem 10.5 (RV-005). *Let $\mathcal{S}$ effectively embed the least-number constructions at level $l \geq 2$ with $k \geq 9$. Then well-definedness of the defining term is Σ_l^0 -hard on $\mathcal{S}$, and no total computable procedure decides it. In particular no trustworthy computable autoformalizer with refusal exists for $\mathcal{S}$, and none exists even with an oracle for $\emptyset^{(l-1)}$. If $\mathcal{S}$ embeds the constructions at every level $l \geq 2$, then the combined problem has $\text{SCI} = \infty$.*

Proof. By the embedding, $e \in d_l \iff N_e^{(l)}$ is well defined, and $e \mapsto N_e^{(l)}$ is computable. Suppose T is a trustworthy autoformalizer computable relative to $\emptyset^{(l-1)}$. Then $e \in d_l \iff T(N_e^{(l)}) \neq \perp$: if $e \in d_l$ the specification is well defined, so by trustworthiness T does not refuse, and if $e \notin d_l$ the specification is not well defined, so T does not return a translation, because a returned translation requires well-definedness. Hence d_l would be computable relative to $\emptyset^{(l-1)}$, contradicting $d_l \notin \Delta_l^0$ from Theorem 10.2. The hardness statement is the reduction $e \mapsto N_e^{(l)}$ composed with the completeness of d_l . The case of all levels follows because each d_l is a subproblem, and Corollary 10.3 gives index l for each. $\square$

The theorem is also conditional on Theorem 10.2, which is the source's result and is not reproved here, and the passage to levels $l \geq 2$ rests on it. The combined claim $\text{SCI} = \infty$ follows from the strictness of the arithmetical hierarchy, since a procedure deciding all levels would decide each d_l relative to $\emptyset^{(l-1)}$. The theorem proves hardness. Completeness for the class would need membership in Σ_l^0 for the whole class, which is not shown. The theorem is conditional on the embedding, and the condition must be proved class by class. The source establishes strong hardness for constructed families of presupposition-resolution problems. It does not follow that every reasonable formulation of semantic correspondence has the same complexity, and a new

theorem that claims hardness for a particular class must identify the class and exhibit the embedding. The chapter's claim is therefore about classes rich enough to define least-number terms over arbitrary Diophantine conditions, which includes the specifications of arithmetic and analysis in ordinary mathematical language but excludes many restricted formalisms.

The trustworthiness conditions are also a matter of convention, as the second condition shows. It treats a specification with an undefined term as untranslatable. The convention is the source's, and Chapter 16 makes the dependence explicit where the theorem of that chapter relies on it.

10.4 Restriction and the open boundary

The hardness of the unrestricted problem does not imply hardness for every restricted class of mathematical texts. This is the statement RV-006, and it is a record of a limitation and not a theorem.

Remark 10.6 (RV-006). The source establishes no decidability result for fewer than nine unknowns, and RV-005 does not apply to a class that fails to embed the least-number constructions. For any proposed restricted correspondence language, a decidability or hardness claim requires its own proof.

Exceeding every fixed finite level of a hierarchy over an unrestricted class says nothing about a class that cannot express the quantifier patterns of the construction. A restricted language in which every definition carries a proof of existence, or in which defining terms range over a bounded domain, may admit a decidable fidelity check, and the constructive proposals of Part VII are directed at such languages. Two cautions apply. The first is that restriction purchases decidability at the price of expressiveness, and the expressiveness lost may include the texts of greatest interest. The second is that the partial result is not an assurance in the other direction: a class that is not known to embed the constructions is not thereby known to be tractable.

10.5 Consequences for the theory of residue

The results have a consequence for the theory of Part V that is easily misstated. They do not say that semantic obligations cannot be handled. They say that no total procedure can, on unrestricted classes, discharge all of them or even decide which of them are dischargeable. The obligations that cannot be settled by computation can still be recorded, preserved through transformations, and reported honestly. That is what the preservation theory of Chapter 17 is for. A pipeline that cannot be trustworthy in the sense of the theorem above can still be sound in a weaker and more useful sense, namely that it never reports as settled an obligation it has only preserved. The computability barrier therefore motivates the shift from certification of correspondence to bounded certification with an explicit residue, and it fixes the sense in which that shift is forced and not merely convenient.

Chapter 11

Impossibility Without Futility

The results of the preceding chapter might be read as a counsel of despair. If well-definedness of a defining term is Σ_l^0 -hard for every $l \geq 2$ on unrestricted specification classes, by reduction from the complete sets d_l , one may conclude that semantic checking is hopeless and that every verification must be accompanied by an irreducible act of faith. That conclusion does not follow, and the present chapter explains why. The existence of undecidable instances limits what a single total procedure can do. It leaves open a large space of procedures that are sound but incomplete, of restricted languages in which the barrier does not arise, and of reporting conventions in which uncertainty is a permitted output. The chapter develops this space and states the design principle that it supports.

11.1 Unrestricted hardness and useful partial procedures

A procedure for checking correspondence can fail in two ways. It can accept an instance whose specification and formalization do not correspond, which is a false acceptance. It can fail to accept an instance that does correspond, which is a missed acceptance. The theorems of Chapter 10 show that on unrestricted classes no computable procedure that always answers can avoid both. They do not show that a procedure must commit either failure at will. A procedure that is permitted to abstain can avoid false acceptance entirely at the price of missed acceptances, and the question of practical interest is how many acceptances can be recovered.

Definition 11.1 (Sound certifier). *Let $\mathcal{F}$ be a class of failure types, each a set of instances at which correspondence fails in a specified way. A certifier is a function κ from instances to $\{\text{accept}, \text{reject}, \text{abstain}\}$. It is sound with respect to $\mathcal{F}$ when $\kappa(V) = \text{accept}$ implies that V lies in no member of $\mathcal{F}$. It is sound when acceptance implies correspondence. It is incomplete when it abstains on some corresponding instance.*

Proposition 11.2 (RV-011). *(i) The certifier that always abstains is sound. (ii) If κ_1, κ_2 are sound with respect to $\mathcal{F}_1, \mathcal{F}_2$ respectively, then the certifier that accepts when both accept and abstains otherwise is sound with respect to $\mathcal{F}_1 \cup \mathcal{F}_2$. (iii) If κ is sound with respect to $\mathcal{F}$ and κ' is obtained from κ by replacing some acceptances or rejections by abstentions, and by no other change, then κ' is sound with respect to $\mathcal{F}$.*

Proof. (i) Acceptance never occurs. (ii) Acceptance by the composite requires acceptance by both, so the instance lies in no member of $\mathcal{F}_1$ and in no member of $\mathcal{F}_2$. (iii) Acceptances of κ' are acceptances of κ . $\square$

The proposition is elementary and its role is to name the structure. Soundness holds by construction, in the sense that it is a property of how acceptance is gated, and the mathematical substance lies in the certificate languages that make non-trivial acceptance possible. A certifier that abstains always is useless, and a certifier that accepts when a checked witness is supplied is useful in exact proportion to the witnesses it can check.

11.2 What soundness can and cannot recover

The hidden-existence family of Chapter 9 shows both sides. Consider the indices e for which the term n_e is well defined, the set d . A certificate that $e \in d$ can be given by supplying a number n together with a proof that $B_e(n)$ holds, that is, that $p_e(n, x_1, \dots, x_k) \neq 0$ for every choice of the unknowns, and for each $j < n$ a tuple $(x_1, \dots, x_k)$ with $p_e(j, x_1, \dots, x_k) = 0$ showing that $B_e(j)$ fails. The second part is checked by evaluation. The first part is a mathematical proof in some formal system. A certifier that accepts e exactly when such a certificate is supplied and checked is sound with respect to false acceptance, since a checked certificate establishes that n is the least element.

Proposition 11.3 (Incompleteness of every effective certificate scheme). *Let $\mathcal{T}$ be a recursively axiomatized theory sound for arithmetic, and let $\kappa_{\mathcal{T}}$ accept e when $\mathcal{T}$ proves $\exists n B_e(n)$. Then the set of accepted indices is a recursively enumerable proper subset of d . More generally, no sound certification scheme whose accepted set is recursively enumerable accepts all of d .*

Proof. The accepted set is recursively enumerable, by enumeration of the theorems of $\mathcal{T}$. It is contained in d by soundness. The set d is Σ_2^0 -complete (Theorem 10.1). A recursively enumerable set is Σ_1^0 , and a Σ_2^0 -complete set is not Σ_1^0 , since otherwise every Σ_2^0 set would be Σ_1^0 by reduction, contradicting the strictness of the arithmetical hierarchy. Hence the accepted set is a proper subset of d . $\square$

The proposition fixes what is lost. Whatever effective certificate language is adopted, there are indices $e \in d$ that it cannot certify, and for such indices the certifier abstains. The abstention is not an error and carries no information about whether $e \in d$. The proposition also fixes what is not lost. The certified set is nonempty and in practice large, because for the specifications that occur in actual mathematical work the existence of the defined object is usually provable in a standard theory. The barrier is a barrier to uniform completeness. It is silent about the frequency of the instances on which a given theory can succeed, a point on which the result of Chapter 10 on restrictions is explicit.

11.3 Restricted languages and domain-specific certificates

A second source of useful procedures is restriction of the specification language. A language in which every defining term carries a proof of existence is the clearest example. In such a language the specification of Example 4.1 would be rejected as ill formed unless the author supplies the existence proof, and the hidden question is moved from the semantic interface, where it is undecidable in general, to the syntactic interface of the language, where it is checked at submission. A proof-carrying definition has the form of the hypothesis-carrying or subtype representations of Chapter 9, and the checking of the proof is an ordinary kernel operation. The price is expressiveness: the author must have the proof.

Domain-specific semantic certificates are a third source. A certificate language tailored to a class of statements, such as bounds in a given family of function spaces, may check correspondence by comparing a normal form of the specification with a normal form of the formalization. Such checks are sound relative to the correctness of the normal form, they are incomplete, and they are decidable. They can detect classes of failure that Chapter 12 documents. A certificate language for norm estimates that records the regularity count of each bound would flag the passage from $m + 4$ to $m + 5$ derivatives mechanically, as an inequality between recorded integers. The failure is not a hidden semantic failure once it is expressed in a vocabulary the checker understands, which is the point of designing the vocabulary.

11.4 Human adjudication and conservative abstention

Some obligations cannot be settled by either procedure, and human adjudication is then the available source of a certificate. The framework does not treat adjudication as outside the system. A judgment by an identified authority that a formalization is faithful is a discharge certificate in the sense of Appendix A, with an authority, an interpretation, a set of dependencies, and a payload consisting of the reasons given. It is historical when accepted and currently warranted as long as its dependencies are. It can be defeated, and when it is, the obligation returns to the residue. The distinction between historical and warranted discharge is what lets adjudication be both used and revised. Where the adjudication was careful and its dependencies are stable, it is a legitimate basis for reliance, and the framework simply records that this is what the reliance rests on.

Conservative abstention is the policy of reporting an obligation as unresolved whenever no sound means of discharge is available. An abstention is not a null output. It is the creation of an open obligation with the content of the instance's correspondence question, and the transition from the instance to its abstention record is preserving by reflexivity, since the successor has the same content as the original. A pipeline that abstains rather than guessing therefore satisfies the preservation theorem of Chapter 17 trivially, and it reports its residue. Abstention is the formal counterpart of the refusal output in the trustworthy autoformalizer of Theorem 10.5, with the difference that the refusal there was required to be correct, and here the abstention is only required to be honest.

11.5 The design principle

The results of this chapter and the preceding one support a principle that governs the architectures of Part VII.

A verification architecture that is adequate for unrestricted classes of specifications must be permitted to report uncertainty. It may not be required to classify every instance as success or failure.

The argument is a direct combination of the two results. By Theorem 16.6, under the convention stated there, the set of indices at which statement correspondence holds is Σ_2^0 -complete, and by Theorem 10.5 the well-definedness question is hard at every level, so a computable architecture that classifies every instance of a class embedding these constructions as faithful or unfaithful, correctly, cannot exist. By Remark 10.6 the barrier is for classes that embed the

constructions, and a restricted class needs its own analysis. By Proposition 11.2, an architecture that classifies some instances and abstains on others can be sound. Since the architecture cannot be both total and correct, it must either misclassify some instances or abstain on some. An architecture that is required to be total is therefore forced to misclassify, and an architecture that is allowed to abstain is not. The principle is a statement about what a requirement of totality costs.

The principle does not say how much abstention is acceptable or where it should be concentrated. These are design questions, and the chapters of Part VII treat them. What the principle excludes is the position that every verification process must terminate in a verdict, and with it the habit of reading the absence of an objection as the presence of a certificate. Impossibility, on the reading developed here, does not make semantic checking futile. It makes the honest reporting of what has not been checked a structural requirement.

Part IV

The Navier-Stokes Case

All statements in this part are taken from Section 3 of Bastounis, Circelli, and Hansen [3], and the numbering of equations in OpenAI’s natural-language paper, *Finite time blowup for Navier-Stokes* [16], is retained as the source reports it. Lean declarations are cited at commit `f9e8bc5b38b6e212696e8a30e3e91517af887bbd` of `openai/NavierStokesAndEuler` [17]. The monograph has compiled none of the Lean code. The chapters of this part were written from the source alone, so every equation number, lemma number, and quoted bound below is the source’s report of those objects. A later text-level reading of the cited passages of OpenAI’s paper and of the cited Lean declarations is recorded in Appendix E, where each claim is marked as confirmed by that reading, as the source’s report, as the monograph’s interpretation, or as unverified; the classifications of this part remain provisional after that reading. The source states that it makes no claim about the correctness of OpenAI’s natural-language proof, and the part adopts the same discipline: what is examined is the relation between two artifacts, never the truth of the theorem they concern.

Chapter 12

A Derivative Lost in Translation

The first documented discrepancy is small enough to state in one line and consequential enough to serve as the paradigm case of a weakening transition. A lemma of the natural-language paper bounds a quantity in terms of a given number of derivatives of the data, and the corresponding Lean result needs one more.

12.1 The two estimates

The discrepancy concerns Lemma 8.6 of OpenAI's paper. Let m be a nonnegative integer, let $\mathbb{T}^2 := \mathbb{R}^2/\mathbb{Z}^2$, and let $F : P \times \mathbb{T}^2 \rightarrow \mathbb{C}$ be smooth with zero mean in the torus variable y . Let $N = v_t \cdot \nabla_y$ with $v_t = (\sqrt{2} - 1, 1)$, and let $N^{-1}F$ denote the unique smooth zero-mean solution φ of $N\varphi = F$. For $r \geq 0$ the norm is

$$\|G\|_{C_y^r} := \max_{a+b \leq r} \sup_{y \in \mathbb{T}^2} |\partial_{y_1}^a \partial_{y_2}^b G(y)|.$$

The natural-language estimate, labelled (8.19), is

$$\|N^{-1}F\|_{C_y^m} \leq C_m \|F\|_{C_y^{m+4}}, \quad (12.1)$$

and the closest Lean estimate is

$$\|N^{-1}F\|_{C_y^m} \leq K_m \|F\|_{C_y^{m+5}}. \quad (12.2)$$

The constants depend on m and on v_t but not on F . The natural-language bound requires $m + 4$ derivatives of the input to control m derivatives of the output, whereas the Lean result requires $m + 5$.

The cited declaration is

```
norm_derivativeWord_inverse_le
NavierStokes/SmoothFamilyTorusInverse.lean, lines 1059 to 1080
```

at the pinned commit. Its hypotheses `hfirst` and `hsecond` bound the `xJet` of order `w.length+5` of the slice and of its swap, and its conclusion bounds the derivative word by `mixedLossConstantw.length*C`. Taking a supremum over multi-indices with $|w| \leq m$ gives

$$\|N^{-1}F\|_{C_y^m} \leq \hat{K}_m \|F\|_{C_y^{m+5}}, \quad \hat{K}_m = \max(K_0, \dots, K_m),$$

which the source describes as strictly weaker than (12.1) because of the extra derivative. A bibliographic caution belongs here. The source’s Figure 3 transcript cites different line numbers, 978 to 999, from the 1059 to 1080 of the pinned-commit citation in its text. The text’s citation is the one used in this monograph, and the discrepancy is recorded as an open item, since it can be resolved only by inspecting the file at the pinned commit.

12.2 Why one derivative is lost

The source traces the gap to a difference of method. The following reconstruction of the two routes is the monograph’s own arithmetic, offered so that the loss of a derivative can be seen as a feature of the argument and not as an accident of bookkeeping, and it is consistent with the exponents the source reports.

Write the Fourier expansion $F(y) = \sum_{k \in \mathbb{Z}^2 \setminus \{0\}} \hat{F}(k) e^{2\pi i k \cdot y}$. Because $N e^{2\pi i k \cdot y} = 2\pi i (v_t \cdot k) e^{2\pi i k \cdot y}$, the solution $N^{-1}F$ has coefficients $\hat{F}(k)/(2\pi i v_t \cdot k)$, and applying a derivative of order at most m multiplies the coefficient by a factor bounded by a constant times $(1 + |k|)^m$. The natural-language proof invokes the divisor bound (6.7),

$$|v_t \cdot k|^{-1} \leq C(1 + |k|),$$

which costs one power of $(1 + |k|)$. The coefficients of a function with finite C_y^{m+4} norm decay like $(1 + |k|)^{-(m+4)}$ times the norm, up to a constant. The product is therefore bounded by a constant times $\|F\|_{C_y^{m+4}} (1 + |k|)^{m+1-(m+4)} = \|F\|_{C_y^{m+4}} (1 + |k|)^{-3}$, and the source states that four further derivatives leave exactly this summable bound, so that $\sum_{k \in \mathbb{Z}^2} (1 + |k|)^{-3}$ converges. Convergence in two dimensions requires the exponent to exceed 2, since $\sum_{k \in \mathbb{Z}^2} (1 + |k|)^{-s}$ is finite if and only if $s > 2$. The exponent 3 is therefore the smallest integer exponent that works, which is why $m + 4$ is the natural count for this route.

The Lean proof takes a different route. It applies `coefficient_seminorm_bound`, which, as the source reports, yields the series $\sum_{k \in \mathbb{Z}^2} (1 + |k_1| + |k_2|)^{-4}$, the exponent 4 being visible in the Lean source. The route is explicit in the proof. It first needs the Fourier coefficient seminorm of order `w.length+1`, which accounts for the divisor-bound factor, and `coefficient_seminorm_bound` then consumes four more derivatives to reach a summable series. The count is $(m + 1) + 4 = m + 5$. In the natural-language route the divisor factor and the decay are balanced against an exponent of 3; in the Lean route the lemma that supplies summability demands an exponent of 4, one more than is needed, and that surplus is the extra derivative.

The same $m + 5$ requirement appears in two further declarations, `inverse_finiteJets` and `mixedJet_inverse_bound`, which in addition use mixed derivatives and are in that respect incomparable with (12.1). The chapter therefore reports not one weaker lemma but a family of Lean results with a common count.

12.3 What the Lean result does and does not establish

The distinction on which the chapter turns is between a verified weaker estimate and a contradiction of the stronger claim. The Lean result does not refute (12.1). Nothing in it asserts that the $m + 4$ bound is false, and nothing in it is inconsistent with that bound. It fails to certify the bound. A reader who learns that the corresponding Lean lemma is proved may take

the natural-language lemma to be confirmed, and the discrepancy shows why that inference is unavailable: what was confirmed is the lemma with one more derivative. The aim is not to decide anything about the Navier-Stokes argument but to show how a single additional derivative changes what has been certified.

It does not follow that the extra derivative is harmful downstream. Whether the rest of the proof can absorb a requirement of $m + 5$ derivatives where it was written to need $m + 4$ depends on how the lemma is applied, and the source does not settle it. The question is nonetheless an obligation, and its form is given by the propagation result of Chapter 17: every statement that uses (12.1) inherits the question of whether its use survives the loss.

12.4 The case in the ledger

Let $o_{8.19}$ be the obligation whose content is (12.1) and o_{Lean} the obligation whose content is (12.2). The transition from the natural-language paper to the formalization has $Q = \{(o_{8.19}, o_{\text{Lean}})\}$, $\Delta = \emptyset$, and $\Theta = \emptyset$, so the certified context of $o_{8.19}$ is $\{\varphi(o_{\text{Lean}})\}$. Two entailments are relevant. The first is that $\varphi(o_{8.19})$ entails $\varphi(o_{\text{Lean}})$, which is elementary: since $\|F\|_{C_y^{m+4}} \leq \|F\|_{C_y^{m+5}}$, any C_m satisfying (12.1) satisfies (12.2) with $K_m = C_m$. The second is the converse, that $\varphi(o_{\text{Lean}})$ entails $\varphi(o_{8.19})$. If that entailment fails, the transition is non-preserving at $o_{8.19}$ and, since the original entails everything in the context, it is a weakening in the sense of Definition A.13.

The source supplies the failure of the converse by describing the Lean estimate as strictly weaker. The monograph records the classification as source-supported and provisional, as it must, because it supplies no independent argument for the non-entailment. The next section explains why the classification also depends on the choice of consequence relation, and why that dependence is not a technicality.

12.5 On the choice of consequence relation

A question arises that the formalism of Appendix A must answer, and it is best faced directly. If $\models_{\mathcal{I}}$ were classical semantic consequence, so that $X \models \psi$ whenever ψ is true in every model of X , then any two true statements would be equivalent, and all true statements would be consequences of the empty set. The Lean estimate (12.2) is presumably true and so is, if the lemma of the natural-language paper is correct, (12.1). Under classical consequence each entails the other, every transition between true contents would be preserving, and the whole analysis would collapse into a question about the truth of the theorem, which is what the part has promised not to address.

The consequence relation of the framework is not classical consequence over all truths. It is relative to the *certified background* of the development: $X \models_{\mathcal{I}} \psi$ means that ψ is derivable from X together with a fixed base of admitted facts and rules, by admitted inference steps. Appendix A requires only that $\models_{\mathcal{I}}$ be reflexive, monotone, and transitive, and those conditions are satisfied by derivability relative to a base. What the choice of base does is fix what counts as available to the downstream steps of a proof. With the base consisting of the certified results of the Lean development, the Lean estimate does not yield (12.1), because no admitted derivation produces it, and the non-entailment means that the stronger bound is not certified from the weaker, not that it is false. The typing of transitions is thus relative to a base, and a transition

may be a weakening relative to one base and conservative relative to another that includes a restoring lemma. This is what the appendix means by a justification that restores entailment, and it is the reason the typing is a statement about the whole certified context.

The consequence for this chapter is a precise one. The claim that the Lean transition is a weakening is the claim that, relative to the certified Lean development, (12.1) is not derivable from (12.2). That claim is checkable in principle by inspection of the development, and the monograph reports it on the strength of the source.

12.6 Three repairs

The ledger states what would have to be done to close the gap, and the three options differ in what they leave in the residue.

The first repair is *discharge*: prove (12.1) in the formal development, by following the natural-language route through the divisor bound with exponent 3. This closes the discrepancy at the lemma, and the obligation $o_{8.19}$ is discharged by the new proof. The second is *assumption*: admit (12.1) as a tracked assumption obligation a with $\varphi(a) = \varphi(o_{8.19})$. The transition then becomes preserving, with $\Theta = \{\varphi(a)\}$, and the formal development carries the open obligation a in its residue wherever the lemma is used. This is the honest form of the conditional theorem: the Lean results hold given the $m + 4$ bound, and the bound is unproved in the development. The third is *acceptance of the weakening*: keep the Lean estimate, record the transition as a weakening, and discharge the obligation of showing that every downstream use tolerates $m + 5$. Each of the downstream obligations is a statement of the form that a particular application of (12.1) remains valid with one more derivative of regularity.

None of the three is selected by the kernel. The framework’s contribution is that all three are expressible and that the choice among them is visible in the residue, in contrast to the unrepaired state in which the extra derivative is invisible to anyone who reads only that the lemma has been formalized.

12.7 What the chapter establishes

In terms of the three judgments of Chapter 16, the Lean declaration is accepted by the kernel, so K holds for the instance. The statement obligation of the natural-language lemma against the Lean statement is not discharged as an equivalence: relative to the certified Lean development, the Lean statement is strictly weaker, and the monograph records that status as source-reported. The argument obligation is a distinct matter. On the source’s description the two proofs follow different routes, as the exponents 3 and 4 in the monograph’s own reconstruction suggest. That reconstruction does not show that the Lean route is a different argument, since it could be the same strategy with a lossier summability step, so the failure of argument correspondence is source-reported and provisional, and the case is offered as one in which all three judgments may separate in a single lemma. This is why the chapter is the monograph’s paradigm case: a verified result, a documented weakening at the level of the statement, and a documented divergence of method, none of which is an error by the kernel and all of which the phrase “the lemma was formalized” conceals.

Chapter 13

The Pressure-Flux Discrepancy

The discrepancy of the previous chapter was a matter of degree: one estimate was the other with an extra derivative. The second documented discrepancy is of a different kind. The two bounds depend on different quantities, the arguments that produce them differ at each step, and neither bound is a corollary of the other on the source's report. The case shows that the typing of transitions in Chapter 17 is not a ranking by strength, and it gives the monograph its example of a lateral transition.

13.1 The setting and the two estimates

The discrepancy concerns the pressure-flux estimate (10.19), which occurs in the proof of the velocity-uniqueness Lemma 10.5. The setting is two solutions (v, P) and (u, p) with $w := v - u$ and $\pi := P - p$. A cutoff φ satisfies $0 \leq \varphi \leq 1$ and equals one on the unit ball. Put $\varphi_R(x) = \varphi(x/R)$ and $\chi_R = \varphi_R^8$. Define

$$A_R := \left(\int \chi_R |\nabla w|^2 \right)^{1/2}, \quad B_R := \|\varphi_R^4 w\|_{L^6},$$

so that A_R is a localized L^2 gradient norm and B_R a localized L^6 velocity norm. With $R \geq 1$, the natural-language bound (10.19) is

$$\left| \int \pi w \cdot \nabla \chi_R \right| \leq \frac{C_T}{R} \left[(B_R + 1) B_R^{1/2} + R^{-3/4} B_R^{3/4} \right], \quad \text{for a.e. } t \in (0, T), \quad (13.1)$$

and the Lean estimate is

$$\left| \int \pi w \cdot \nabla \chi_R \right| \leq C_T \left[(B_R^{1/2} + 1) \left(\frac{A_R}{R} + \frac{1}{R^2} \right) + R^{-7/4} B_R^{3/4} \right], \quad \forall t \in (0, T). \quad (13.2)$$

The natural-language bound is stated purely in terms of B_R , whereas the Lean estimate additionally depends on A_R . The cited declaration is `exists_uniform_actual_pressure_flux_bound`, at line 576 of `NavierStokes/R3/PressureFlux.lean`, with related bounds in `exists_uniform_canonicalCutoffFlux_bound` and `canonicalCutoffFlux_bound`.

Equation (10.17) of the natural-language paper bounds B_R using A_R . It cannot be used to eliminate A_R from the Lean estimate so as to recover (10.19), and the source concludes that the two bounds are not directly comparable. Two features of the displays make the lack of

comparability visible. The Lean bound carries an additional dependence on A_R and so is, in that respect, weaker than a bound in B_R alone. It holds for every $t \in (0, T)$ and not merely almost every t , and so is, in that respect, stronger. A bound that is weaker in one respect and stronger in another is not ordered by implication.

13.2 Three differences of argument

The source identifies three differences between the proofs. First, the Lean proof does not use boundedness of the Riesz transform from $L^{3/2}$ to $L^{3/2}$. It uses a Sobolev embedding to move to L^2 , together with L^2 boundedness of the Riesz transform, and the extra derivative in the embedding is the reason A_R appears. Second, it applies Hölder's inequality with different exponents. Third, it shows the integral to be continuous in time, which removes the qualifier “for almost every t ”.

The divergence can be followed term by term. The natural-language proof replaces π by

$$\pi_* = \sum_{i,j} R_i R_j g_{ij}, \quad g_{ij} = v_i v_j - u_i u_j = w_i w_j + w_i u_j + u_i w_j,$$

where R_i denotes the Riesz transform. It factors the integrand into $\varphi_R^4 \pi_*$ and a test function containing φ_R^3 . It then uses the commutator identity (10.18),

$$\varphi_R^4 \pi_* = \sum_{i,j} R_i R_j (\varphi_R^4 g_{ij}) + \sum_{i,j} [\varphi_R^4, R_i R_j] g_{ij},$$

and bounds the first term in $L^{3/2}$ by Hölder's inequality and the $L^{3/2}$ boundedness of the Riesz transforms, citing Stein, which gives $\|R_i R_j (\varphi_R^4 g_{ij})\|_{3/2} \leq C''(B_R + 1)$.

The Lean proof factors out φ_R^2 instead, with test function $w \cdot \varphi_R^5 \nabla \varphi_R$. It applies Hölder with exponents 6 and 6/5. It bounds $\|R_i R_j (w \cdot \varphi_R^5 \nabla \varphi_R)\|_6$ by a Sobolev embedding in three dimensions, interchange of the gradient with the Riesz transform, and L^2 boundedness of the Riesz transforms, which produces the $A_R/R + 1/R^2$ term. It obtains the $B_R^{1/2} + 1$ factor from $\|\varphi_R^2 g_{ij}\|_{6/5}$ through the pointwise bound $|\varphi_R^2 g_{ij}| \leq 2|u||w| + |\varphi_R w|^2$ and the interpolation

$$\|\varphi_R w\|_{12/5} \leq \|w\|_2^{3/4} \|\varphi_R^4 w\|_6^{1/4} \leq C B_R^{1/4},$$

a step with no counterpart in the proof of Lemma 10.5. The Lean proof also works with $w = u - v$ and $\pi = p - P$, the opposite sign convention. The two arguments share a goal and a decomposition of π into Riesz transforms, and beyond that share little.

13.3 Two cautions from the source

The source adds two cautions, and the chapter preserves both. The first concerns the apparent strictness of (10.19). Since the purpose of Lemma 10.5 is to establish $w = 0$, the right-hand side of (13.1) is eventually zero and is stricter than the Lean bound. But this is established only after the whole proof of the lemma. At the stage where the estimate is used, w is not yet known to vanish, and the comparison available then is the one between the displayed expressions. The retrospective strictness cannot be used to rank the two estimates for the purpose of the argument in which they occur, and the reader should not import it into the comparison.

The second caution concerns a related Lean bound in `R3StressPressureEstimate.lean`, `regularized_flux_Bound`. It differs from (10.19) and does not appear to be used in the proof of the final theorem. Its existence is evidence of the variety of nearby estimates in the formal development. It does not bear on the discrepancy at hand, and the monograph does not rely on it.

The source also records, as in the previous chapter, that the examination is of the relation between artifacts and not of the correctness of the natural-language argument. The Lean estimate does not refute (10.19), and it does not certify it.

13.4 The case in the ledger

Let $o_{10.19}$ be the obligation whose content is (13.1) and o_{Lean} that of (13.2). The transition has a single successor, no discharges, and no assumptions, so the certified context of $o_{10.19}$ is $\{\varphi(o_{\text{Lean}})\}$. The two relevant questions are whether $\varphi(o_{10.19}) \models \varphi(o_{\text{Lean}})$ and whether $\varphi(o_{\text{Lean}}) \models \varphi(o_{10.19})$, both relative to the certified background of the development in the sense of the previous chapter. The source's report is that neither entailment is available by the obvious means. The first would require passing from a bound in B_R for almost every t to a bound in B_R and A_R for every t , and the second would require eliminating A_R , which (10.17) does not permit. On that report the transition is of lateral type in the sense of Definition A.13.

The classification is source-supported and provisional. The monograph supplies neither a countermodel nor an analytic argument establishing both non-entailments, and the typing should be read as recording that the source's assessment places the estimates in the fourth cell of the table, subject to independent verification. If one of the entailments were established, the transition would be reclassified as a weakening or a strengthening. In either case the chapter's methodological point stands: the framework represents a discrepancy that is not a loss of content in the sense of weakening, and it does not force the discrepancy into a generic category of mistranslation.

13.5 Statement fidelity and argument fidelity apart

The two cases of Chapters 12 and 13 together illustrate the separation of the judgments that Chapter 16 proves abstractly. In the derivative case, the statements differ by a shift in a parameter and the proofs diverge in a lemma used for summability. In the pressure-flux case, the statements differ in their dependence on a quantity and in their quantifier over time, and the proofs differ at almost every step. In both cases the kernel accepts the Lean declaration. In both the statement obligation against the natural-language estimate is not discharged as an equivalence. And in both the argument obligation is reported by the source to differ, a report that I record as provisional. The pressure-flux case shows more clearly than the derivative case that the failures of statement fidelity and argument fidelity can be tracked separately: the shared goal and shared decomposition of the pressure into Riesz transforms make the statements comparable in outline, while the divergence of method makes the argument obligation fail in a way that is independent of how the statement obligation is eventually resolved.

The source observes that the case motivates exactly this: tracking statement fidelity and argument fidelity as separate obligations. The monograph's contribution is to supply the vocabulary for that tracking, and to attach to each discrepancy a type, a set of repairs, and a

statement of what remains in the residue. As in the previous chapter, nothing in the examination bears on whether the natural-language proof is correct. What the two chapters establish is the narrower and firmer point that a formal verification can succeed in a way that leaves the relation between the verified artifact and the announced one unsettled, and that the unsettled relation can be described precisely.

Chapter 14

What the Navier-Stokes Formalization Establishes

Chapters 12 and 13 documented two discrepancies between OpenAI’s natural-language statements and the Lean statements that stand in for them. This chapter asks what follows. It does not ask whether the Navier-Stokes blow-up argument is correct, a question on which the source takes no position and which the monograph does not address. It asks what epistemic standing the verified artifacts have given the documented discrepancies, which claims remain justified, which have been withdrawn, and which were never supported by the verification at all.

14.1 Three kinds of evidence

Three different claims are often compressed into the statement that a result has been formally verified. The first is that the theorem is true. The second is that a particular proof of it is valid. The third is that an announced result has been adequately documented for independent review. Evidence for the first bears on the mathematical facts, evidence for the second on the derivation, and evidence for the third on the relation between the announcement and the artifacts that are supposed to support it.

A successful Lean check is evidence of the second kind with respect to the Lean statement and its Lean proof. It is not evidence of the first kind with respect to the natural-language theorem unless the Lean statement corresponds to it, and the first two documented discrepancies, on the source’s report and with the provisional classification of Chapters 12 and 13, indicate that for specific lemmas it does not correspond as an equivalence. The non-entailment is the source’s claim; no independent certificate of it is offered here. It is evidence of the third kind only to the extent that the artifacts can be read against the announcement, and a discrepancy of the kind documented in Chapters 12 and 13 reduces that extent. The three kinds are logically independent. A true theorem may be announced with an invalid proof, a valid proof may support a weaker theorem than the one announced, and a correct proof of the announced theorem may be impossible to check against the announcement.

14.2 Claims that remain justified, in question, or unsupported

The ledger of Chapter 17, applied to what the documented discrepancies establish, sorts claims into three groups.

Claim	Status	Basis
The cited Lean declarations are accepted by the Lean kernel	Justified, as reported	Source’s report. Not compiled by the monograph
<code>norm_derivativeWord_inverse_le</code> establishes an estimate requiring $m + 5$ derivatives	Justified, as reported	Source’s citation of the declaration at the pinned commit; text of the declaration read at that commit (Appendix E)
That estimate establishes (8.19) of the natural-language paper	In question	The Lean estimate is weaker relative to the certified Lean development
The Lean pressure-flux estimate establishes (10.19)	In question	The two estimates are not directly comparable on the source’s report
The natural-language proof is incorrect	Unsupported	The source disclaims this
The natural-language proof is correct	Unsupported	Neither the Lean artifacts nor the source address it
The remaining Lean declarations faithfully formalize their counterparts	Unexamined	Not examined by the source or the monograph

Table 14.1: Epistemic standing of claims about the Navier-Stokes formalization.

The distinction between in question and unsupported matters. A claim is in question when it was made or could be made on the strength of the formalization and the source’s reported discrepancies, if they stand, would defeat it; in the vocabulary of Appendix A it is open with a source-reported non-entailment, and it is not refuted, since no certificate of the negation is claimed. An unsupported claim is one that no part of the evidence addresses, in either direction. The claim that the natural-language proof is incorrect belongs to the second group. The documented discrepancies are discrepancies of translation between two artifacts, and a mistranslation of a correct lemma is as compatible with them as a mistranslation of an incorrect one. The source says as much, and the monograph adopts the disclaimer as binding. In the vocabulary of Chapter 17, the Lean artifacts and the natural-language paper are separate objects, and the relation between them is the subject of the residue. The residue is not a verdict on the theorem.

The final row of the table records a large unexamined region. The source reports two discrepancies and states in Remark 3.4 that the displayed examples do not exhaust the mistranslations. The remainder of the formal development has not been audited in the way the two cases have, and the audit is not complete. The monograph therefore draws no conclusion about the fidelity of the rest, either that it is faithful or that it is not. A reader who infers fidelity of the remainder from the absence of reported discrepancies makes an inference from silence, and a reader who infers infidelity from the existence of two discrepancies makes an inference from a sample whose selection is not described.

14.3 The Euler claim and the limits of prediction

The source's Section 3.3 concerns OpenAI's announced proof for the Euler equations. It treats the announcement only by prediction. The source states that a careful examination was not carried out, and that its assessment of a very high likelihood of substantial mistranslation rests on preliminary examination. No documented Euler discrepancy therefore exists, and the monograph does not treat the Euler case as a third example. The distinction is between a documented discrepancy, which is an observed relation between two specified artifacts, and a forecast, which is an expectation about artifacts that have not been compared. A forecast may be well founded and is still not evidence of the same kind. The ledger records the Euler case, if at all, as an open obligation whose content is that the Euler formalization corresponds to the Euler announcement, with status open and a note that the source's expectation bears on it without discharging or refuting it.

14.4 The mechanism: acceptance loops

Section 3.4 of the source supplies a mechanism by which discrepancies of this kind are produced, and the mechanism is an instance of the separation proved in Chapter 16. The acceptance loop in question revises a proof until it proves the formal statement. Its termination test is kernel acceptance of the revised proof against a fixed formal proposition F .

Proposition 14.1 (Indifference of the acceptance loop). *Let F be fixed and let π' be any proof with $\Gamma \vdash \pi' : F$. The acceptance test returns the same result on π' whether π' is (a) a repair of the original argument, correcting an error while following its structure, or (b) a replacement, an unrelated proof of F . If the original argument is incorrect, replacement is available as a route to acceptance and repair may not be.*

Proof. The test is a function of (Γ, π', F) alone, as in Theorem 16.1. Repair and replacement are distinguished only by the relation of π' to the original argument P , which does not enter the test. If P is incorrect and no repair of P yields a proof of F in Γ , any proof of F is a replacement. $\square$

The consequence is that the loop counts both the repair of an error and the replacement of a correct argument as success. It also gives an incentive to find a different proof when the original is incorrect, since a different proof is the only route to acceptance in that case. This is the structure of Example 2.1 in the source, where the chatbot's translation succeeded by finding the correct multiplicities and so did not translate the argument supplied. The mechanism does not establish that any particular proof in the Navier-Stokes development was produced by replacement, and the source does not claim it. It establishes that a loop of this form cannot distinguish replacement from repair, so a reported success from such a loop is not evidence of argument fidelity.

14.5 Back-translation does not escape the problem

Remark 3.5 of the source anticipates the natural response: translate the accepted formal proof back into natural language and compare. The remark observes that this requires a further check that the back-translation represents the formal proof faithfully and that its reasoning

matches the original argument. The point is structural, and the ledger shows it. The chain from the original argument P to the formal proof π to the back-translation $B(\pi)$ consists of two transitions. The comparison of $B(\pi)$ with P is a check on the composite only if each transition is preserving, in the sense of Theorem 17.3. The back-translation transition introduces its own obligation o_B , that $B(\pi)$ corresponds to π . Agreement of $B(\pi)$ with P in outline establishes the composite relation only when o_B is discharged, and discharging o_B is a correspondence problem of the same kind as the one the round trip was meant to address. Round-trip agreement does not entail semantic preservation, which is RV-007, proved in Chapter 8. The back-translation moves the obligation. It does not discharge it.

14.6 The state of the residue

The residue of the Navier-Stokes formalization, as far as the monograph can describe it from the source, has the following shape. There are two statement-equivalences that are open, with non-entailment reported by the source and a provisional classification, the derivative lemma and the pressure-flux lemma, each with its type and its repairs recorded. There are the argument obligations for each lemma, failing independently. There is an open obligation of unknown size for the unexamined declarations. There is an open obligation for the Euler formalization, supported only by a forecast. There is, finally, the procedural obligation that the process that produced the artifacts tests acceptance and not fidelity. None of these bears on the truth of the theorem, and each bears on how much weight the announcement of a verified proof can carry.

A further entry belongs to the monograph itself. Its reading of the Lean artifacts and of the natural-language paper is second-hand, through the source. The equation numbers, lemma numbers, and quoted passages are as reported, and the Lean declarations have not been compiled. The monograph's own claims about the case are therefore conditional on the accuracy of the source, and that conditionality is recorded as an open obligation of the monograph in the list of items to verify.

Part V

A Theory of Verification Residue

Chapter 15

Obligations That Survive Verification

A successful run of a proof checker is an event with a precise content: a term has been found to inhabit a type in a stated environment. Almost every claim that is subsequently made on the strength of that event goes beyond this content. It is said that a theorem has been proved, that a published argument has been confirmed, that a question has been answered, or that a result may be relied upon in further work. Each of these claims carries a commitment that the kernel never examined, and the purpose of this chapter is to give those commitments a uniform representation so that they can be counted, tracked, and eventually discharged. The representation is the obligation record, and the collection of obligations that remain unsettled after a verification event is the *verification residue*. The formal definitions are collected in Appendix A. The present chapter motivates them, fixes their intended reading, and shows on small cases how they behave.

15.1 What a successful check leaves unsettled

Consider the polynomial case with which the monograph opens. The stated theorem is that $p(x) = x^3 - x^2 - x + 1$ is nonnegative for $x \geq -1$. The supplied natural-language argument asserts the factorization $p(x) = (x - 1)(x + 1)^2$, which is incorrect, since $p(x) = (x + 1)(x - 1)^2$. A language model asked to translate the argument into Lean returned a proof that compiles, having found the correct multiplicities in the course of the translation. The kernel accepts the term. The first baseline obligation, that $\Gamma \vdash \pi : F$, is discharged. Nothing else has been settled by that fact. The statement obligation, that the Lean proposition expresses the stated theorem, is plausible here and can be discharged by inspection, but inspection is a separate act with a separate certificate. The argument obligation, that the Lean proof follows the supplied argument, is false: the supplied argument is invalid, and the formal proof does not formalize it. A faithful translation of that argument would have failed to compile, and it is the success of the translation that conceals the discrepancy.

The example shows that the three baseline obligations of a verification event can take three different statuses at once. The first is discharged, the second can be discharged with additional evidence, and the third is refuted. A report of the form “the proof was verified” collapses these into a single value and in doing so asserts something false of the third. The theory developed in this part prevents the collapse by giving each obligation a status of its own and defining the residue as what remains.

15.2 The obligation record

An obligation is a record $o = (\tau, \varphi, \rho, E, D, \sigma)$ whose fields have the following intended readings. The type τ classifies the obligation. The types that recur in the monograph are *formal* (kernel acceptance of a stated term), *statement* (correspondence of a formal proposition with a specification), *argument* (correspondence of a formal derivation with a natural-language argument), *dependency* (adequacy of a definition, imported lemma, or axiom on which a result relies), *interpretive* (the choice of $\equiv_{\mathcal{I}}$ and of the intended interpretation), *procedural* (properties of the process that produced the artifact, such as the acceptance loop of Chapter 14), and *assumption* (a premise admitted during a transformation and recorded for later discharge). The content φ is the proposition the obligation asserts, stated in a content language equipped with a consequence relation. The provenance ρ records where the obligation arose and which act introduced it. The evidence E is the collection of artifacts bearing on the content, including certificates, counterexamples, and expert judgments. The dependencies D record the other obligations on which the establishment of this one relies. The status σ is one of five values.

The five statuses are **open**, **supported**, **discharged**, **refuted**, and **defeated**. An obligation is **open** when no evidence has yet been admitted that bears decisively on its content. It is **supported** when evidence exists that falls short of an accepted certificate, for example a body of test cases or an informal argument that has not been checked. It is **discharged** when an accepted certificate establishes the content. It is **refuted** when an accepted certificate establishes the negation of the content. It is **defeated** when a certificate that once established the content has been overridden, by the discovery of a flaw in the certificate or in something it depends on, without a certificate for the negation having been obtained. The distinction between the last two is substantive. A defeated certificate leaves the content undetermined, whereas a refutation settles it negatively, and conflating them would allow the failure of an argument for a proposition to be reported as the falsity of the proposition.

An obligation whose discharge loses warrant takes the status **defeated** when a recorded defeater targets its certificate or a dependency, and **open** when the certificate is retracted without a defeater, so that the earlier evidence is no longer admitted as decisive; the policy is the ledger's, and the historical record keeps both facts. Each obligation carries exactly one status. Questions that might seem to call for a sixth status are handled in other fields. Conflicting evidence, where some admitted evidence favors the content and some opposes it, is recorded as a property of E , and the status remains **open** or **supported** according to the policy of the ledger until a certificate is accepted. A policy that distinguished a status **contested** would be a legitimate alternative, and the partition results of Appendix A would then range over six values and not five. What is not permitted is for the status of one obligation to be a set of values, because the residue accounting of the next section depends on the status function being single valued.

15.3 Five ways of being unsettled

The informal vocabulary of verification does not distinguish several quite different reasons why an obligation may fail to be discharged, and the framework is organized so that they can be told apart. A *logical failure* is a case where a derivation is invalid or a statement is false, and the corresponding obligation is refuted. *Semantic uncertainty* is a case where it is not known whether a formal statement means what it was intended to mean; the statement obligation is

open, and no amount of kernel checking bears on it. *Missing evidence* is a case where a relevant check has not been carried out, and the obligation is open or supported. *Conflicting evidence* is a case where checks disagree, and it is recorded in E . *Defeated certification* is a case where an earlier discharge no longer stands, and the obligation is defeated.

These distinctions are borrowed in structure from the architecture of Adversaria, where claims, evidence, and defeaters are tracked in a public graph, and the application to mathematical formalization is a proposed extension and not a result of the source paper on which the case material of this monograph rests. The borrowing is not decorative: the notion of warranted discharge in Section 15.5 below requires exactly the defeater semantics that Adversaria supplies, because a certificate that depends on a defeated assumption must lose its standing without the history of its acceptance being rewritten.

15.4 The residue and its partition

For a finite set $\mathcal{O}$ of obligations attached to an artifact, the residue is the set of obligations whose status is not **discharged**. Because the status function is single valued, the residue decomposes as the disjoint union

$$\mathcal{R}(\mathcal{O}) = \mathcal{R}_{\text{open}} \sqcup \mathcal{R}_{\text{supported}} \sqcup \mathcal{R}_{\text{refuted}} \sqcup \mathcal{R}_{\text{defeated}},$$

which is Proposition A.4 of the appendix. The statement is definitional and is recorded as such. Its value is that it forces the accounting to be complete: every obligation of an artifact is either discharged or lies in exactly one of four named components. A report that cites only the number of open obligations is incomplete in a specified way, namely that it omits the refuted and defeated components, which are the components that most directly bear on whether the artifact may be relied on.

A residue is not an error. The point is easily lost, because the word suggests a defect. A residue may consist entirely of assumptions that are standard in the field, interpretive choices that are uncontroversial, and statement obligations that every reader would discharge on inspection. What distinguishes a well-run verification from a poorly run one is not the absence of a residue but its legibility: that each member is named, that its status is honestly recorded, and that nothing has been moved out of the residue by relabeling. The residue of a formal proof of a classical theorem in a mature library may be small and benign. The residue of an announced resolution of a major open problem, formalized by an automated translation that has not been audited, may be large, and its size is not a measure of the quality of the underlying mathematics.

15.5 Historical and warranted discharge

Discharge has two readings, and the monograph is explicit about which is in force at any point. A discharge is *historical* when it records that a certificate was accepted by an identified authority under an identified interpretation with an identified set of dependencies. It is *currently warranted* when, in addition, every dependency of the certificate is itself warranted and no defeater is recorded against the certificate. The first is a fact about the past and is never revised. The second is a function of the present ledger and can be lost.

The need for the distinction arises whenever an assumption introduced at an early stage supports certificates at later stages. A formalization may import a library lemma whose statement is later found to differ from its documentation. Every certificate that relied on the lemma remains in the history, but none remains warranted, and the obligations they discharged, unless another certificate for them remains warranted, are no longer warranted-discharged and take one of the other statuses according to the policy of the ledger (Definition A.9). Proposition A.8 of the appendix states this precisely and shows that, for a fixed set of accepted certificates, warrant is antitone in the set of recorded defeaters, while the historical record only accumulates. A verification record that supports only the historical reading gives an inaccurate picture of the evidential state, and one that supports only the warranted reading erases the history that makes revision auditable.

15.6 Theorem correctness and proof fidelity

Two properties are routinely run together, and the obligation framework separates them. *Theorem correctness* is the property that the stated result is true. *Proof fidelity* is the property that a given proof is a faithful formalization of a given argument. A new formal proof of a true theorem may be mathematically valuable without being a faithful formalization of the supplied proof, as in the polynomial example. A faithful formalization of an invalid argument should expose the failure and not silently repair it. The first corresponds to the statement and formal obligations, the second to the argument obligation, and the independence of the two is the subject of the next chapter.

The distinction has a practical consequence for any acceptance procedure that treats compilation as success. If the only test applied to a revised translation is whether it compiles, the procedure counts the repair of an error and the replacement of a correct argument by a different one as the same outcome, which is the mechanism recorded in Section 3.4 of the source and discussed in Chapter 14. The obligation framework makes the omitted test explicit: the argument obligation was not tested, and its status therefore remains what it was before the revision.

Chapter 16

The Non-Discharge Principle

The three judgments of a verification instance, kernel acceptance $K(V)$, statement correspondence $C(V)$, and argument correspondence $A(V)$, are distinct relations, and a principal claim of the monograph is that none of the first two entails the next. This chapter proves the separation results in a form that can be verified from the definitions of Appendix A. The results are elementary, and the care that they require lies in stating exactly which features of an instance each judgment depends on. The chapter closes by connecting the separation to the computability barrier of Chapter 10, where the gap between K and C is shown to lie beyond Δ_2^0 , in a precise sense and under stated conventions.

16.1 Kernel acceptance does not entail correspondence

The key observation is that $K(V)$ does not mention most of the instance. By definition $K(V) \iff \Gamma \vdash \pi : F$, and this predicate is a function of (Γ, π, F) alone. It is invariant under any change to the originating specification N , the purported argument P , and the interpretation $\mathcal{I}$. The judgment $C(V)$, by contrast, is a relation between $\llbracket N \rrbracket_{\mathcal{I}}$ and $\llbracket F \rrbracket_{\mathcal{I}}$ and so depends on N and $\mathcal{I}$ in an essential way.

Theorem 16.1 (RV-001). *Suppose there are specifications N_1, N_2 and a formal proposition F with $\Gamma \vdash \pi : F$ such that $\llbracket N_1 \rrbracket_{\mathcal{I}} \equiv_{\mathcal{I}} \llbracket F \rrbracket_{\mathcal{I}}$ and $\llbracket N_2 \rrbracket_{\mathcal{I}} \not\equiv_{\mathcal{I}} \llbracket F \rrbracket_{\mathcal{I}}$. Then, writing $V_j = (N_j, P_j, \mathcal{I}, \Gamma, F, \pi)$, one has $K(V_1) \wedge C(V_1)$ and $K(V_2) \wedge \neg C(V_2)$. In particular there is no function g with $C(V) = g(K(V))$.*

Proof. Both instances share (Γ, π, F) , so $K(V_1) \iff K(V_2)$, and both hold by the hypothesis $\Gamma \vdash \pi : F$. The correspondence judgments differ by the choice of N_j . If $C = g \circ K$ then $C(V_1) = g(\text{true}) = C(V_2)$, contradicting $C(V_1) \wedge \neg C(V_2)$. $\square$

The hypothesis is satisfied whenever the semantic domain contains two inequivalent denotations and one of them is the denotation of a provable formal proposition. For a concrete instance, let F be the Lean proposition expressing that there are infinitely many odd numbers, let N_1 be the sentence “there are infinitely many odd numbers”, and let N_2 be “there are infinitely many primes”. Both sentences are true, and both have proofs that compile. For F to correspond to the first and not the second, $\equiv_{\mathcal{I}}$ must be an intensional relation, for example identity of the quantified dependencies in the sentence, since under logical equivalence of true

sentences the two correspond equally. The example is offered under that choice, which Appendix A allows. Example 4.4 of the source supplies the corresponding substitution of “prime” for “odd”: the altered sentence is a different true sentence that compiles equally well. Kernel acceptance of F is blind to which of the two was meant.

16.2 Statement fidelity and argument fidelity are independent

Theorem 16.2 (RV-002). *Call an argument-correspondence relation strict if it requires each justified inferential step of P to have a counterpart in π , a counterpart being a step of π that performs the same inference. Under any strict relation $\mathcal{A}_{\mathcal{I}}$ for which a change of basis and a rewrite using symmetry are different inferences, there is an instance with $K(V) \wedge C(V) \wedge \neg A(V)$.*

Proof. The instance is Example 2.2 of the source. For a real symmetric matrix A the specification N asserts $\text{tr}(A^2) \geq 0$. The supplied argument P diagonalizes A in an orthonormal eigenbasis, so that $\text{tr}(A^2) = \sum_i \lambda_i^2 \geq 0$. The Lean proof π omits the change of basis and uses the symmetry $A = A^T$ to write $\text{tr}(A^2) = \sum_{i,j} a_{ij}^2$, a sum of squares. Both arguments are correct, so $K(V)$ holds, and the formal proposition expresses N , so $C(V)$ holds. The diagonalization step of P has no counterpart in π in the sense just stated, so $A(V)$ fails under the stated requirement. The facts that both proofs are accepted by the kernel and that the Lean proof takes the stated route are as reported by the source and have not been checked here. A more liberal relation that identified the two computations after a change of basis would give $A(V)$, and the theorem is stated for the stricter family. $\square$

The theorem is conditional on the choice of $\mathcal{A}_{\mathcal{I}}$, and the condition is not a loophole. A relation that identified arguments with the same conclusion would make A collapse into C and would render the notion of a faithful formalization of a proof empty, since any proof of the same statement would count. The requirement on steps is one natural structure under which the question of fidelity to an argument is meaningful, and the theorem is a theorem about the strict family and not about every conceivable relation. The two proofs of the example are both correct, and the lesson of the example is that correctness of both does not make them the same argument. This case is not an instance of error, and the monograph does not treat it as one.

16.3 Strengthened hypotheses and substituted tasks

Two further separation results describe how a valid proof can fail to settle the original question. They are stated here for the substitution of statements and tasks, which Chapter 7 develops in detail.

Proposition 16.3 (RV-004). *Let $\models_{\mathcal{I}}$ satisfy the deduction property, $X \cup \{a\} \models_{\mathcal{I}} b$ if and only if $X \models_{\mathcal{I}} (a \rightarrow b)$, and let H, H' be propositions with $H' \models_{\mathcal{I}} H$ and $H \not\models_{\mathcal{I}} H'$ (so H' is strictly stronger than H). Then:*

- (i) *for every G , a derivation of $H \rightarrow G$ yields a derivation of $H' \rightarrow G$;*
- (ii) *the converse fails in general: for $G := H'$, the implication $H' \rightarrow G$ is derivable while $H \rightarrow G$ is not.*

Consequently a derivation of $H' \rightarrow G$ settles $H \rightarrow G$ only for those G for which $H \rightarrow G$ is separately derivable, for instance by a restoring lemma.

Proof. (i) From $H \rightarrow G$ and $H' \models H$, the premises $\{H \rightarrow G, H'\}$ derive G by cut and the deduction property, so $H \rightarrow G \models H' \rightarrow G$. (ii) $H' \rightarrow H'$ is derivable by reflexivity and the deduction property. By the deduction property again, $H \rightarrow H'$ is derivable from the empty premise set if and only if $H \models_{\mathcal{I}} H'$, which is excluded by hypothesis (relative to the certified background, so that the claim is non-derivability and not falsity). The final sentence is a restatement: since the conclusion $H' \rightarrow G$ is the weaker of the two, derivability of $H \rightarrow G$ is not determined by it. $\square$

The proposition is stated for the consequence relation of Appendix A, with the deduction property added as a hypothesis on $\mathcal{L}$, since the appendix does not assume a connective. An informal reading is that the strengthened-hypothesis theorem is the weaker theorem, and proving it leaves the stronger one open.

Proposition RV-015 (Proposition 7.3), proved in Chapter 7, states the substitution result for acceptance-only certification: acceptance of a candidate s as a solution to a substituted task T' establishes $s \in \text{Sol}(T)$ for every acceptable s if and only if $\text{Sol}(T') \subseteq \text{Sol}(T)$, under soundness and completeness of the certifier for T' , and the values of the certifier do not determine the inclusion, which must be separately justified.

When $T \equiv_{\mathcal{I}} T'$ the inclusion holds trivially, so for a sound and complete certifier equivalence of tasks is sufficient for transfer. It is not necessary, since the inclusion suffices. In the statement-level model of Chapter 7, a task whose statement is stronger than the intended one is restricting and transfers, and a task whose statement is weaker is enlarging and does not, which is Proposition 16.3 read in that model: a theorem with a strengthened hypothesis is the weaker theorem. The favorable direction cannot be read off from acceptance.

Proposition 16.4 (Partial discharge, RV-016). *Suppose discharge certificates are sound, in that a discharged obligation has true content; that kernel acceptance is a discharge certificate for O_{formal} and that a true statement or argument obligation is dischargeable by some authority; that the class of instances contains those of Theorems 16.1 and 16.2 under a strict relation; and that discharge of each of the three baseline obligations O_{formal} , $O_{\text{statement}}$, O_{argument} requires a certificate of its own, so that no rule discharges one from another. Then discharge of O_{formal} does not entail discharge of $O_{\text{statement}}$ or of O_{argument} .*

Proof. By Theorem 16.1 there is an instance with $K(V)$ and $\neg C(V)$. The content of O_{formal} is true, so it can be discharged; the content of $O_{\text{statement}}$ is false, so by soundness it cannot be discharged. By Theorem 16.2 there is an instance with $K(V)$, $C(V)$ and $\neg A(V)$, in which the first two can be discharged and, by soundness, the third cannot. $\square$

The proposition is a consequence of the two preceding theorems and is recorded because it is the form in which the separation enters the ledger: it licenses recording different statuses for the three obligations of one event.

16.4 The gap is not computable at level two: kernel acceptance versus correspondence

The separation results above exhibit single instances. A stronger statement concerns whole families, and it is available because the source constructs a family in which kernel acceptance is uniformly trivial while statement correspondence, under the convention stated below, is Σ_2^0 -complete as a set of indices.

Fix $k \geq 9$ and a computable enumeration $(p_e)_{e \in \mathbb{N}}$ of integer-coefficient polynomials in $k + 1$ variables. Let $B_e(n)$ be the predicate that $p_e(n, x_1, \dots, x_k) \neq 0$ for every $(x_1, \dots, x_k) \in \mathbb{N}^k$, and let n_e denote the least n with $B_e(n)$ when one exists. The specification N_e introduces $r_e = 1/(n_e + 1)$ and asserts the identity $(r_e + 1)^2 = r_e^2 + 2r_e + 1$. The formal proposition F_e defines n_e as the infimum of $\{n : B_e(n)\}$ in a library whose infimum on the empty set is 0, so that F_e is defined for every e and, when no such n exists, assigns the default $n_e = 0$ and $r_e = 1$. The identity is a polynomial identity and the proof π_e closes by ring normalization for every e .

The proof of the theorem below needs one fact about the formal representation that kernel acceptance of π_e does not supply. The proof π_e establishes the polynomial identity for whatever value n_e takes, and it is silent on whether that value is the least n of the specification. That fact is a separate lemma.

Lemma 16.5 (Faithful denotation under nonemptiness). *Let $S_e = \{n \in \mathbb{N} : B_e(n)\}$. If $S_e \neq \emptyset$, then the formal term $\inf S_e$ denotes the least element of S_e , so that $\llbracket F_e \rrbracket_{\mathcal{I}}$ assigns to n_e the same natural number as $\llbracket N_e \rrbracket_{\mathcal{I}}$ and, consequently, assigns to r_e the same rational number $1/(n_e + 1)$.*

Proof. Every nonempty set of natural numbers has a least element, by well-ordering, and the infimum operation on subsets of $\mathbb{N}$ is characterized by the two properties that, for nonempty S , $\inf S \in S$ and $\inf S \leq s$ for every $s \in S$. These are the library facts that the formal development must cite, and in Lean’s Mathlib they are `Nat.sInf_mem` and `Nat.sInf_le`. The element of S that is below every element of S is unique, so $\inf S_e$ is the least element. The step from n_e to $r_e = 1/(n_e + 1)$ is the application of the same function to equal arguments. $\square$

The lemma is conditional on nonemptiness. For empty S_e the library convention assigns $\inf \emptyset = 0$, which is the silent default, and the lemma is silent. The names of the library facts are those of the source’s setting and are to be checked against the library version in use when the development is written.

Theorem 16.6 (Hidden existence). *Adopt the convention that a specification whose defining term is undefined is equivalent to no formal proposition. Then $K(V_e)$ holds for every e and is decidable uniformly in e , whereas $\{e : C(V_e)\} = d$, where $d = \{e : \exists n B_e(n)\}$. By Theorem A.3 of the source, d is Σ_2^0 -complete under many-one reductions. In particular no total computable procedure decides $C(V_e)$ uniformly in e , even with an oracle for the Halting problem.*

Proof. Kernel acceptance holds for every e because the identity is polynomial and its proof does not depend on n_e . This fact alone says nothing about correspondence, which is why Lemma 16.5 is needed below. Suppose $e \in d$. Then n_e exists and the specification N_e is defined. By Lemma 16.5 the formal definition denotes the same least n , and hence the same r_e , so the two denotations agree and $\llbracket N_e \rrbracket_{\mathcal{I}} \equiv_{\mathcal{I}} \llbracket F_e \rrbracket_{\mathcal{I}}$. Suppose $e \notin d$. Then N_e is undefined, while F_e is defined with value $n_e = 0$, so by the convention the denotations are not equivalent. The set of e satisfying $C(V_e)$ is therefore exactly d . The completeness of d is Theorem A.3 of

the source, proved there by reduction from the set of indices of non-total partial computable functions using the nine-unknown refinement of the DPRM theorem and Post's theorem. A Σ_2^0 -complete set is not decidable relative to $\emptyset'$ and so is not decidable. $\square$

The convention in the statement is not innocent and is stated openly. It identifies the question of correspondence with the question of well-definedness, and it is the stated position of the source that a faithful translation of an undefined term must refrain from assigning it a value. A reader who preferred a convention under which undefined specifications correspond to everything, or to a partial formal object, would obtain a different set, and the theorem would be correspondingly different. What the theorem shows is conditional in exactly this respect. Given the convention, there is a family of instances on which the kernel is trivially satisfied and the set of indices at which the correspondence judgment holds is Σ_2^0 -complete. The convention makes $\equiv_{\mathcal{I}}$ an equivalence only on the defined denotations, as recorded in Appendix A. A statement of the same kind at every level $l \geq 2$, using the alternating-quantifier constructions of Theorem A.4, would need a level- l analogue of this theorem for the total-default formalization, which I do not state here. Theorem 10.5 of Chapter 10 concerns the different question of well-definedness and trustworthy autoformalizers.

The theorem does not say that correspondence is undecidable for mathematical texts as they occur in practice. The family is constructed, the reductions are effective, and a restriction of the specification class to texts that do not embed the least-number construction may admit reliable checking. That qualification is the subject of RV-006 and the source establishes no decidability result for fewer than nine unknowns. The theorem should be read as showing that, on unrestricted specification classes of this kind, statement correspondence is not decidable even relative to $\emptyset'$, a level-two barrier, while kernel acceptance is uniformly decidable. Whether the barrier rises with the level, as it does for well-definedness (Theorem 10.5), is not established here for correspondence itself.

16.5 A candidate non-discharge principle

The results above suggest a general principle that the monograph adopts as a guide and does not claim as a theorem. For a verification system that reports acceptance, there is a nonempty set of obligations whose status cannot be determined from acceptance, and for sufficiently expressive specification classes no total computable procedure can determine all of them. The first half is proved by Theorem 16.1. The second half is proved on the hidden-existence family by Theorem 16.6, and its extension to arbitrary classes requires a reduction from the hidden-existence construction into the class, which must be exhibited class by class. The principle is therefore a research agenda with two proved instances, and the remainder of the part is concerned with what can be done about it. The answer developed in the next chapter is that the obligations that cannot be discharged can nevertheless be preserved.

Chapter 17

Preservation, Discharge, and the Transformation of Obligations

The preceding chapter showed that a successful verification leaves obligations that the kernel cannot discharge, and that for sufficiently expressive specification classes no total procedure can discharge them all. The question that follows is practical as well as theoretical. Verified artifacts do not stay as they are. They are translated, revised, repaired, specialized, combined with other artifacts, and imported into further developments. At each transformation the obligations attached to the input must be accounted for in the output. This chapter develops the central constructive notion of the monograph, which is the preservation of obligations through transformations, and separates it from the notion of discharge with which it is routinely confused. The formal definitions are those of Appendix A, and the theorems proved there are restated here with their interpretation and their limits.

17.1 Why traceability is not enough

A natural first proposal for tracking obligations through a transformation is to require that every obligation of the input be accounted for in the output: either it is recorded as discharged, or it has a successor. This is traceability, and it is mechanically checkable. It is also insufficient, and the insufficiency is the reason the rest of the theory is needed.

Suppose an input obligation asserts a bound requiring $m + 4$ derivatives of the data, and the transformation replaces it by a successor asserting the corresponding bound with $m + 5$. The input obligation has a successor, and the ledger is complete. But the successor is a weaker claim. A reader who inspects the ledger and finds every obligation accounted for will conclude that nothing has been lost, and that conclusion is false. What the ledger lacks is a record that the content of the successor suffices for the content of the original. Traceability records where an obligation went. Preservation records whether what it became is enough.

Accordingly the theory has two layers. The syntactic layer concerns the form of the ledger and is decidable by inspection. The semantic layer concerns entailment between contents and is certified, not computed. The two layers have separate theorems, and keeping them separate is the safeguard against the error just described, namely mistaking a well-formed ledger for a semantically adequate one.

17.2 Transitions, certificates, and the entailment condition

A transformation f_i from an artifact X_{i-1} to an artifact X_i is accompanied by a transition certificate $T_i = (Q_i, \Delta_i, \Theta_i)$. The relation Q_i links obligations of the input to their successors in the output. The set Δ_i lists input obligations recorded as discharged at the transformation, each with a certificate that is currently warranted in the ledger state preceding it. The set Θ_i lists the assumptions the transformation is permitted to use. The certified context of an input obligation o is the set consisting of the contents of its successors, the contents of the discharged obligations, and the permitted assumptions. The transformation *preserves* o when the certified context entails the content of o :

$$\text{Ctx}_i(o) \models_{\mathcal{I}} \varphi(o).$$

It is preserving when it preserves every input obligation.

Two features of this definition deserve emphasis. First, the successors enter as premises along with the discharged obligations and the assumptions. Nothing requires that any one successor entail the original, and a conjunction of weaker successors may jointly entail it. The test is applied to the total certified context, which is the reading of RV-019 given in Section 17.7. Second, the permitted assumptions are restricted. Without a restriction the condition is vacuous, because an assumption equal to $\varphi(o)$ would secure preservation of o for free. Definition A.11 of the appendix requires that each assumption either belong to the authorized context fixed with the instance or be the content of a tracked assumption obligation introduced by the transition, and that the dependency relation remain acyclic. An assumption introduced during a transformation therefore enters the residue and remains there until it is discharged. Preservation on the strength of an untracked assumption is not permitted, and preservation on the strength of a tracked one is permitted and honestly reported.

17.3 The typing of transitions

At each obligation the certified context stands in one of four relations to the original content, and the transition is typed accordingly. Write Ctx for the certified context of o . If $\text{Ctx} \models \varphi(o)$ and $\varphi(o) \models \text{Ctx}$, the transition is *conservative*: what was required is exactly what is now certified. If $\text{Ctx} \models \varphi(o)$ but $\varphi(o) \not\models \text{Ctx}$, it is *strengthening*: the original is preserved, and the context contains claims the original did not, which are a new burden to be discharged. If $\text{Ctx} \not\models \varphi(o)$ but $\varphi(o) \models \text{Ctx}$, it is *weakening*: everything now certified follows from the original, and the original does not follow from it, so content has been lost. If neither entailment holds, it is *lateral*: the transition has replaced the obligation by something incomparable. Conservative and strengthening transitions are preserving. Weakening and lateral transitions are not. The classification is exhaustive and exclusive by construction, and it is recorded at the level of individual obligations, with a transformation counted as preserving only if every obligation is handled by a preserving case.

17.4 Traceability: the syntactic theorem

A ledger is *well formed* when every transition records a disposition for every inherited obligation, that is, each input obligation is either in Δ_i or has at least one successor. For an obligation o of

the initial artifact, the successor sets $S_k(o)$ are obtained by composing the successor relations. The following is Theorem A.15 of the appendix.

Theorem 17.1 (RV-020). *In a well-formed ledger, every originating obligation either has a certified discharge in its ancestry or has a nonempty set of successors at the final artifact. Well-formedness and these two alternatives are decidable for finite ledgers by inspection of the successor relations and discharge records.*

Corollary 17.2 (RV-017). *In a well-formed ledger, an originating obligation with no certified discharge in its ancestry has a successor at the final artifact. No obligation leaves the ledger without a recorded disposition. A discharge that is not recorded as a member of some Δ_k lies outside the formalism, and the corollary does not exclude it.*

The proof is an induction on the length of the chain and is given in the appendix. The theorem is intentionally weak. It says that nothing disappears without record, and it says nothing about whether the recorded dispositions are adequate. Its value lies in being checkable even when the entailment relation is not. A Lean formalization of the ledger structure can verify well-formedness and compute the ancestry sets without any commitment about the semantics of the contents. The result is therefore the part of the theory that can be machine checked today. It is also the part most likely to be mistaken for the whole, which is why the next theorem is stated alongside it.

17.5 Compositional preservation: the semantic theorem

Theorem 17.3 (RV-008). *Let $T_1, \dots, T_n$ be transition certificates over a consequence relation satisfying reflexivity, monotonicity, and cut, and for o an obligation of the initial artifact let each T_k be preserving at every obligation reached from o . let $\Gamma_n(o)$ be the set consisting of the contents of the obligations in $S_n(o)$ together with all discharged contents and permitted assumptions accumulated along the chain. Then $\Gamma_n(o) \models_{\mathcal{I}} \varphi(o)$.*

The proof, given in the appendix as Theorem A.17, is an induction on n . In outline: at each step the context of any intermediate obligation is contained in the cumulative context, so by monotonicity the cumulative context entails the content of every intermediate obligation; the cumulative context of the previous step is thereby entailed element by element, and cut transfers the earlier entailment of $\varphi(o)$ to the new cumulative context.

The theorem is conditional, and the conditions are visible in its statement. It requires preservation at the obligations reached from o and the three structural properties of the consequence relation. It does not require that the consequence relation be decidable, and it does not assert that any premise is true. This last point is what makes the entailment a result about preservation and not about truth. Admissibility of the certificates, soundness of the certificates, and soundness of the consequence relation for the interpretation enter only the second statement of the appendix theorem, which concerns establishment. The premises of the entailment are the contents of the composite successors $S_n(o)$, the contents of the obligations recorded in discharge sets along the chain, and the permitted assumptions. If all of these are established at the evaluation time and the soundness hypotheses hold, then $\varphi(o)$ is transported, in the appendix's sense that it holds under the interpretation. A member of $S_n(o)$ that is not warranted-discharged belongs to the residue of the final artifact. If the content of a recorded

discharge or a tracked assumption is not established, its source obligation is not warranted-discharged and belongs to the transport shortfall, and to the residue of the final artifact only if the ledger carries it forward; a defeated authorized assumption has no source obligation and lies in no residue. In these cases $\varphi(o)$ has been preserved and has not been transported.

17.6 Preservation is not discharge

The independence of the two notions is the contribution that the monograph regards as central, and it is stated as a proposition rather than left as a slogan.

Proposition 17.4 (RV-018). *Preservation of an obligation need not entail its discharge, and discharge of one obligation need not preserve another. An obligation recorded in the discharge set of a transition is preserved at that transition.*

The appendix proves the first two parts by explicit ledgers. For the first, a chain in which each step replaces an open obligation by an identical open successor is preserving at every step by reflexivity and discharges nothing. For the second, a transition that discharges one obligation and leaves another with a successor too weak to recover it is well formed and non-preserving at the second, so discharge of the first supplies a premise and does not supply the content missing from the second. A third part records that an obligation recorded in the discharge set of a transition is trivially preserved at that step, because its content belongs to its own certified context.

The first half has a corollary that gives the monograph its title in miniature.

Corollary 17.5 (RV-009). *A verified continuation can retain unresolved originating obligations.*

A system can therefore be rigorous about an obligation it has not settled. It can carry a statement correspondence obligation through a sequence of translations and repairs, certify at each step that nothing has been lost, and arrive at the final artifact with the obligation exactly as open as it began. This is a precise sense in which a verification pipeline may be sound without being complete: it never reports as settled what it has only preserved. The value of such a pipeline is that the final report is honest about the residue, and a reader may decide whether the residue is acceptable.

17.7 Weakening: the Navier-Stokes cases

The preservation condition is a condition on the total certified context. A successor that is strictly weaker than its predecessor does not by itself make a transition non-preserving, because other successors, discharged obligations, or admissible assumptions may supply what the weaker successor lacks. Conversely a transition with a single weaker successor, no discharges, and no assumptions is non-preserving. This is Proposition A.20 of the appendix, and it is the formal statement of RV-019.

The derivative case of Chapter 12 is the paradigm. Let o be the obligation whose content is the natural-language estimate (8.19) of the source,

$$\|N^{-1}F\|_{C_y^m} \leq C_m \|F\|_{C_y^{m+4}},$$

and let o' be the obligation whose content is the Lean estimate

$$\|N^{-1}F\|_{C_y^m} \leq K_m \|F\|_{C_y^{m+5}},$$

cited at the pinned commit as `norm_derivativeWord_inverse_le`. Since $\|F\|_{C_y^{m+4}} \leq \|F\|_{C_y^{m+5}}$, the first bound implies the second with $K_m = C_m$, so $\varphi(o) \models \varphi(o')$. The source reports the second as strictly weaker, that is, as not implying the first. With $\Delta = \Theta = \emptyset$ and o' the only successor, the certified context is $\{\varphi(o')\}$ and the transition from the natural-language paper to the formalization is of weakening type at o . The classification depends on the failure of the converse entailment, which is the source's claim, and the monograph records it as such pending independent verification of that claim. The entailment relation here is derivability from the certified background of the development and not classical consequence over truths, as Chapter 12 explains, and without that reading the classification would be vacuous. The classification also depends on the premise set. If the formalization contained a further certified lemma deriving the $m + 4$ bound, the context would change and the type with it. The type is a statement about the whole context, and the example is accordingly evaluated against the full set of premises available in the formalized development and not against one declaration in isolation.

The pressure-flux case of Chapter 13 is a different type. The natural-language estimate (10.19) bounds the pressure-flux integral in terms of B_R alone, and the Lean estimate bounds it in terms of B_R and also A_R . The source reports that the two are not directly comparable, since equation (10.17) bounds B_R using A_R but cannot be used to eliminate A_R from the Lean estimate to recover (10.19). On that report neither content entails the other, and the transition is classified as lateral. The quantifier on time also differs: the natural-language estimate is for almost every t and the Lean estimate for every t (Chapter 13), which together with the extra dependence on A_R is what makes the pair incomparable and not merely ordered. The classification is source-supported and provisional: the incomparability is the source's assessment of the two bounds, and no countermodel or analytic argument establishing both non-entailments is supplied in this monograph. The same holds for the converse non-entailment in the derivative case above. The example matters because it shows that the typing is not a ranking by strength. A discrepancy between a natural-language lemma and its formalization need not be a loss of content in the sense of weakening, and the framework represents the difference without flattening it into a generic mistranslation. As in the derivative case, no conclusion about the correctness of the full natural-language proof is drawn, in keeping with the source's own disclaimer: the Lean results do not refute the natural-language estimates, and they fail to certify them.

17.8 Propagation through dependency structures

A formal development is a graph of declarations, each depending on definitions and lemmas that precede it. Correspondence obligations attach to individual declarations: the definition of a quantity corresponds, or does not, to the notion in the source. The question of RV-013 is how such obligations propagate along the dependency graph.

Say that a declaration x *semantically depends* on declarations $y_1, \dots, y_k$ when there are maps g_x on denotations and g_x^b on formal counterparts with $\llbracket x \rrbracket_{\mathcal{I}} = g_x(\llbracket y_1 \rrbracket_{\mathcal{I}}, \dots, \llbracket y_k \rrbracket_{\mathcal{I}})$, the formal counterpart of x being obtained by applying g_x^b to the formal counterparts of the y_j .

A *congruence certificate* for x is an accepted certificate establishing that this construction preserves correspondence under the intended interpretation: whenever the formal counterpart of each y_j corresponds to y_j , the formal counterpart of x corresponds to x .

Proposition 17.6 (RV-013, conditional propagation of correspondence). *Let x semantically depend on $y_1, \dots, y_k$. (i) Suppose the correspondence obligation of each y_j is warranted-discharged, and a congruence obligation c_x , of dependency type with the content that correspondence of all the y_j implies correspondence of x , is warranted-discharged on the strength of an accepted congruence certificate. Then the correspondence obligation of x may be discharged by a certificate whose dependencies are c_x and the correspondence obligations of the y_j , and that discharge is currently warranted as long as c_x and each y_j remain warranted-discharged and no defeater targets the new certificate. If the new certificate is the only one for x , the obligation of x is warranted-discharged exactly then. (ii) Suppose some y_j has a correspondence obligation in the residue, and no invariance or bypass certificate establishes correspondence of x independently of y_j . Then this dependency-based route does not justify discharge of the correspondence obligation of x .*

Proof. (i) The congruence obligation c_x has the content of a conditional: correspondence of every y_j implies correspondence of x . Its antecedent is supplied by the discharged obligations, and the conclusion follows by modus ponens, the new discharge having c_x and the obligations of the y_j as its dependencies, which lie in the allowed set of obligations and authorized assumptions. By Definitions A.7 and A.9 its warrant is the conjunction of theirs together with the absence of a defeater on the certificate itself, and it is lost if any of them loses warrant. (ii) The route consists of the congruence certificate together with the correspondence of all the y_j . If one antecedent is not established, the conditional cannot be detached, and no other premise of the route bears on x . $\square$

Discharging the correspondence obligations of all dependencies is not sufficient by itself, and the congruence certificate is therefore an explicit hypothesis of (i). A declaration constructed from faithful parts may fail to be faithful if the construction does not preserve the relevant semantic relation, for example if it applies a faithful definition under a quantifier whose range differs from the source's. The second clause is deliberately weaker than the statement that x cannot be faithful. An unresolved dependency may turn out to be irrelevant to x , either because the denotation of x does not in fact vary with it or because x can be shown to correspond by a route that does not use it. That irrelevance is a further claim, and it is recorded by an invariance or bypass certificate, which is itself an obligation. The proposition says that the dependency graph cannot be used as evidence of correspondence when a dependency is unresolved, and that the residue of x includes, in the sense of a recorded route, the residue of the dependencies it semantically uses. A single silent default in a definition, as in the hidden-existence family of the preceding chapter, thereby enters the residue of every statement that uses the defined quantity unless an invariance certificate removes it.

17.9 Optional information and the link to admissible degradation

The earlier work on admissible degradation concerns continuation under loss: a process may drop information without invalidating what it continues to do, provided what is dropped is not

what justifies the continuation. The preservation condition provides the formal counterpart for obligations. A transformation may drop content that no originating obligation requires, which in the present vocabulary is optional content, and such a transformation can still be conservative or strengthening at the obligations that matter, since preservation is assessed against the contents of those obligations. It may not silently discard an obligation that justifies the authority of the continued artifact, which is the case in which a weakening or lateral transition is mislabeled as preserving. The distinction between permitted and forbidden loss is thereby reduced to the entailment condition, with the set of originating obligations playing the role of the authority-bearing content.

17.10 What remains

The principal mathematical work that remains concerns the certificates themselves. The theorems above are conditional on the soundness of the certificates that witness entailment and discharge, and the theory does not say when a certificate justifies what it claims. Three questions are open. The first is how to certify an entailment between natural-language contents and formal contents, where the consequence relation is relative to an interpretation that is itself informal. The second is how to certify that a refinement of a specification, such as the introduction of a more explicit definition, preserves the contents that depended on the original. The third is how the interpretation is itself allowed to change along a chain, which the present formalism holds fixed and which the audit section of the appendix records as a limitation. These questions are the subject of the constructive proposals of Part VII, and the Lean prototype described in Chapter 21 is designed to test whether recording obligations and transition certificates improves the reliability of certification claims. The criterion for that test is narrow. It is whether known unresolved obligations are preserved or detected and are not reported as settled.

Chapter 18

Epistemic Immutability and the Provenance of Proof

A proof artifact changes. Statements are corrected, arguments are repaired, definitions are replaced, and libraries are updated beneath a development that was accepted last month. Mathematical improvement of this kind is the ordinary life of a formal development and is not a defect. A defect arises when the history of an improvement is represented as evidence about what was true before the improvement. The correction of an error is evidence that the corrected version is better, and it is evidence about the earlier version only in the negative sense that the earlier version needed correction. The chapter proposes a provenance model in which this distinction is structural, so that it cannot be blurred by an editorial choice, and states the invariant that the model secures.

18.1 Versions, events, and the append-only ledger

The model separates the objects that a verification event involves. The *source text* is the natural-language specification and argument as originally announced. The *formalization* is the formal proposition submitted for checking. The *proof* is the term accepted by the kernel. A *revision* is a later formalization or proof offered in place of an earlier one. A *correspondence judgment* is a human or procedural act asserting that a formal object corresponds to a source object. Each of these is a *version*, identified by a content hash, and each is attached to the obligations of Chapter 15.

Definition 18.1 (Ledger). *A ledger is a finite sequence of events $e_1, e_2, \dots, e_N$. Each event has a kind (introduction of a version, introduction of an obligation, acceptance of a certificate, recording of a defeater, supersession of one version by another, retraction), the identifiers of the versions and obligations it concerns, an authority, a timestamp, and the hash of the preceding event. The ledger is append-only: events are added at the end and no event is modified or deleted. The state at time t is the function of $e_1, \dots, e_t$ that assigns each obligation its current status and each certificate its historical and warranted standing.*

Supersession is an explicit event. When a version v' replaces a version v , the ledger records an edge from v' to v and a transition certificate in the sense of Appendix A stating which obligations of v have successors in v' , which are discharged, and which assumptions are used.

Without the certificate the supersession is recorded as unjustified, and the obligations of v are treated as having no recorded disposition, which the well-formedness condition of Theorem A.15 exposes.

18.2 Correction and retrospective certification

Two operations are easily confused and the model separates them.

Correction replaces a version by another and certifies the new one. The certificates attached to v' are about v' . The obligations of v remain in the ledger with their statuses at the time of supersession, so that if the argument of v was found invalid, the argument obligation of v is recorded as refuted and stays refuted.

Retrospective certification attaches to v a certificate whose validity depends on v' , and presents the result as evidence that v was sound. The operation is not illegitimate in every case, since a lemma proved later can legitimately discharge an obligation raised earlier, but it must be recorded as what it is: a discharge of an obligation of v whose dependency set includes a certificate about v' , with a timestamp later than the introduction of v . The historical record then shows that the obligation was open until the later event.

Proposition 18.2 (RV-012, ledger form). *In an append-only ledger, for every version v and every time t , the historical record of the obligations of v up to t is determined by events $e_1, \dots, e_t$ and is unchanged by any later event. In particular no event concerning a later version v' alters the status history of the obligations of v . A discharge of an obligation of v at time s is a new event at s , and the interval before s is recorded with the earlier status.*

Proof. The state at time t is a function of the prefix $e_1, \dots, e_t$ by definition. Events after t are appended and do not change the prefix. The status history of an obligation of v at times $\leq t$ is read from that prefix. A discharge at s is an event e_s and affects statuses at times $\geq s$ only. $\square$

The proposition is definitional, a property of how the ledger is built, and it is stated because the model is useful only if the invariant is explicit. Its content is that a correction cannot be passed off as a historical fact: the ledger has no operation that rewrites the earlier status.

18.3 A worked case

The polynomial example of Chapter 1 illustrates the distinction. Version v_1 consists of the stated theorem, the supplied natural-language argument with the incorrect factorization, and no formal proof. The translation produced by a language model is version v_2 : a Lean proof that compiles, found by correcting the multiplicities. Its obligations are the three baseline ones. The transition from v_1 to v_2 is preserving at the statement obligation, since the formal proposition expresses the theorem, and it is not preserving at the argument obligation. Since v_1 has no proof term, the argument obligation of Appendix A (correspondence of P and π) is not yet defined for it, and I use an obligation of argument-validity type, whose content is that the supplied argument is valid. That obligation is refuted, and the transition certificate contains no successor for it that carries its content, because the argument of v_2 is a different argument. The ledger therefore records: the theorem is established by v_2 ; the original argument is refuted;

the formal proof does not formalize the original argument. Each of these is true, and a report that said only that the translation succeeded would conflate them.

The mathematical value of the repair is not in question. Finding a correct proof where the supplied one was wrong is useful. What the model prevents is the inference that the original argument has been vindicated by the existence of a verified proof, an inference that the acceptance loop of Chapter 14 invites.

18.4 Supersession, retraction, and independent certification

Three further operations complete the model. *Supersession* records that v' replaces v and carries a transition certificate. A superseded version remains in the ledger and its obligations keep their statuses. *Retraction* withdraws a certificate or a version. It is recorded as a defeater event, so the obligations that depended on the retracted certificate lose their warranted standing by Proposition A.8 and revert in status, while the history retains the fact that they were once discharged. *Independent certification* is a second certificate for an obligation already discharged.

Definition 18.3 (Independence). *Two certificates c, c' for the same obligation are independent when their authorities are distinct and the dependency closures $\text{dep}^*(c)$ and $\text{dep}^*(c')$ in the dependency graph of Definition A.6 are disjoint apart from the authorized context.*

Independence is a structural property of the dependency graph and is not a claim about the honesty or competence of the authorities. Its use is that, by the obligation-level rule of Definition A.9, an obligation with two independent certificates loses warrant only if both lose it, whereas a discharge supported by two certificates that share a dependency loses warrant when the shared dependency is. Two checks by the same kernel on the same library are not independent in this sense, and the ledger records them as correlated. This is the formal reading of the common intuition that a second opinion must be genuinely second.

18.5 Tamper evidence

The ledger form of RV-012 is a property of the data model. A stronger statement is available when the ledger is stored in a medium that can be altered. Chain each event to its predecessor by a cryptographic hash, so that the hash of e_k is a function of its content and of the hash of e_{k-1} .

Proposition 18.4 (RV-012, tamper-evident form). *Assume the hash function is collision resistant. If an event e_k with $k < N$ is altered after the head hash h_N has been recorded by an independent party, then the recomputed chain no longer yields h_N , except with negligible probability.*

Proof. Altering e_k changes its content and so, by collision resistance, its hash. The hash of e_{k+1} takes that hash as an input and so changes in turn, and likewise for each later event, so the recomputed head differs from h_N unless a collision occurs at some link. $\square$

The proposition is conditional on collision resistance and on the independence of the party that holds h_N , and these are assumptions of the tamper model, not facts the monograph establishes. It establishes the limited claim that, under the model, retrospective alteration is

detectable. It does not prevent alteration, and it does not address the case in which the entire ledger and its head are replaced consistently before any independent party has recorded the head.

18.6 Provenance in practice: two training reports

Two recent reports on model training, neither concerned with proof, practice the discipline that the model formalizes, and they serve as worked examples. Both are preprints, and their figures are reported as the authors give them, without reproduction.

Chen et al. [4] (arXiv:2610.10426) study the joint improvement of a language-model policy and the runtime harness that formats its prompts and handles its errors. Their central observation is that the value of an execution trajectory for training depends on the harness under which it was generated, and so a trajectory is recorded with a fingerprint that hashes the prompt template, tool bindings, and processors it ran under, and not merely a harness identifier. In the ledger vocabulary the fingerprint is part of the provenance ρ of the evidence, and the practice is the requirement that evidence be used only under conditions matching those of its production, which is the condition for transferring a certificate from one environment to another. They compare a recipe that pools successful trajectories from sibling candidate harnesses with a recipe that keeps only trajectories whose fingerprint matches the adopted harness. Under identical search and promotion protocol, the pooled recipe supplies 149 to 308 trajectories per iteration and yields no accepted model update in the 9B-parameter comparison, while the matched recipe supplies 30 to 50 and yields two accepted updates totaling +6 solved tasks. The authors take care over what the comparison does not show: the recipes differ jointly in several choices, so the table compares complete recipes, and the prompt-completion pair counts of the two recipes overlap.

The same report separates attribution by a promotion rule. A candidate harness is evaluated with the policy held fixed, and a candidate policy with the harness held fixed, on a held-out promotion split whose task identifiers are verified disjoint from those used to generate candidate updates, and a candidate is adopted only if the number of newly solved tasks exceeds the number newly broken. In the vocabulary of this chapter the promotion split is a source of certificates independent of the evidence that produced the candidate, in the sense of the definition of independence above, since their dependency closures are disjoint, and the rule records which component each accepted gain is credited to. The accepted gains, cumulated per component, are a ledger of the form that Section 18.1 describes, with an entry for each decision and the rejected candidates retained.

Cheng et al. [6] (arXiv:2610.02462) make a distinction that is the ledger’s distinction between prospective and retrospective certification. Their predictive relations are fitted on development states and judged on unseen ones, and they state that each prediction keeps the status it was given whatever the outcome. They mark which comparison forms were pre-specified before a test and which are retrospective refits, and they mark with a dagger a selection made after the test results were seen. A prospective prediction that succeeds is evidence of the relation. A retrospective refit that matches is evidence only that the form can be fitted, and the reader is told which is which. The practice is the one the ledger enforces structurally: the record of when a claim was fixed is part of the claim, and a later event cannot move it.

18.7 What the model does not do

The model does not prevent mathematical improvement, and it does not require that old versions be preserved in a form that remains usable. It requires that the claims made about old versions remain reconstructible. A library can be updated beneath a development, and when that happens the certificates that depended on the old library lose warranted standing, and the ledger shows which. The concern is not to freeze mathematics. It is to prevent the history of an improvement from being misrepresented as evidence of earlier correctness, and to make that prevention a property of the record and not of the diligence of whoever writes the report.

Part VI

Objectives, Optimization, and Institutional Scale

Chapter 19

Reward Functions and the Production of Substitution

The preceding parts concern verification of an artifact after it exists. This chapter concerns the process that produces artifacts when that process is optimized against a verifier. In reinforcement learning on mathematical reasoning, a reward is computed by checking the final answer, sometimes the steps, and in formal settings the acceptance of a proof by a kernel. A system trained against such a reward is shaped by what the reward distinguishes, and the question of the chapter is what a reward can distinguish about fidelity to the task. The starting point is first-person testimony of Daniel Ketterer [11], who trains language models on mathematical reasoning and reports having watched models make errors that cancel and still be rewarded, and misrepresent a problem, solve the different problem correctly, and be rewarded. That account is used here as testimony about failure modes. The two public events that Ketterer cites, an Erdős-conjecture result and a breach at Hugging Face, are specific historical claims that the monograph does not rely on until they are independently verified.

19.1 Reward as an equivalence relation

Let H be a set of reasoning trajectories, $R : H \rightarrow \mathbb{R}$ a reward function, and $Q : H \rightarrow \{0, 1\}$ a predicate indicating that a trajectory satisfies the obligations of the task. The reward induces an equivalence relation, $h_1 \sim_R h_2$ when $R(h_1) = R(h_2)$.

Theorem 19.1 (RV-010, factorization). *Task satisfaction factors through the reward, meaning $Q = g \circ R$ for some $g : \mathbb{R} \rightarrow \{0, 1\}$, if and only if $R(h_1) = R(h_2)$ implies $Q(h_1) = Q(h_2)$ for all $h_1, h_2 \in H$. If the condition fails, no classifier that depends only on the scalar reward decides Q correctly on every trajectory.*

Proof. If $Q = g \circ R$ then equal rewards give equal values of Q . Conversely if the condition holds, define $g(r)$ as the common value of Q on $R^{-1}(r)$, which is well defined, and $Q = g \circ R$. If the condition fails, choose h_1, h_2 with $R(h_1) = R(h_2)$ and $Q(h_1) \neq Q(h_2)$; any classifier of the form $g \circ R$ gives the same output on both and is wrong on one. $\square$

The theorem is elementary. Its force comes from its reading: the reward signal considered alone lacks the information needed to decide the task predicate whenever two trajectories with the same reward differ in whether the task was satisfied. It does not show that training cannot

favor the correct trajectory. A training process has access to more than the reward of a single trajectory, including the distribution of trajectories and the structure of the model, and may in practice drift toward correct ones. The theorem is a statement about what the reward value contains.

A quantitative companion measures how much is lost. Let μ be a probability measure on H with R taking countably many values.

Proposition 19.2 (Irreducible error of reward-based classification). *The minimum, over all functions g , of the probability under μ that $g(R(h)) \neq Q(h)$ equals*

$$\varepsilon(R, Q) = \sum_r \min(\mu(R = r, Q = 1), \mu(R = r, Q = 0)),$$

attained by taking $g(r)$ to be the majority value of Q on $\{R = r\}$. It is zero if and only if $Q = g \circ R$ holds μ -almost surely.

Proof. For each reward level r , the classifier errs on the mass $\mu(R = r, Q \neq g(r))$, which is minimized by choosing $g(r)$ as the value of Q carrying the larger of the two masses, leaving the smaller. Summing over levels gives the stated minimum. It vanishes exactly when for each r one of the two masses is zero, which is almost sure factorization. $\square$

The quantity $\varepsilon(R, Q)$ is a property of the reward and the task, not of any learner. A nonzero value says that no function of the reward can agree with the task predicate on all but a vanishing set of trajectories, and a small positive value says that the failure is rare but irreducible. It is the quantitative form of the observation that the verifier is, in training, the objective.

19.2 Correct answers do not identify correct reasoning

The related result RV-003 concerns outcome-based rewards, in which R depends only on the final answer.

Proposition 19.3 (RV-003). *Let R depend only on the final answer. Then any two trajectories with the same final answer receive the same reward, and hence, if one is a valid solution and the other is not, R does not factor the task predicate.*

The construction of such pairs is elementary. A trajectory that computes $2 + 3$ as 5 and one that computes $2 \cdot 3 - 1$ as 5 share the answer and differ in validity. More to the point, a trajectory may restate the problem $x^2 = 9$ with $x > 0$ as $x^3 = 27$ and solve the latter correctly. Every step after the restatement is valid for the problem the model actually solved, the final answer 3 is correct, and the reward is positive. The only flawed step is the restatement, and recognizing that it is flawed requires knowing what the original problem meant. This is the substitution problem of Chapter 7 observed from the side of training, and it is not repaired by checking steps.

Process-based rewards check each step against the preceding one. They fail on substitution in the same way, for a structural reason. A step reward is a predicate $V(s_i \mid s_{i-1})$, and the restatement step s_1 is judged relative to its predecessor, which is the problem as the model represents it. If the predecessor is itself the substituted problem, then every step including

the restatement passes. The restatement is checked against the original problem only if the original is supplied to the step verifier, which makes the step verifier a statement-correspondence checker, with all the difficulties of Part III.

Proof-checker rewards, in which $R(h) = 1$ exactly when the kernel accepts, instantiate Theorem 16.1 directly: K is blind to the originating specification, so R is constant on instances that differ in whether the statement corresponds. The proof-checker reward is therefore an objective that rewards verified formal statements, and its quality as a proxy for the task is exactly the quality of the formalization step that precedes it.

19.3 The pipeline as a chain of transformations

The reward is one link in a longer chain. Write

$$T \xrightarrow{S} S(T) \xrightarrow{R} R(S(T)) \xrightarrow{\text{opt}} \pi_\theta,$$

where T is the intended task, $S(T)$ its operational specification, $R(S(T))$ the reward built from the specification, and π_θ the learned policy. Each arrow is a transformation in the sense of Chapter 17, and the obligations of the task are carried or lost at each. A faithful specification paired with a defective reward is a failure at the second arrow. A correct reward exploited by a policy that finds unintended ways to maximize it is a failure at the third. A policy that behaves reliably in training and differently under deployment is a failure of a later transformation from training conditions to deployment conditions. They are three different failures located at three different arrows, and the preservation theorem of Chapter 17 applies to the chain as a whole: if each arrow carries a certificate and is preserving, the task obligation is preserved to the policy, and if any arrow lacks one the task obligation is preserved to that point and the chain beyond it records the gap as residue.

19.4 Objective admissibility and task admissibility

The pipeline view separates two notions that review of the objective tends to merge. *Objective admissibility* concerns whether the optimization criterion is acceptable: whether the specification and reward, reviewed before training, express the intended class of tasks. It is a property of the first two arrows over a distribution of problems. *Task admissibility* concerns whether a particular operation performed by the trained system preserves the obligations of a particular task. It is a property of an individual transition from a novel problem T_x to the representation the model constructs for it at inference time.

A certificate for the first is a universal statement over the training distribution D : for every $x \in D$, the transition $T_x \rightarrow S(T_x) \rightarrow R(S(T_x))$ is preserving. A certificate for the second is a per-instance statement about a problem that may lie outside D . The first does not give the second. A universal statement over D says nothing about $x \notin D$, and the model's interpretation of a new problem is built during inference, where no reviewed objective intervenes. A model may substitute a nearby problem without any defect in the reward definition. The first admissibility does not give the second, as just argued. The converse, that task admissibility can hold for a given instance while the objective is inadequate, is plausible, since a particular transition can preserve its obligation under a defective objective, but I do not give a witness here and do not claim it as a result.

Ketterer proposes concentrating human understanding where the objective is set, on the grounds that objectives are few and traces are many. The proposal is sound as an allocation of attention and addresses the first admissibility. It does not address the second, and the pipeline view shows why the second cannot be delegated to the first: the arrows at which the second admissibility operates are the ones that occur after the review.

19.5 Reviewability and adequacy

A further distinction concerns what review can accomplish. An objective is *reviewable* when a person can read its definition at reasonable cost. It is *adequate* when it rewards only legitimate solutions across the distribution of possible problems. The two are different, and the second is not implied by the first. A scorer that checks whether a final expression equals a reference value is short to read. Establishing that it rewards only legitimate solutions requires reasoning about every way a sufficiently capable optimizer might satisfy it, and the length of the definition is no measure of the effort.

The point is the one made about the trivial identity of Chapter 9: syntactic simplicity is not evidence of semantic adequacy. In ledger terms, reviewability is the condition that the content of the objective obligation can be stated, and adequacy is the condition that the obligation can be discharged. A reviewed objective that has not been shown adequate is an obligation in the residue, with status **supported** at best, and a human review is a certificate with an authority and a dependency set, which can be defeated if the optimizer finds an exploit the reviewers did not anticipate. The review is a contribution to governance and is not a once-and-for-all solution.

19.6 Illustrations from training reports

The structure analyzed in this chapter appears, in a milder and fully documented form, in recent reports on the training of language models. The reports below are not studies of mathematical fidelity, and none of them concerns a proof assistant. They are used here as worked examples of three structural features: an objective whose value is blind to a component of the task, a proxy that disagrees with the task predicate, and a training distribution that differs from the deployment distribution. Every figure is as reported by the authors of preprints that the monograph has read but not reproduced, and none of the claims is independently verified.

An objective blind to omissions

Tang and Yang [21] (arXiv:2610.09639) study on-policy distillation in a controlled synthetic setting that separates factual knowledge, represented by random lookup tables, from compositional skill, represented by the order in which known facts are combined. A student S_0 is distilled from teachers that differ in what they add: held-out facts (T_f), branching-tree composition (T_s), or neither ($T_\emptyset = S_0$). Averaged over four models, students distilled from T_s reach 79.10 percent greedy accuracy on unseen tree structures against 0.72 percent under the control, while students distilled from T_f reach 13.12 percent on single-step queries about held-out facts and 0.54 percent on chains of such facts. Replacing reverse KL by forward KL substantially restores factual transfer. On natural-language benchmarks with Qwen3-4B-Base the same asymmetry appears: the initial student scores 11.25 on recent factual questions and 17.41 on competition

mathematics, and after distillation from a reasoning teacher the scores are 10.75 and 23.88, while after distillation from a knowledge teacher, whose own score on the factual benchmark is 84.55, they are 9.64 and 19.40.

The authors attribute the asymmetry to the direction of the divergence. Reverse KL evaluates discrepancies under the student’s own distribution and therefore penalizes the student only on tokens it already produces with substantial probability. The relevant elementary fact can be stated exactly. If the teacher assigns probability $p > 0$ to a token and the student assigns it probability ϵ , the contribution of that token to the reverse divergence is $\epsilon \log(\epsilon/p)$, which tends to 0 as $\epsilon \rightarrow 0$, whereas its contribution to the forward divergence is $p \log(p/\epsilon)$, which grows without bound. The statement concerns the term for the omitted token and not the whole objective, since probability mass that the student places elsewhere is penalized, but it shows what the reverse objective contributes about an omitted target: almost nothing.

In the terms of Theorem 19.1, the training objective is a reward whose level sets contain students that differ in whether they have acquired the held-out fact. The two benchmark scores in the final table also show why a single aggregate would mislead. A student distilled from the reasoning teacher improves by 6.47 points on one benchmark and changes by -0.50 on the other, so a pooled score would register a gain that is attributable to one capability only. The controlled design is what makes the two task predicates separately measurable, and the report is an instance of the practice recommended in this chapter: the predicates are separated by construction and not inferred from a total.

A proxy that disagrees with the task

Cheng et al. [6] (arXiv:2610.02462) predict the capability loss caused by compression, and they measure loss as token-weighted cross-entropy on reference completions. They record that task accuracy and execution success are evaluated separately from this continuous endpoint. In their selection experiments they report that termination and the evaluation target are separate factors: a supervised end marker restores termination and raises the exact-match score of one student, while for the Pythia models the disagreement between loss and exact match persists, so that a selector must state whether it optimizes loss or exact match. This is a documented case in which a proxy and a task predicate rank candidates differently, and the authors respond as the framework recommends, by requiring the choice of objective to be stated and not leaving it implicit in the metric.

A training distribution that differs from deployment

Wang et al. [23] (arXiv:2610.09346) motivate OnlineQAT by an observation about the transition from training to deployment. Existing recovery methods for low-bit quantized models are optimized on fixed completions or teacher-generated answers, whereas the deployed quantized model conditions on prefixes that it generates itself, and quantization errors can move the model into states that are absent from the offline recovery data. In the pipeline of Section 19.3, this is a failure of preservation at the arrow from training conditions to deployment conditions: a certificate that holds on the prefixes visited in training does not cover the prefixes visited at deployment. The remedy proposed, training on the student’s own sampled prefixes, is an attempt to enlarge the domain on which the training certificate holds, and not a discharge of the obligation for states outside it.

19.7 Design implications

Three implications follow for the design of training objectives, none of which is a theorem. Rewards that depend on more than the final answer reduce the irreducible error of Proposition 19.2 only if the additional information discriminates trajectories that differ in Q , and a measure of that discrimination, the conditional impurity within reward levels, can be estimated on labeled samples. Statement-level checks, which compare the restatement with the original, must be treated as correspondence checks with the limits of Chapter 10, so the architecture should expect abstention and record it. And reporting of training outcomes should separate the reward obtained from the status of the task obligation, as the ledger does, so that a high reward is not reported as evidence of fidelity.

Chapter 20

Mathematical Publication Under Conditions of Abundance

The results of the preceding parts concern single verification events. This chapter asks what happens to the verification residue when the number of events grows faster than the capacity of anyone to interpret them. The question is institutional, and the chapter treats it with the same discipline as the rest of the monograph: the formal content is stated as what it is, an accounting argument, and the empirical reports on which the discussion relies are marked as reports at one remove.

20.1 Reports at one remove

Section 5.1 of Bastounis, Circelli, and Hansen [3] reports several developments of 2026, and the chapter was written without reviewing any of the primary sources. A later reading of some of them is recorded in Appendix E, which confirms the Meta paper’s headline figures and the two quoted passages of the declarations and leaves the community thread, the Fermat’s Last Theorem report and the attribution of the September declaration unverified. Everything not confirmed there is recorded here as the source’s account, and nothing in the argument below depends on their accuracy. The source reports that in spring 2026 Meta claimed to have translated 26 mathematics textbooks into Lean, yielding about 45,000 verified declarations and roughly 500,000 lines, citing Rammal and coauthors [18], arXiv:2605.29955. It reports that the claim was disputed in a Lean community Zulip thread, quoting Jack McCarthy, Brian Nugent, and Christian Merten, and that the project’s semantic verification was mainly performed by other AI systems. It further cites a formalization of Fermat’s Last Theorem attributed to Anthropic at 13 million lines of Lean [5], the Leiden Declaration of June 2026, and a September 2026 declaration titled “A Severe Misalignment of AI in Mathematics” initiated by 25 Fields medalists. Before any of these is relied on in an argument, the Meta paper, the Zulip thread, and the two declarations must be read in the original. Until then they serve only to indicate the scale of the phenomenon the chapter analyzes, and every figure above is the source’s report of the cited material.

20.2 The accounting argument

Let artifacts be produced over time, and let each artifact X carry a finite set of semantic obligations, those of Chapter 15 that are not settled by the kernel, namely the statement, argument, dependency, and interpretive obligations. Write $a(t)$ for the number of semantic obligations introduced up to time t and $b(t)$ for the number discharged with currently warranted discharge. The residue of the collection at time t is, as in Appendix A, the set of obligations whose status is not discharged, which includes the open, supported, refuted, and defeated ones, and I count an obligation as discharged only while its discharge is currently warranted. With that convention $|\text{Res}(t)| = a(t) - b(t)$, and since b can decrease when a defeater is recorded, b is not monotone.

Proposition 20.1 (Residue growth). *Suppose the rate of introduction of semantic obligations is at least $\alpha > 0$ per unit time over an interval, and the net rate of warranted discharge, which is the rate of new warranted discharges less the rate at which warrants are lost, is at most $\beta < \alpha$. Then $|\text{Res}(t)|$ grows by at least $(\alpha - \beta)$ per unit time over that interval, and in particular is unbounded if the condition persists.*

The proposition is an identity of accounting and is stated to make a point explicit that discussion of abundance often leaves implicit. Kernel verification scales with computation, so the rate of introduction of artifacts can be made as large as the computation allows. Semantic discharge requires interpretation, and the capacity for it does not scale in the same way. A regime in which production is cheap and interpretation is expensive is a regime in which the residue grows, regardless of how reliable the kernel is. The size of the verified corpus is then not a measure of what has been settled. It is, if the semantic obligations are not discharged, a measure of what has been produced.

The argument is not an argument against automated production, and it is silent on the rates. If the rate of automated discharge, by sound certifiers of the kind described in Chapter 11, keeps pace with the rate of introduction, the residue does not grow. What the proposition asserts is that the question of whether it does is an empirical question about rates and not something verified by the kernel's acceptance of each artifact.

A related observation concerns the means of discharge. If the semantic verification of a corpus is performed mainly by other automated systems, as the source reports for the Meta project, then the discharge certificates carry the authority of those systems and their dependency sets include the systems' own interpretive steps. By the definition of independence in Chapter 18, certificates produced by systems that share training, data, or architecture may be correlated, and a discharge supported only by such certificates has the warrant that a single source would have, to the extent that the correlation is high. The ledger does not decide whether this is acceptable. It records the dependency structure so that the question can be asked.

20.3 Volume is a poor measure of contribution

Automated generation can produce many valid but redundant propositions. The notion of redundancy can be made precise with the consequence relation of Appendix A, which is derivability from a certified background $\mathcal{B}$.

Definition 20.2 (Redundancy and increment). *A proposition ψ is redundant relative to $\mathcal{B}$ if $\mathcal{B} \models \psi$. The content of a set Ψ of propositions relative to $\mathcal{B}$ is the class of propositions derivable from $\mathcal{B} \cup \Psi$ modulo mutual derivability, and the increment of Ψ is the extent to which this class exceeds the class derivable from $\mathcal{B}$ alone.*

Proposition 20.3 (Redundant output has zero increment). *If every member of Ψ is redundant relative to $\mathcal{B}$, then Ψ has zero increment: the propositions derivable from $\mathcal{B} \cup \Psi$ are exactly those derivable from $\mathcal{B}$.*

Proof. If $\mathcal{B} \models \psi$ for every $\psi \in \Psi$, then by cut (C3) any proposition derivable from $\mathcal{B} \cup \Psi$ is derivable from $\mathcal{B}$. The reverse inclusion is monotonicity. $\square$

The proposition shows that the number of verified declarations bears no fixed relation to the increment in what is known. A collection of declarations may be large and have zero increment, and a collection of one declaration may have a large one. The measure of contribution is the increment, and it is relative to a background, so that the same declaration can be redundant in one setting and novel in another. The remedy is not to count less but to report the quantities that matter, each independently.

20.4 Five things that should not be confused

The chapter separates five activities that a single count of verified results tends to merge. *Discovery* is the production of a new mathematical fact or idea. *Verified theorem production* is the production of a formal artifact whose statement the kernel accepts. *Expository understanding* is the production of an account from which a reader comes to understand why a result holds. *Research usefulness* is the extent to which a result enables further work. *Institutional recognition* is the acceptance of a result by the bodies that confer standing on it. They are logically independent. A discovery may be unverified, a verified artifact may be a redundant restatement, a verified theorem may carry no exposition, a useful result may be unrecognized, and a recognized one may be of no further use.

In the vocabulary of Chapter 15, each corresponds to a different set of obligations and a different set of certifiers. The multidimensional status of Chapter 21 has its own components (kernel, statement, argument, dependency, and review), and the activities listed here are not mapped one to one onto them; what is retained is that assessments along separate dimensions are kept separate. Proposition A.22 applies: because the components are incomparable in general, no scalar ranking of artifacts by their combined merit is faithful to the order, and any such ranking embeds a weighting that should be stated.

20.5 A publication architecture

The proposal of the chapter is a publication architecture in which formal validity, semantic fidelity, novelty, provenance, and interpretive value are reported independently. Formal validity is the kernel status, recorded with the environment in which acceptance occurred. Semantic fidelity is the status of the statement and argument obligations, with the certificates that support it and their independence. Novelty is the increment relative to a stated background. Provenance is the ledger of Chapter 18. Interpretive value is the record of expository and

usefulness judgments by identified authorities, which are certificates like any other and can be defeated.

A publication in this architecture does not carry a verdict of accepted or rejected. It carries a profile, and consumers of the profile may weigh the components according to their purposes. The architecture does not remove the need for human interpretation. It makes the allocation of interpretive effort visible, so that a corpus that is large and mostly open on its semantic components is reported as such. The aim is to prevent a regime of abundance from being read as a regime of settled results.

Part VII

Architectures of Responsible Verification

Chapter 21

A Residue-Aware Verification System

The preceding chapters define the objects and prove the properties of a theory of verification residue. This chapter specifies a system that implements the theory, in the sense of a design whose components correspond to the definitions and whose guarantees correspond to the theorems. The specification is the constructive culmination of the monograph, and it is written with the discipline that has governed the rest: the system verifies what can be verified mechanically, records what cannot, and does not claim to solve unrestricted semantic equivalence.

21.1 Proof checking as one component

In the system, a proof checker is one component of a larger process. A claim retains its natural-language formulation and its argument as received. It retains the explicit interpretation $\mathcal{I}$ under which correspondence is to be assessed, the formal representations offered for it, the proof artifacts accepted for them, the dependency information linking declarations, and the unresolved correspondence obligations. The system supports independent formalization attempts of the same claim by different parties, the comparison of the resulting theorem statements, and the detection of strengthened hypotheses and weakened conclusions among them. It records discrepancies explicitly, with the type of Definition A.13 where a certificate for the type is supplied, and with the status of an open obligation otherwise.

The comparison of statements is the point at which the system most visibly relies on the framework. Given two formalizations F_1, F_2 of one claim, the system does not decide equivalence. It accepts a certificate of entailment in either direction, checks it with the kernel against the certified background, and reports one of four outcomes: equivalent, F_1 stronger, F_2 stronger, or unrelated by available certificates. The last outcome is an abstention in the sense of Chapter 11 and not a claim of incomparability. A strengthened hypothesis and a weakened conclusion both produce a weaker statement, so both appear as a certified entailment from the intended statement to the formalized one without the converse. The two are distinguished by which component differs, the hypothesis sets or the conclusions, and the system reports that component alongside the direction. A direction of entailment alone does not identify the kind of change.

21.2 The multidimensional status

Each claim has a status with its own components, each carrying its own evidential and adjudicative semantics:

$$\mathcal{S} = (s_{\text{kernel}}, s_{\text{statement}}, s_{\text{argument}}, s_{\text{dependency}}, s_{\text{review}}).$$

The kernel component records acceptance in a stated environment. The statement and argument components record the status of the corresponding obligations, with the certificates that support them. The dependency component records the status of the obligations on which the claim relies through the dependency graph, and by Proposition 17.6 it is the place where an unresolved definition enters the residue of the statements that use it. The review component records human adjudication as certificates with identified authorities. A kernel-verified theorem does not imply that the statement or argument components are satisfied, and the system cannot be configured to report it as though it did.

The question of whether the five components can be summarized by one ranking has a definite answer under stated assumptions. Take each component to range over a finite chain of evidence levels with at least two elements, and order statuses componentwise. By Proposition A.22, if two statuses are incomparable, which occurs whenever one has stronger kernel evidence and the other stronger statement evidence, then no map into the reals both preserves and reflects the order. The following companion statement gives the minimum number of coordinates.

Proposition 21.1 (RV-014, dimension). *The product of d chains, each with at least two elements, ordered componentwise, has order dimension d . Consequently a faithful order embedding of the status order $\mathcal{S}$ with five component chains into a product of real lines requires at least five real coordinates, and five suffice.*

The statement is a standard fact of order theory, the order dimension of a product of d non-trivial chains being d . The upper bound d is immediate: embed each chain order-isomorphically into $\mathbb{R}$. For the lower bound, take $0 < 1$ in each chain and let a_i be the element with 1 in coordinate i and 0 elsewhere, and b_j the element with 0 in coordinate j and 1 elsewhere. Then $a_i \leq b_j$ if and only if $i \neq j$. Suppose an order embedding into $(\mathbb{R}^m, \leq)$ exists. Since $a_i \not\leq b_i$, some coordinate $c(i)$ has $a_i(c(i)) > b_i(c(i))$. If $c(i) = c(j) = c$ for $i \neq j$, then $a_i \leq b_j$ and $a_j \leq b_i$ give $a_i(c) \leq b_j(c) < a_j(c) \leq b_i(c) < a_i(c)$, a contradiction. So c is injective on $\{1, \dots, d\}$ and $m \geq d$. This is the standard example of Dushnik and Miller [8]. The proposition is the precise sense in which a status structure with five independent components cannot be compressed into fewer numbers without loss of the ordering, and it is the reason the system reports the components separately. A scalar may still be presented as a convenience, and when it is, the system labels it as a lossy summary and states the weighting.

A concrete instance is available in a recent report on recovery of ultra-low-bit language models. Wang et al. [23] (arXiv:2610.09346) compare their method OnlineQAT with ReasoningQAT on Qwen3-1.7B at two-bit weights over six benchmarks, and report the unweighted mean over the six as the headline score. Their per-task scores at W2A16 are as follows.

Under the componentwise order on the six scores, neither method dominates the other: OnlineQAT is higher on three benchmarks and lower on three. The two are incomparable in the sense required by Proposition A.22, and the mean ranks OnlineQAT above ReasoningQAT by 0.44, so the map to the mean does not reflect the order, since the inequality of the means would

	GSM8K	MMLU-R	MATH	IFEval	LCB	GPQA-D	AVG
ReasoningQAT	56.60	43.80	37.80	33.80	5.30	15.20	32.08
OnlineQAT	59.82	46.33	36.00	32.49	4.83	15.66	32.52

Table 21.1: Per-task scores at W2A16, as reported by Wang et al.

require ReasoningQAT to be componentwise below OnlineQAT. The same holds at W3A16 with a single task, IFEval, on which OnlineQAT is lower by 0.8. The authors’ own text is exact about this: the gain at two bits is described as modest and task-dependent, and a scope statement notes that the comparison covers one model without multiple training seeds or an offline reverse-KL control, that it changes both the prefix distribution and the divergence direction so does not isolate either, and that it supports an empirical comparison and not a causal or compute-efficiency claim. The statement is an instance of recording the residue of a result next to the result. The point of the example is not that the mean is a poor summary, which the authors do not claim, but that the status order of any multi-task comparison contains incomparable pairs, and that a scalar summary resolves them by an implicit weighting that the report should state.

Cheng et al. [6] (arXiv:2610.02462) state the same structure in their introduction as a design requirement: a compression configuration that preserves overall performance may still degrade a required capability. Their measurements on twelve public models show that the loss change under pruning and under quantization differs in sign and in magnitude across mathematics, code, and question answering, with the ordering of capabilities changing from one model family to another. A single score cannot represent this, and the report accordingly presents the three capabilities separately and makes the selection of a method depend on the capability and the budget.

21.3 The ledger and the guarantees it supports

The core of the system is an obligation ledger kept apart from the proof assistant’s environment. It stores obligations, certificates, transition certificates, versions, and events as in Chapters 15, 17, and 18. Its guarantees are of two layers, as the theory requires.

The syntactic layer is checked mechanically. The ledger verifies that every transition records a disposition for every inherited obligation, which is well-formedness, and it computes the successor sets and ancestry discharges, from which Theorem A.15 yields that no obligation leaves the ledger without a recorded disposition. It verifies that the dependency graph is acyclic, and that every permitted assumption is either authorized or tracked, which are the admissibility conditions of Definition A.11. It verifies that the append-only discipline is respected and that hash chains are intact. All of these are decidable by inspection of finite structures, and none of them depends on the semantics of the contents.

The semantic layer is recorded as conditional. The ledger stores the entailment certificates of transition certificates and checks them with the kernel when they are formal. It does not decide whether the certificates are sound for the intended interpretation, and a preservation result such as Theorem A.17 is reported with its hypotheses listed, notably the soundness of the certificates and the tracked assumptions that remain in the residue. The ledger can therefore verify that an unresolved question has been preserved without answering the question. This is the system’s central capability, and its limits are the limits of the theory: it prevents silent

loss, and it does not supply the missing semantics.

21.4 A Lean 4 prototype

An experimental implementation in Lean 4 is specified here. The specification is a design. A first set of declarations has been written and is documented in Appendix D, but it has not been compiled, no proof has been checked, and two theorem statements carry the placeholder `sorry`; the prototype is therefore not a formal verification of any result of this monograph. The data structures follow Appendix A. The following sketch indicates the intended shape, and it should be read as unchecked pseudocode in Lean syntax; the declarations of Appendix D differ from it in detail.

```
inductive Status | open | supported | discharged | refuted | defeated
```

```
structure Obligation (L : Type) where
  id      : Nat
  kind    : ObligationKind          -- formal | statement | argument |
                                     -- dependency | assumption | ...
  content : L
  status  : Status
  deps    : List Nat
```

```
structure Transition (L : Type) where
  succ    : List (Nat x Nat)        -- Q_i
  disch   : List Nat                -- Delta_i
  assumed : List L                  -- Theta_i
```

```
def WellFormed (inp : List (Obligation L)) (t : Transition L) : Prop :=
  forall o in inp, o.id in t.disch or exists p in t.succ, p.1 = o.id
```

```
theorem trace (chain : List (Transition L)) ... :
  -- every originating obligation has a discharge in its ancestry
  -- or a nonempty successor set at the final artifact
```

The theorem `trace` is Theorem A.15 and is the natural first target, since its proof is a finite induction independent of the semantics of L . Acyclicity of the dependency graph and the admissibility conditions would follow as further checked predicates. The semantic layer would be represented by a type of entailment witnesses indexed by a parameter standing for the consequence relation, with the soundness assumption carried as an explicit hypothesis of the preservation theorem.

The experiment is designed with a narrow criterion. The prototype is not expected to solve semantic equivalence and no result of the monograph suggests it could. The criterion is whether recording obligations and transition certificates improves the reliability of certification claims, measured as the detection or preservation of known unresolved obligations. A test suite follows from the cases of the monograph: the polynomial example with the incorrect factorization, in which the argument obligation is refuted; the symmetric-matrix example, in

which statement correspondence holds and argument correspondence fails; the hidden-existence case with the silent default, in which the hypothesis-carrying representation is expected to leave an assumption obligation in the residue; and the derivative and pressure-flux cases, in which the expected outputs are a weakening and a lateral transition, each marked source-supported. A pass is a ledger whose residue contains the seeded obligation with the expected status. A failure is a report that shows the seeded obligation settled. The suite tests whether the system refuses to report as settled what is known to be unsettled, and it can say nothing about unknown discrepancies.

21.5 What the system does not claim

The system does not decide correspondence, and by the results of Chapter 10 no system of this form can on unrestricted classes. It does not certify the soundness of the certificates it stores. It does not replace human interpretation, which it records as certificates with identified authorities. It does not guarantee that the kernel's environment matches the one the ledger records, a dependence the system exposes by storing the environment. It claims that the layers of a verification are separated and tracked, that the syntactic layer is checked mechanically, and that the semantic layer is reported honestly as conditional. These are modest claims and they are the ones the theory supports.

Chapter 22

The Limits of Certification and the Future of Mathematical Understanding

The monograph began with a proof that compiled and an argument that it did not follow. It ends by asking what it means to know that a mathematical problem has been solved when the proof is formally valid, the problem is expressed in natural language, and the relation between them may not be completely decidable. The question has two familiar answers, and the monograph rejects both.

22.1 Two answers that fail

The first answer holds that formal verification settles every relevant epistemic question. On this view a proof that the kernel accepts is a proof, and anything further is superstition about trusted bases. The answer fails because the kernel's jurisdiction is a relation between a term and a type in a stated environment, and the questions of Chapter 16 lie outside it. Kernel acceptance is blind to the specification, to the argument, and to the interpretation, and no refinement of the kernel can change that, since the blindness is a property of what the judgment is a judgment of. The hidden-existence family shows that the questions it leaves open are not merely unattended but, on unrestricted classes, hard at every level of the arithmetical hierarchy for well-definedness, and hard at the second level for correspondence under the stated convention.

The second answer holds that because semantic uncertainty cannot be eliminated, machine-generated and machine-verified mathematics is worthless, or at least cannot be trusted. This answer fails for a reason developed in Chapter 11. The existence of undecidable instances limits a single total procedure and leaves a large space of sound, incomplete, and restricted procedures. A proof that compiles is genuine evidence about the formal statement. The two Navier-Stokes cases do not show that the Lean development is worthless, and they do not show that the natural-language argument is wrong. They show that the relation between the two is unsettled and can be described precisely. To conclude from uncertainty that nothing has been learned is as unwarranted as to conclude from acceptance that everything has.

22.2 Bounded certification

The alternative is a theory of bounded certification. Its requirements are three, and each has a counterpart in the formal development. A system should state exactly what it establishes, which is the discharged obligations with their currently warranted certificates, the authorities, the interpretation, and the dependencies. It should preserve what it cannot establish, which is the entailment condition of Chapter 17, under which unresolved obligations are carried through transformations without being relabeled as settled. And it should expose the conditions under which further evidence could change a result's status, which is the dependency graph with its defeaters, so that for each discharged obligation the system can list what would revoke the discharge, and for each open one what would settle it.

The three requirements share a feature. None of them eliminates uncertainty. Each makes the uncertainty a structured object that can be inspected, transported, and reduced. The residue of a verification is a precise list of what remains to be done, and a corpus with a small, benign, well-described residue is more trustworthy than a corpus with no recorded residue, because the second kind cannot distinguish a settled question from an unasked one.

22.3 Preservation is not discharge

The distinction that the monograph regards as its contribution is that between preserving an obligation and discharging it. A pipeline can rigorously carry an unresolved semantic obligation through a sequence of translations and revisions, certifying at each step that nothing has been lost, and arrive at the end with the obligation exactly as open as it began. The preservation is a mathematical fact with a proof, Theorem A.17, conditional on the soundness of its certificates. The openness is a fact about the evidence. Preservation does not imply discharge, and discharge of one obligation does not secure preservation of another, and Proposition A.18 shows both can occur. An obligation recorded in the discharge set of a transition is preserved at that transition, so the independence is asymmetric at a single recorded discharge. A culture of verification that treats preservation as discharge claims too much, and one that treats the absence of discharge as the absence of preservation discards what has been secured.

The distinction supports a modest and a more ambitious reading. The modest reading is a discipline of reporting: say what was checked, say what was carried, and say what remains. The ambitious reading is a research program in which the certificates that justify entailment and discharge are themselves objects of study, with the questions listed at the end of Chapter 17 concerning the certification of entailments between natural-language and formal contents, the justification of refinements, and the changing of interpretations along a chain. These remain open.

22.4 Understanding

The title of the chapter refers to the future of mathematical understanding, and the monograph has said little about understanding directly. It has treated the judgment of correspondence as an act performed by identified authorities, recorded as certificates, and defeasible. It has not treated understanding as a verifier that can be inserted at one point of a pipeline and discharge the semantic residue there. Chapter 19 argued that understanding at the point where objectives are set is necessary and not sufficient, because the interpretation of novel problems

is constructed after the objective is reviewed. The more general lesson is that understanding is not located at one stage. It is exercised wherever a correspondence is asserted, and a system that makes those assertions visible and attributable distributes the burden of understanding where it belongs and makes it possible to see where it has not been exercised.

It would be a mistake to read the monograph as a pessimistic account of formal methods. Its conclusions about what verification does not establish are conclusions about the limits of a jurisdiction, and a jurisdiction whose limits are known is more useful than one whose limits are assumed to coincide with those of the mathematical problem. The work closes by distinguishing the elimination of uncertainty, which the results of Part III show to be unavailable on unrestricted classes, from its responsible representation, which is available and which this monograph has tried to specify. Verification becomes more trustworthy when the system no longer mistakes the limits of its own jurisdiction for the limits of the mathematical problem.

Appendix A

Formal Definitions and Logical Conventions

This appendix fixes the definitions on which the preservation results of the monograph depend. It is written to be self-contained, so that the two central results, the traceability theorem (RV-020) and the compositional preservation theorem (RV-008), can be derived from it without appeal to the prose of the main text. Each definition is stated before it is used, and each result is accompanied by an explicit statement of which hypotheses it consumes. A closing section records the points at which writing the specification exposed a circularity or an unstated assumption in the outline, because those points determine where the main text must be careful.

A.1 Verification instances and the three judgments

Let **Spec** be a class of natural-language specifications, **Arg** a class of natural-language arguments, and **Form** a class of formal propositions. A *verification instance* is a tuple

$$V = (N, P, \mathcal{I}, \Gamma, F, \pi),$$

in which $N \in \mathbf{Spec}$ is the originating specification, $P \in \mathbf{Arg}$ its purported argument, $\mathcal{I}$ the intended interpretation, Γ the formal environment of definitions, axioms, imports, and dependencies, $F \in \mathbf{Form}$ the formalized proposition, and π the proof term. The interpretation $\mathcal{I}$ is part of the data of the instance and is held fixed throughout a chain of transformations. A change of interpretation is not treated as an incidental step and would itself require a transition certificate, a point taken up in Section A.11.

Three judgments are distinguished. Kernel acceptance is $K(V) : \Longleftrightarrow \Gamma \vdash \pi : F$. Statement correspondence is $C(V) : \Longleftrightarrow \llbracket N \rrbracket_{\mathcal{I}} \equiv_{\mathcal{I}} \llbracket F \rrbracket_{\mathcal{I}}$, where $\llbracket \cdot \rrbracket_{\mathcal{I}}$ assigns a denotation in a common semantic domain and $\equiv_{\mathcal{I}}$ is an explicitly chosen equivalence relation, which may be logical equivalence, equality of solution sets, preservation of quantified dependencies, or a stronger intensional relation. Argument correspondence is $A(V)$, a relation between the justified inferential steps of P and those of π , which cannot be reduced to agreement of conclusions. The definition presupposes that N is resolved in the context in the sense of Chapter 5, so that $\llbracket N \rrbracket_{\mathcal{I}}$ is a single denotation; for ambiguous N the statement obligation is relative to a chosen reading. Where a denotation may be undefined, as for a presupposing description, $\equiv_{\mathcal{I}}$ is an equivalence relation on the defined denotations only. No result in this appendix depends on the particular choice of $\equiv_{\mathcal{I}}$; the choice enters only through the contents of the obligations introduced below.

A.2 Contents and the consequence relation

Obligations have contents, and contents must stand in a relation of consequence for preservation to be a precise notion.

Definition A.1 (Content language and consequence). *Let $\mathcal{L}$ be a set, the content language, whose elements are the propositions that obligations assert. A consequence relation relative to $\mathcal{I}$ is a relation $\models_{\mathcal{I}} \subseteq \mathcal{P}(\mathcal{L}) \times \mathcal{L}$ satisfying, for all $X, Y \subseteq \mathcal{L}$ and $\psi, \chi \in \mathcal{L}$:*

(C1) Reflexivity: *if $\psi \in X$ then $X \models_{\mathcal{I}} \psi$.*

(C2) Monotonicity: *if $X \models_{\mathcal{I}} \psi$ and $X \subseteq Y$ then $Y \models_{\mathcal{I}} \psi$.*

(C3) Cut: *if $X \models_{\mathcal{I}} \chi$ for every $\chi \in Y$, and $Y \models_{\mathcal{I}} \psi$, then $X \models_{\mathcal{I}} \psi$.*

For a set Y and a proposition ψ the notation $X \models_{\mathcal{I}} Y$ abbreviates that $X \models_{\mathcal{I}} \chi$ for every $\chi \in Y$.

These are the conditions of a Tarskian consequence relation [22]. The relation is not classical semantic consequence over all truths. Under classical consequence any two true contents would be equivalent and every true content would follow from the empty set, so every transition between true contents would be preserving and the framework would collapse into a question about truth. The relation $\models_{\mathcal{I}}$ is instead derivability from the premises together with a *certified background* $\mathcal{B}_{\mathcal{I}}$, a fixed base of admitted facts and rules, by admitted inference steps. Reflexivity, monotonicity, and cut hold for such a relation. The typing of transitions is accordingly relative to $\mathcal{B}_{\mathcal{I}}$, and a transition that is a weakening relative to one background may be conservative relative to a background that includes a restoring lemma. A failure of entailment means that no admitted derivation is available, and does not assert that the content is false. Nothing further is assumed. In particular the relation is not assumed decidable, and in general it is not, so every use of $\models_{\mathcal{I}}$ in a ledger is mediated by a certificate. Nor is it assumed compact, although every instance used below involves finite sets. The notation $X \models_{\mathcal{I}} \psi$ is used in place of a conjunction of the members of X , which avoids committing the content language to a conjunction connective.

A.3 Obligations, statuses, and the residue

Definition A.2 (Obligation). *An obligation is a record $o = (\tau, \varphi, \rho, E, D, \sigma)$ in which τ is a type drawn from a set of obligation types that includes the distinguished type **assumption**, $\varphi(o) \in \mathcal{L}$ is the content, ρ the provenance, E the available evidence, D the set of obligations on which o depends, and $\sigma(o) \in \Sigma$ the current status, where*

$$\Sigma = \{\text{open}, \text{supported}, \text{discharged}, \text{refuted}, \text{defeated}\}.$$

*Each obligation carries exactly one status. The status **supported** records evidence short of an accepted certificate and must not be read as **discharged**. The status **defeated** records that an accepted certificate has been overridden and does not assert that the content is false, which is the role of **refuted**.*

Definition A.3 (Residue). *For a finite set $\mathcal{O}$ of obligations, the verification residue is*

$$\mathcal{R}(\mathcal{O}) = \{o \in \mathcal{O} : \sigma(o) \neq \text{discharged}\}.$$

Proposition A.4 (Completeness of the residue partition). *For every finite $\mathcal{O}$,*

$$\mathcal{R}(\mathcal{O}) = \mathcal{R}_{\text{open}} \sqcup \mathcal{R}_{\text{supported}} \sqcup \mathcal{R}_{\text{refuted}} \sqcup \mathcal{R}_{\text{defeated}}, \quad \mathcal{R}_s = \{o \in \mathcal{O} : \sigma(o) = s\},$$

and $\mathcal{O} = \mathcal{R}(\mathcal{O}) \sqcup \{o \in \mathcal{O} : \sigma(o) = \text{discharged}\}$. The four components are pairwise disjoint and their union is $\mathcal{R}(\mathcal{O})$.

Proof. Because σ is a function into Σ , the sets $\{o : \sigma(o) = s\}$ for $s \in \Sigma$ partition $\mathcal{O}$. Removing the block indexed by **discharged** leaves the remaining four blocks, whose union is $\mathcal{R}(\mathcal{O})$ by definition. $\square$

The proposition is definitional, and it is stated for exactly that reason: its content is the requirement that the status model assign one status per obligation. If a later chapter allowed an obligation to carry two statuses, for instance **supported** and **defeated** simultaneously, the partition would fail and the residue accounting of the main text would have to be revised.

Three baseline obligations accompany every instance V : O_{formal} with content $\Gamma \vdash \pi : F$, $O_{\text{statement}}$ with content $\llbracket N \rrbracket_{\mathcal{I}} \equiv_{\mathcal{I}} \llbracket F \rrbracket_{\mathcal{I}}$, and O_{argument} with content that P and π correspond. Only the first is settled by kernel acceptance.

A.4 Certificates, historical discharge, and warranted discharge

Discharge is a property of one obligation, and it has two readings that the theory keeps apart.

Definition A.5 (Discharge certificate). *A discharge certificate for an obligation o is a tuple $c = (\alpha, \mathcal{I}, \text{dep}(c), \text{pf})$ recording the accepting authority α , the interpretation $\mathcal{I}$ under which acceptance occurred, a finite set $\text{dep}(c) \subseteq \mathcal{O} \cup \mathcal{A}$ of obligations and authorized assumptions on which the acceptance depends, and a payload pf constituting the evidence.*

Here $\mathcal{A} \subseteq \mathcal{L}$ is the *authorized context*, the set of assumptions fixed with the verification instance and accepted without a tracking obligation, for instance the axioms of the stated foundation. A ledger state at time t consists of the obligations, their accepted certificates, and a set Def_t of recorded defeaters, each of which targets a certificate or an authorized assumption.

Definition A.6 (Dependency graph). *At a ledger state t the dependency graph G_t (the graph of the state, with the edges contributed by every transition accepted up to t) has as vertices the obligations, the accepted certificates, and the authorized assumptions. Its edges are of three kinds, all read as “depends on”: obligation-dependency edges from each obligation o to every member of its dependency field $D(o)$; certificate-dependency edges from each accepted certificate c to every member of $\text{dep}(c)$, together with an edge from each obligation to the certificate that discharges it; and assumption-use edges from each obligation o whose preservation in some transition T_i uses a permitted assumption $\theta \in \Theta_i$, in the sense that T_i is preserving at o and $\theta \in \Theta_i$ (the edge is created for every member of Θ_i at every obligation at which T_i is preserving, so that no redundant assumption escapes the acyclicity test), to the vertex that supplies θ , which is θ itself when $\theta \in \mathcal{A}$ and the tracked obligation a_θ otherwise.*

A certificate or a defeater is accepted into the ledger only if the resulting graph is acyclic, so that G_t is acyclic at every ledger time t and not only at transition times. The three kinds of edge are listed separately because a cycle may close through a mixture of them: an obligation

may depend on a tracked assumption whose discharge certificate depends, through a second obligation, on the first. Acyclicity is therefore required of G_t as a whole and not of each family of edges separately.

Definition A.7 (Historical and warranted discharge). *A discharge of o by c is historical at time t if c was accepted at some time $s \leq t$. Assume that G_t is acyclic, so that, G_t being finite, the relation of dependency is well founded. The discharge is currently warranted at time t if c is historical, no defeater in Def_t targets c , and every member of $\text{dep}(c)$ is either an authorized assumption against which no defeater in Def_t is recorded, or an obligation whose own discharge is currently warranted at time t .*

The definition is by recursion on the well-founded dependency order and is therefore unambiguous. Historical discharge is a record and is never rewritten. Warranted discharge is a function of the current ledger state.

Proposition A.8 (Defeasibility of discharge, RV-021). *Let $t \leq t'$ be ledger times with $\text{Def}_t \subseteq \text{Def}_{t'}$ and no certificate removed, and assume the dependency graphs G_t and $G_{t'}$ are acyclic. Then (i) every historical discharge at t is historical at t' ; (ii) if in addition no certificate is accepted in the interval $(t, t']$, then every discharge currently warranted at t' is currently warranted at t ; and (iii) there exist ledgers in which a discharge is currently warranted at t and not at t' .*

Proof. (i) holds because acceptance times only accumulate. For (ii), proceed by well-founded induction on the dependency order. Suppose the discharge of o by c is warranted at t' . Since no certificate is accepted in $(t, t']$, c was accepted at a time $\leq t$ and is historical at t , and the same holds for the certificate of every obligation in $\text{dep}(c)$. Then no defeater in $\text{Def}_{t'}$ targets c , hence none in $\text{Def}_t \subseteq \text{Def}_{t'}$ does. Each member of $\text{dep}(c)$ is an authorized assumption undefeated at t' , hence at t , or an obligation whose discharge is warranted at t' , hence at t by the induction hypothesis. Thus c is warranted at t . For (iii), let o_1 be discharged by c_1 with $\text{dep}(c_1) = \{a\}$ for an authorized assumption a , warranted at t with $\text{Def}_t = \emptyset$, and let a defeater targeting a be recorded at t' . Then c_1 is historical at both times and warranted only at t . $\square$

Without the extra hypothesis of (ii) the implication fails: a certificate accepted at s with $t < s \leq t'$ and undefeated is warranted at t' and is not even historical at t . What the proposition states is therefore antitonicity in the defeaters for a fixed set of certificates, together with monotone accumulation of the historical record. It formalizes the point that a defeated assumption propagates upward through every certificate that depends on it, while leaving the historical record intact. The status of o_1 at t' reverts from **discharged** to **open** or **defeated**, according to the policy of the ledger, and the history records both facts.

Definition A.9 (Obligation-level warrant, status, and establishment). *An obligation o is warranted-discharged at t if some accepted certificate for o is currently warranted at t . In the recursion of Definition A.7, “an obligation whose own discharge is currently warranted” means an obligation that is warranted-discharged in this sense, so that an obligation with several certificates loses warrant only when all of its certificates have lost it. The stored status is tied to warrant by the convention $\sigma_t(o) = \text{discharged}$ if and only if o is warranted-discharged at t ; when it is not, $\sigma_t(o)$ is one of the other four values according to the policy of the ledger, and the historical record of the certificate is kept. Residue and its partition are evaluated with*

σ_t , and I write $\mathcal{R}_t$ and $\mathcal{R}_{s,t}$ where the time matters. A proposition $\psi \in \mathcal{L}$ is established at t if $\psi \in \mathcal{A}$ and no defeater in Def_t targets it, or $\psi = \varphi(o)$ for an obligation o that is warranted-discharged at t . The certificates are sound if every proposition established at t holds under $\mathcal{I}$, and $\models_{\mathcal{I}}$ is sound for $\mathcal{I}$ if $X \models_{\mathcal{I}} \psi$ and every member of X holds under $\mathcal{I}$ imply that ψ holds under $\mathcal{I}$. The predicate “holds under $\mathcal{I}$ ” is a predicate $\text{Hold}_{\mathcal{I}} \subseteq \mathcal{L}$ given as part of the data of the instance; for contents that concern $\mathcal{I}$ itself, such as the content of $O_{\text{statement}}$, it is supplied by the authority for the intended interpretation, and the second statement of Theorem A.17 is relative to it. The entailment certificates that witness preservation are assumed correct for $\models_{\mathcal{I}}$. Both soundness conditions are hypotheses about the authorities and the interpretation. They are not consequences of (C1) to (C3), and nothing in the ledger can verify them.

A.5 Transitions and transition certificates

Let $X_0, X_1, \dots, X_n$ be artifacts and $f_i : X_{i-1} \rightarrow X_i$ transformations, with finite obligation sets $\mathcal{O}_i$ attached to X_i .

Definition A.10 (Transition certificate). A transition certificate for f_i is a triple $T_i = (Q_i, \Delta_i, \Theta_i)$ where $Q_i \subseteq \mathcal{O}_{i-1} \times \mathcal{O}_i$ is the successor relation, $\Delta_i \subseteq \mathcal{O}_{i-1}$ is the discharge record, each member carrying a discharge certificate that is currently warranted in the ledger state t_i^- immediately before the acceptance of T_i , and $\Theta_i \subseteq \mathcal{L}$ is a finite set of permitted assumptions. Write $\text{succ}_i(o) = \{o' : (o, o') \in Q_i\}$. The ledger starts at a time t_0 at which X_0 and $\mathcal{O}_0$ are recorded, and the certificate T_i is accepted at a time $t_i > t_{i-1}$, with t_i^- the state immediately before it. The graph G_{t_0} is required to be acyclic. Warrant of the members of Δ_i is evaluated in the state t_i^- , which does not yet contain the assumption-use edges of T_i , so that the definition of warrant is not applied to the graph that T_i itself extends.

Definition A.11 (Admissibility). A transition certificate T_i is admissible when (A1) every $\theta \in \Theta_i$ satisfies either $\theta \in \mathcal{A}$, or $\theta = \varphi(a_\theta)$ for some obligation $a_\theta \in \mathcal{O}_i$ of type **assumption**, either introduced by the transition or carried from $\mathcal{O}_{i-1}$ through Q_i ; and (A2) the dependency graph G_i of Definition A.6, including the assumption-use edges created by the transition, remains acyclic.

Condition (A1) forbids the introduction of an untracked assumption. An assumption not in the authorized context enters the ledger as a new obligation and therefore enters the residue until it is discharged. Condition (A2) is the form of non-circularity that is required, and it applies to the dependency graph as a whole, which includes obligation dependencies, certificate dependencies, and assumption-use edges. It is not enough to forbid an assumption identical to the content being preserved, because a circle may close through several steps, for instance when the discharge of an assumption obligation depends on the obligation whose preservation it was introduced to secure. Acyclicity also guarantees that Definition A.7 is well posed. It does not guarantee soundness. An acyclic ledger may rest on a certificate that is wrong, on an assumption that is false, or on a consequence relation that fails to track the intended interpretation, and acyclicity excludes only the circular justification of one claim by itself. The semantic hypotheses of Theorem A.17, that the certificates are sound for $\models_{\mathcal{I}}$ under $\mathcal{I}$, are separate and are not implied by (A1) and (A2).

Definition A.12 (Context and preservation). For a transition certificate T_i and $o \in \mathcal{O}_{i-1}$ the

certified context of o is

$$\text{Ctx}_i(o) = \{\varphi(o') : o' \in \text{succ}_i(o)\} \cup \{\varphi(d) : d \in \Delta_i\} \cup \Theta_i.$$

The transition f_i is preserving at o when $\text{Ctx}_i(o) \models_{\mathcal{I}} \varphi(o)$, and preserving when it is preserving at every $o \in \mathcal{O}_{i-1}$.

The entailment is certified and not computed: the certificate must contain a witness of $\text{Ctx}_i(o) \models_{\mathcal{I}} \varphi(o)$ that a checker for $\models_{\mathcal{I}}$ accepts. Because $\models_{\mathcal{I}}$ is relative to $\mathcal{I}$ and undecidable in general, a preservation claim has the status of the checker that accepts the witness, and an unchecked claim is itself an open obligation.

Definition A.13 (Typing of transitions). *For $o \in \mathcal{O}_{i-1}$ and $\text{Ctx} = \text{Ctx}_i(o)$, the transition is of exactly one of four types at o :*

	$\varphi(o) \models_{\mathcal{I}} \text{Ctx}$	$\varphi(o) \not\models_{\mathcal{I}} \text{Ctx}$
$\text{Ctx} \models_{\mathcal{I}} \varphi(o)$	conservative	strengthening
$\text{Ctx} \not\models_{\mathcal{I}} \varphi(o)$	weakening	lateral

Conservative and strengthening transitions are preserving. Weakening and lateral transitions are non-preserving. A strengthening transition preserves $\varphi(o)$ but imports the burden of discharging stronger content, and a weakening transition loses content, in that everything certified follows from $\varphi(o)$ but $\varphi(o)$ does not follow from it.

The notation $\varphi(o) \models_{\mathcal{I}} \text{Ctx}$ means $\{\varphi(o)\} \models_{\mathcal{I}} \chi$ for every $\chi \in \text{Ctx}$. The four cases are exhaustive and mutually exclusive by construction.

A.6 Traceability: RV-020 and RV-017

The first of the two central results is purely syntactic and does not use $\models_{\mathcal{I}}$.

Definition A.14 (Well-formed ledger). *A ledger $(T_i)_{i=1}^n$ is well formed when every T_i records a disposition for every inherited obligation: for each $o \in \mathcal{O}_{i-1}$, either $o \in \Delta_i$ or $\text{succ}_i(o) \neq \emptyset$ (or both).*

Define the composite successor sets by $S_0(o) = \{o\}$ and $S_k(o) = \bigcup_{o' \in S_{k-1}(o)} \text{succ}_k(o')$. Say that $o \in \mathcal{O}_0$ has a *certified discharge in its ancestry up to n* if for some $k \leq n$ there is $o' \in S_{k-1}(o)$ with $o' \in \Delta_k$.

Theorem A.15 (Traceability of dispositions, RV-020). *Let $(T_i)_{i=1}^n$ be a well-formed ledger. For every $o \in \mathcal{O}_0$, either o has a certified discharge in its ancestry up to n , or $S_n(o) \neq \emptyset$. Moreover, the property of well-formedness, and the computation of $S_n(o)$ and of the ancestry discharge, are decidable for finite ledgers by inspection of Q_i and Δ_i alone.*

Proof. Suppose o has no certified discharge in its ancestry up to n . It is shown by induction on $k \leq n$ that $S_k(o) \neq \emptyset$. For $k = 0$, $S_0(o) = \{o\}$. If $S_{k-1}(o) \neq \emptyset$ and $k \leq n$, choose $o' \in S_{k-1}(o)$. By hypothesis $o' \notin \Delta_k$, so well-formedness gives $\text{succ}_k(o') \neq \emptyset$, and $\text{succ}_k(o') \subseteq S_k(o)$. Decidability is immediate from finiteness: each condition is a finite check of set membership and non-emptiness. $\square$

Corollary A.16 (No silent discharge, RV-017). *In a well-formed ledger, an originating obligation that has no certified discharge in its ancestry has a successor at the final artifact X_n .*

The theorem is deliberately weak and the weakness is the source of its value. It guarantees that nothing disappears from the ledger without a recorded disposition. It says nothing about whether the recorded dispositions are semantically adequate, which is the content of the second central result. A ledger can be well formed and non-preserving, for instance when a successor is a weakening of its predecessor, and the independence of the two properties is established in Proposition A.18.

A.7 Compositional preservation: RV-008

Define the cumulative assumption and discharge pool $A_0 = \emptyset$ and $A_k = A_{k-1} \cup \{\varphi(d) : d \in \Delta_k\} \cup \Theta_k$, and the cumulative context of $o \in \mathcal{O}_0$ by

$$\Gamma_k(o) = \{\varphi(o') : o' \in S_k(o)\} \cup A_k.$$

Theorem A.17 (Compositional semantic preservation, RV-008). *Let $(T_i)_{i=1}^n$ be a ledger of transition certificates over a consequence relation $\models_{\mathcal{I}}$ satisfying (C1) to (C3), fix $o \in \mathcal{O}_0$, and suppose each T_k is preserving at every obligation of $S_{k-1}(o)$. Then*

$$\Gamma_n(o) \models_{\mathcal{I}} \varphi(o).$$

Now suppose in addition that the ledger is admissible, that the certificates are sound and $\models_{\mathcal{I}}$ is sound for $\mathcal{I}$ in the sense of Definition A.9, and that at the evaluation time t every member of $\Gamma_n(o)$ is established at t . Then $\varphi(o)$ holds under $\mathcal{I}$; I say that it is transported, to distinguish this from being established in the sense of Definition A.9, which would require that o itself be warranted-discharged. Every member of $\Gamma_n(o)$ has a source: itself in $\mathcal{A}$; or $\varphi(o')$ for $o' \in S_n(o)$; or $\varphi(d)$ for some $d \in \Delta_k$, $k \leq n$; or a member of $\Theta_k \setminus \mathcal{A}$, whose source is the tracked assumption obligation required by (A1). If a member $\varphi(d)$ or a tracked assumption is not established at t , then the corresponding source obligation is not warranted-discharged at t ; if a member of $\mathcal{A}$ is not established, it is defeated. Transport through $\Gamma_n(o)$ fails to that extent, although by (C2) transport through a subset that entails $\varphi(o)$ may still be available.

Proof. By induction on k it is shown that $\Gamma_k(o) \models_{\mathcal{I}} \varphi(o)$. For $k = 0$, $\Gamma_0(o) = \{\varphi(o)\}$, and (C1) gives the claim. Assume $\Gamma_{k-1}(o) \models_{\mathcal{I}} \varphi(o)$ and consider $\Gamma_k(o)$. Take $o' \in S_{k-1}(o)$. Since T_k is preserving at o' , $\text{Ctx}_k(o') \models_{\mathcal{I}} \varphi(o')$. Now $\text{Ctx}_k(o') \subseteq \Gamma_k(o)$, because $\text{succ}_k(o') \subseteq S_k(o)$ and the discharge contents and permitted assumptions of T_k lie in A_k . Monotonicity (C2) gives $\Gamma_k(o) \models_{\mathcal{I}} \varphi(o')$. Next take $\psi \in A_{k-1}$. Then $\psi \in A_k \subseteq \Gamma_k(o)$ and (C1) gives $\Gamma_k(o) \models_{\mathcal{I}} \psi$. Since $\Gamma_{k-1}(o) = \{\varphi(o') : o' \in S_{k-1}(o)\} \cup A_{k-1}$, it has been shown that $\Gamma_k(o) \models_{\mathcal{I}} \chi$ for every $\chi \in \Gamma_{k-1}(o)$. Combining with $\Gamma_{k-1}(o) \models_{\mathcal{I}} \varphi(o)$ and applying cut (C3) yields $\Gamma_k(o) \models_{\mathcal{I}} \varphi(o)$.

For the second statement, $\Gamma_n(o) \models_{\mathcal{I}} \varphi(o)$ by the first. Every premise is established at t by hypothesis, hence holds under $\mathcal{I}$ because the certificates are sound, and $\models_{\mathcal{I}}$ is sound for $\mathcal{I}$, so $\varphi(o)$ holds under $\mathcal{I}$. The description of the sources is by the definitions of $S_n(o)$, A_n and, for $\Theta_k \setminus \mathcal{A}$, condition (A1). A premise $\varphi(d)$ with $d \in \Delta_k$ is established when d is warranted-discharged at t , and an assumption $\theta \in \Theta_k \setminus \mathcal{A}$ is established when the obligation tracking it is warranted-discharged; the contrapositives give the last sentences. The converses can fail, since the same content may be established through another obligation or through $\mathcal{A}$. $\square$

The first statement uses reflexivity, monotonicity, and cut, and preservation only at the obligations reached from o ; this local form is what the proof needs, and preservation of the whole transition is stronger. It does not use admissibility or well-formedness, since the certified context and preservation are defined for any triple $(Q_i, \Delta_i, \Theta_i)$, and it does not use decidability of $\models_{\mathcal{I}}$. Admissibility enters the second statement: (A1) supplies the tracked source obligation of each assumption outside $\mathcal{A}$, and (A2) makes the notion of current warrant well posed. Where the obligation o itself lies in Δ_1 (the only possibility for an originating obligation unless the record persists across artifacts), its content is a premise of its own cumulative context and the first statement holds by (C1) from that point. The obligations whose status blocks transport belong to the various $\mathcal{O}_k$. Whether they appear among the obligations of the final artifact X_n is a separate matter: for the descendants of o it is answered by well-formedness and Theorem A.15, and for sources of premises in $\mathbf{A}_n$ it is answered only if the ledger is also required to carry them forward, which is not assumed. The entailment holds with or without that requirement, and I call the set of source obligations not warranted-discharged at t the *transport shortfall*, as opposed to the residue of X_n .

A.8 Independence of preservation and discharge: RV-018

Proposition A.18 (Independence, RV-018). *Suppose $\mathcal{L}$ contains propositions p, q, r with $\{p, q\} \not\models_{\mathcal{I}} r$, where r is taken as $\varphi(o)$ in part (ii) and q as $\varphi(d)$. Then:*

- (i) *there is, for every $n \geq 1$, an admissible ledger of length n in which every T_i is preserving at o_{i-1} and in which the status of every obligation in $S_n(o)$ is open;*
- (ii) *there is an admissible, well-formed ledger in which the discharge of an obligation d is warranted, yet the transition is not preserving at some other obligation $o \neq d$;*
- (iii) *if $d \in \Delta_i$ then $\text{Ctx}_i(d) \models_{\mathcal{I}} \varphi(d)$, so an obligation recorded in the discharge set of a transition is preserved at that transition.*

Proof. (i) Take $\mathcal{O}_0 = \{o\}$ with $\varphi(o) = p$ and status **open**. Let each T_i have $Q_i = \{(o_{i-1}, o_i)\}$ with $\varphi(o_i) = p$ and status **open**, $\Delta_i = \emptyset$, $\Theta_i = \emptyset$. Then $\text{Ctx}_i(o_{i-1}) = \{p\}$ and (C1) gives preservation at every step, while no obligation is ever discharged. (ii) Take $\mathcal{O}_0 = \{d, o\}$ with $\varphi(d) = q$ and $\varphi(o) = r$. Let T_1 have $\Delta_1 = \{d\}$ with a warranted certificate, $Q_1 = \{(o, o')\}$ with $\varphi(o') = p$, $\Theta_1 = \emptyset$, and $\text{succ}_1(d) = \emptyset$. The ledger is well formed. Here $\text{Ctx}_1(o) = \{p, q\}$ because the discharge record contributes $\varphi(d) = q$ to every certified context. Since $\{p, q\} \not\models_{\mathcal{I}} r$ the transition is non-preserving at o although d is warranted-discharged. The point is that the discharge of d supplies a premise and does not supply the missing content of o . (iii) For $d \in \Delta_i$ the content $\varphi(d)$ belongs to $\text{Ctx}_i(d)$ by the definition of the certified context, and (C1) gives $\text{Ctx}_i(d) \models_{\mathcal{I}} \varphi(d)$. The membership $d \in \Delta_i$ is a recorded fact about the transition, and the certificate of d is warranted at t_i^- , before T_i is accepted, so d is already discharged when the transition records it. $\square$

Part (iii) shows that the independence is asymmetric at a single obligation. Preservation without discharge occurs by (i), and discharge without preservation occurs at a different obligation, by (ii). An obligation recorded in Δ_i is preserved at T_i . An obligation whose status is discharged but which is not recorded in Δ_i may have only a weaker successor, and is then not preserved at its own step, so the asymmetry holds for the recorded discharge and not for the

status as such. A discharge can also make another obligation preserved, for instance when the contents coincide, so the claim established is that discharge of one obligation need not preserve another, and not that it never does.

Corollary A.19 (A verified continuation can retain unresolved obligations, RV-009). *Part (i) shows a chain preserving an open obligation through any number of transformations without discharging it. In a well-formed ledger, Theorem A.15 shows that an obligation with no certified discharge in its ancestry has a successor at the final artifact, and Theorem A.17 gives the entailment of its content from the cumulative context. Whether the tracked assumptions on which preservation relied are carried to the final artifact depends on whether the ledger requires it.*

A.9 Weakening and the total certified context: RV-019

Proposition A.20 (Preservation is a property of the total context, RV-019). *The transition f_i fails to preserve o if and only if $\text{Ctx}_i(o) \not\models_{\mathcal{I}} \varphi(o)$. In particular: (i) a successor $o' \in \text{succ}_i(o)$ whose content $\varphi(o')$ is strictly weaker than $\varphi(o)$ does not by itself cause failure; and (ii) a transition with a single successor strictly weaker than $\varphi(o)$, empty Δ_i and empty Θ_i , is non-preserving at o .*

Proof. The first sentence is the definition of preservation. For (i), assume $\mathcal{L}$ contains propositions p, q and a proposition m with $\{p, q\} \models_{\mathcal{I}} m$, $\{p\} \not\models_{\mathcal{I}} m$, $\{q\} \not\models_{\mathcal{I}} m$, $m \models_{\mathcal{I}} p$ and $m \models_{\mathcal{I}} q$ (a meet of p and q ; its existence is a hypothesis on $\mathcal{L}$, since $\mathcal{L}$ is not assumed to have a conjunction), let $\varphi(o) = m$, and let $\text{succ}_i(o) = \{o_p, o_q\}$ with contents p and q . Each successor is strictly weaker than $\varphi(o)$, yet $\text{Ctx}_i(o) = \{p, q\} \models_{\mathcal{I}} \varphi(o)$, so the transition is preserving. For (ii), $\text{Ctx}_i(o) = \{\varphi(o')\}$ and $\{\varphi(o')\} \not\models_{\mathcal{I}} \varphi(o)$ because $\varphi(o')$ is strictly weaker, that is, $\varphi(o) \models_{\mathcal{I}} \varphi(o')$ and $\{\varphi(o')\} \not\models_{\mathcal{I}} \varphi(o)$. $\square$

The derivative-loss example of Chapter 12 is, on the source's report and subject to the provisional status recorded there, an instance of (ii) in the following sense. The natural-language estimate asserts $\|N^{-1}F\|_{C_y^m} \leq C_m \|F\|_{C_y^{m+4}}$, and the Lean statement asserts the same with $\|F\|_{C_y^{m+5}}$ on the right. Every F satisfying the first hypothesis class satisfies the second, so the second is implied by the first and not conversely, and a proof of the second does not entail the first unless a restoring lemma or admissible assumption is added to the context. The example must be evaluated against all premises available in the formalized development and not against the single declaration, as (i) shows.

A.10 Representation of certification status: RV-014

Definition A.21 (Certification order). *A certification order is a partial order $(S, \preceq)$ on statuses, for instance the product order on triples of evidence levels for kernel acceptance, statement correspondence, and argument correspondence.*

Proposition A.22 (No faithful scalar, RV-014). *If $(S, \preceq)$ contains two incomparable elements, there is no function $s : S \rightarrow \mathbb{R}$ with*

$$x \preceq y \iff s(x) \leq s(y) \quad \text{for all } x, y \in S.$$

Proof. Let x, y be incomparable, so that $x \not\leq y$ and $y \not\leq x$. If such an s existed, then $s(x) > s(y)$ and $s(y) > s(x)$, which is impossible, since $\leq$ on $\mathbb{R}$ is total. $\square$

A scalar summary is therefore either non-reflecting, ranking statuses that are incomparable, or non-preserving. The substantive companion question is the least d such that $(S, \preceq)$ order-embeds into $(\mathbb{R}^d, \leq)$ with the componentwise order. For a finite poset this is its order dimension in the sense of Dushnik and Miller, and the statement of Chapter 21 will be restricted to finite certification orders, for which the quantity is well defined.

A.11 Audit of the specification

Writing the definitions above in full exposed five points at which the outline required refinement, and these constrain the main text.

First, the non-circularity condition proposed for permitted assumptions, that Θ_i alone must not entail $\varphi(o)$, is not the right condition. An assumption identical to $\varphi(o)$ that is recorded as a tracked open obligation is not dishonest, because the obligation reappears in the residue, and preservation then reports exactly what it should, namely that o is preserved on the strength of an unproved assumption. The genuine circularity is a cycle in the dependency graph, taken over obligation dependencies, certificate dependencies, and assumption-use edges together, which is why Definition A.11 requires (A2) instead. Acyclicity is a structural condition and does not by itself secure soundness.

Second, the premises $\varphi(d)$ for $d \in \Delta_i$ enter the certified context whether or not the discharge remains warranted. Entailment is therefore a logical statement and the transport of establishment from premises to conclusion is the separate second statement of Theorem A.17. The two must not be merged in the prose.

Third, the interpretation $\mathcal{I}$ is held fixed along the chain. A transformation that changes the interpretation, for instance by moving from real-valued to rational-valued semantics, is outside the present formalism and would require a transition between two consequence relations, one relative to $\mathcal{I}$ and one to $\mathcal{I}'$, with a certified translation of contents. This is recorded as a limitation and a candidate extension.

Fourth, warrant is defined relative to a dependency graph that the transition being certified itself extends, and the time indices t_i^- and t_i of the definition of a transition certificate are there to prevent a circularity between the warrant of the discharge record and the graph in which it is evaluated. Fifth, the entailment condition has the status of a certified claim. Theorem A.15 is mechanically checkable, Theorem A.17 is conditional on soundness of the certificates for $\models_{\mathcal{I}}$, and the monograph asserts nothing stronger about the second than that it makes preservation a precise and composable property distinct from discharge.

Appendix B

Computability Background and Derivations

This appendix collects the computability-theoretic facts that Chapters 9, 10, 11 and 16 use, and separates what is proved here from what is imported. Three kinds of statement appear and are marked. A result marked *standard* is a textbook fact of recursion theory and is cited and not proved. A result marked *source* is a theorem of Bastounis, Circelli, and Hansen [3] that is reported and not reproved. A result marked *derived* is proved in this appendix from the other two kinds. The aim is that no step in the main text that appeals to the arithmetical hierarchy rests on an unmarked claim.

B.1 Standard facts

Write W_e for the e -th recursively enumerable set and $\emptyset'$ for the Halting set. The following are standard.

- (S1) *Normal form.* A set $A \subseteq \mathbb{N}$ is Σ_l^0 if $A = \{e : \exists z_1 \forall z_2 \cdots Qz_l R(e, \vec{z})\}$ for a decidable R , and Π_l^0 if its complement is Σ_l^0 . A set is Σ_1^0 exactly when it is recursively enumerable.
- (S2) *Post's theorem* [20, 19]. A is Σ_{l+1}^0 if and only if it is recursively enumerable relative to $\emptyset^{(l)}$, and A is $\Delta_{l+1}^0 = \Sigma_{l+1}^0 \cap \Pi_{l+1}^0$ if and only if it is computable relative to $\emptyset^{(l)}$.
- (S3) *Hierarchy theorem* [20]. For every $l \geq 1$ there is a Σ_l^0 set that is not Π_l^0 . In particular $\Sigma_l^0 \neq \Pi_l^0$ and $\Delta_l^0 \subsetneq \Sigma_l^0$.
- (S4) *Completeness* [20]. The set $\text{NTOT} = \{e : \exists n \varphi_e(n) \uparrow\}$ is Σ_2^0 -complete under many-one reductions.
- (S5) *DPRM* [7, 15], *nine unknowns* (Jones [10]). Every recursively enumerable relation $S(\vec{a})$ has a polynomial q with integer coefficients such that $S(\vec{a}) \iff \exists x_1, \dots, x_9 \in \mathbb{N} q(\vec{a}, x_1, \dots, x_9) = 0$. The number of unknowns, nine, is what the main text calls the bound $k \geq 9$; padding with dummy unknowns gives all larger k .

Lemma B.1 (Derived: a complete set is outside the lower classes). *If A is Σ_l^0 -complete under many-one reductions, then $A \notin \Pi_l^0$, hence $A \notin \Delta_l^0$, and A is not computable relative to $\emptyset^{(l-1)}$. In particular a Σ_2^0 -complete set is neither recursively enumerable nor co-recursively enumerable.*

Proof. By (S3) choose a Σ_l^0 set C that is not Π_l^0 . Since A is complete, $C \leq_m A$ by a computable g with $e \in C \iff g(e) \in A$. If A were Π_l^0 , then $C = g^{-1}(A)$ would be Π_l^0 , because Π_l^0 is closed under computable preimages, a contradiction. Hence $A \notin \Pi_l^0 \supseteq \Delta_l^0$. By (S2) the sets computable relative to $\emptyset^{(l-1)}$ are Δ_l^0 , so A is not among them. For $l = 2$: a Σ_1^0 set and a Π_1^0 set are both in $\Delta_2^0 \subseteq \Sigma_2^0 \cap \Pi_2^0$, and a Σ_2^0 -complete set is not Π_2^0 , so it is not Σ_1^0 or Π_1^0 , the latter since $\Pi_1^0 \subseteq \Pi_2^0$. $\square$

B.2 The set d

Fix $k \geq 9$ and a computable enumeration $(p_e)_{e \in \mathbb{N}}$ of the integer-coefficient polynomials in $k+1$ variables. As in Chapter 9, put $B_e(n) \iff \forall \vec{x} \in \mathbb{N}^k p_e(n, \vec{x}) \neq 0$ and $d = \{e : \exists n B_e(n)\}$.

Proposition B.2 (Derived: d is Σ_2^0). *The set d is Σ_2^0 .*

Proof. The relation $p_e(n, \vec{x}) \neq 0$ is decidable in $(e, n, \vec{x})$, since p_e is computed from e and evaluated on natural numbers. The set $\{(e, n) : \forall \vec{x} p_e(n, \vec{x}) \neq 0\}$ is therefore Π_1^0 , as a universal quantifier over the computable bijective image of $\mathbb{N}^k$ in $\mathbb{N}$ can be absorbed into one quantifier. Prefixing $\exists n$ gives a Σ_2^0 set. $\square$

Theorem B.3 (Source, Theorem A.3, with the derivation made explicit). *For $k \geq 9$ the set d is Σ_2^0 -complete under many-one reductions.*

Derivation from (S4) and (S5). By Proposition B.2 the set is Σ_2^0 . For hardness, let $S = \{(e, n) : \varphi_e(n) \downarrow\}$, which is recursively enumerable. By (S5) there is a polynomial $q(e, n, x_1, \dots, x_9)$ with $\varphi_e(n) \downarrow \iff \exists \vec{x} q(e, n, \vec{x}) = 0$. For each e the polynomial $q(e, \cdot)$ in the variables $(n, \vec{x})$ occurs in the enumeration, at an index $f(e)$ that can be found effectively, after padding to k unknowns if $k > 9$. Then $\varphi_e(n) \uparrow \iff \forall \vec{x} q(e, n, \vec{x}) \neq 0 \iff B_{f(e)}(n)$, so $e \in \text{NTOT} \iff \exists n B_{f(e)}(n) \iff f(e) \in d$. Thus $\text{NTOT} \leq_m d$ and (S4) completes the argument. $\square$

The derivation is the one the source gives, and the cited ingredients are (S4) and (S5). The appendix therefore adds no new content to the source's proof of this theorem and records that it rests on those two facts.

Corollary B.4 (Derived). *For $k \geq 9$: (i) d is not computable relative to $\emptyset'$, so no total procedure decides $e \in d$ even with an oracle for the Halting problem; (ii) d is neither recursively enumerable nor co-recursively enumerable; (iii) every sound certification scheme whose set of accepted indices is recursively enumerable accepts a proper subset of d .*

Proof. (i) and (ii) are Lemma B.1 with $l = 2$. (iii) Let $\text{Acc} \subseteq d$ be the accepted set and suppose it is recursively enumerable. If $\text{Acc} = d$ then d would be recursively enumerable, contradicting (ii). $\square$

B.3 What the corollary does not say

Three limits of the result are recorded because they are easily misread.

First, the hardness is uniform. For a single fixed e the statement $e \in d$ is a fact, either true or false, and nothing here says that a proof of it cannot be found. The corollary concerns

the absence of a single procedure that works for all e and, in part (iii), the absence of a single recursively enumerable certificate scheme that is both sound and complete. Chapter 11 draws the practical consequence that such schemes can be sound and partial, and no more.

Second, the bound $k \geq 9$ is the state of the literature on (S5). The results do not assert decidability for fewer unknowns, and the absence of a hardness proof for $k = 8$ is not evidence of decidability there.

Third, the corollary concerns the set d and not, without a further convention, the set of indices at which statement correspondence holds. The step from one to the other is the subject of the next section.

B.4 From d to statement correspondence

Proposition B.5 (Derived: the hidden-existence family under the convention). *Let, for each e , $V_e = (N_e, P_e, \mathcal{I}, \Gamma, F_e, \pi_e)$ be the instance of Chapter 9. Assume (a) $\Gamma \vdash \pi_e : F_e$ for every e , uniformly in e ; (b) the convention that a specification whose defining term is undefined corresponds to no formal proposition; and (c) Lemma 16.5, that for $e \in d$ the formal term denotes the least n with $B_e(n)$. Then $\{e : K(V_e)\} = \mathbb{N}$, $K(V_e)$ is decidable uniformly in e , and $\{e : C(V_e)\} = d$.*

Proof. By (a), $K(V_e)$ holds for every e , and a uniform proof-checking procedure decides it. If $e \in d$ then by (c) the denotations of N_e and F_e agree, since the identity $(r_e + 1)^2 = r_e^2 + 2r_e + 1$ is then stated of the same rational $r_e = 1/(n_e + 1)$ in both. So $C(V_e)$ holds. If $e \notin d$ then N_e has an undefined defining term, so by (b) it corresponds to no formal proposition, and $C(V_e)$ fails. Hence $\{e : C(V_e)\} = d$. $\square$

Corollary B.6 (Derived: a gap beyond Δ_2^0 , conditional). *Under the hypotheses of Proposition B.5, and for $k \geq 9$, there is no total computable function, and no function computable relative to $\emptyset'$, that decides $C(V_e)$ uniformly in e , while $K(V_e)$ is decidable. The set $\{e : C(V_e)\}$ is neither recursively enumerable nor co-recursively enumerable.*

Proof. Combine Proposition B.5 with Corollary B.4. $\square$

The corollary is conditional on three things that the main text states: the uniform provability (a), the convention (b), and the faithful denotation of the library term on nonempty sets (c). The convention is the source's and a reader who adopts a different one, for instance that undefined specifications correspond to the default value, obtains a different set and a different theorem.

B.5 Trustworthy autoformalizers

Chapter 10 defines a trustworthy autoformalizer with refusal for a class $\mathcal{S}$. The following derivation records the reduction used in Theorem 10.5.

Proposition B.7 (Derived: RV-005 at level l). *Let $\mathcal{S}$ effectively embed the least-number constructions at level $l \geq 2$, that is, there is a computable $e \mapsto N_e^{(l)} \in \mathcal{S}$ with $N_e^{(l)}$ well defined iff $e \in d_l$, and suppose $d_l \notin \Delta_l^0$ (Theorem 10.2 of the source). If T is a trustworthy autoformalizer with refusal for $\mathcal{S}$, then T is not computable relative to $\emptyset^{(l-1)}$.*

Proof. Suppose T is computable relative to $\emptyset^{(l-1)}$. Trustworthiness gives $T(N) \neq \perp \iff N$ is well defined: if $T(N) \neq \perp$ then N is well defined by the first condition, and if N is well defined then $T(N) \neq \perp$ by the second. Hence $e \in d_l \iff T(N_e^{(l)}) \neq \perp$, and d_l would be computable relative to $\emptyset^{(l-1)}$, so $d_l \in \Delta_l^0$ by (S2), a contradiction. $\square$

The proposition is relative to the source's Theorem A.4 for the hardness of d_l , which is not reproved here. For $l = 2$ it is a consequence of Theorem B.3 and Lemma B.1. The statement that the combined problem over all l has $\text{SCI} = \infty$ follows from the equivalence of SCI_A with the arithmetical hierarchy recorded by the source, together with the fact that for each l the problem d_l is a subproblem requiring index at least l .

B.6 Summary of dependencies

Claim	Rests on	Status
$d \in \Sigma_2^0$	Definition of d	Derived
d is Σ_2^0 -complete	(S4), (S5)	Source, derivation recorded
No decider with $\emptyset'$ oracle	Lemma B.1	Derived
No complete r.e. certificate scheme	Corollary B.4(iii)	Derived
$\{e : C(V_e)\} = d$	(a), (b), (c)	Derived, conditional
RV-005 at level l	Theorem A.4 of the source	Source, reduction derived
$\text{SCI} = \infty$	Equivalence of SCI_A with the hierarchy	Source

Appendix C

A Catalogue of Counterexamples

The separation results of the monograph are proved by exhibiting instances, and this appendix collects them in one place with the properties each instance has and lacks. Each entry states what is being separated from what, gives the construction in enough detail to be checked, and records the status of the entry. Three statuses are used, and an entry may carry two. *Verified* means that the claims about the construction are checked in the entry itself by elementary computation or argument. *Source-reported* means that a feature of a real artifact is taken from the account of Bastounis, Circelli, and Hansen and has not been checked here. *Toy* means a deliberately small instance that exhibits the structure and carries no empirical content.

C.1 Instances that separate the three judgments

C1. A correct proof of a true theorem by a different argument than the one supplied (Chapter 1)

Separates kernel acceptance and statement correspondence from validity of the supplied argument. *Construction.* The theorem is $p(x) = x^3 - x^2 - x + 1 \geq 0$ for $x \geq -1$. The supplied argument asserts $p(x) = (x-1)(x+1)^2$. Expanding gives $(x-1)(x^2+2x+1) = x^3+x^2-x-1 \neq p(x)$, so the factorization is wrong. The correct factorization is $(x+1)(x-1)^2 = (x+1)(x^2-2x+1) = x^3 - x^2 - x + 1$, and nonnegativity follows for $x \geq -1$ from $x+1 \geq 0$ and $(x-1)^2 \geq 0$. *Status.* The arithmetic is verified. That a model's Lean translation corrected the multiplicities and compiled is source-reported.

C2. A faithful statement proved by a route without counterparts (Chapter 16, RV-002)

Separates $K \wedge C$ from A . *Construction.* For real symmetric A , the claim is $\text{tr}(A^2) \geq 0$. One argument diagonalizes, giving $\sum \lambda_i^2$. The other uses symmetry, $\text{tr}(A^2) = \sum_{i,j} a_{ij}a_{ji} = \sum_{i,j} a_{ij}^2$. Both are correct. *Status.* The two computations are verified. That the Lean proof takes the second route is source-reported. The separation holds under a strict argument-correspondence relation that counts a change of basis and a symmetry rewrite as different inferences, and a liberal relation would not separate them.

C3. Two true statements, one proof (RV-001)

Separates K from C . *Construction.* The formal statement says there are infinitely many odd numbers. The specifications N_1 , “there are infinitely many odd numbers”, and N_2 , “there are infinitely many primes”, are both true. The kernel judgment is identical for both. Correspondence distinguishes them only under an intensional equivalence relation, since under logical equivalence of true sentences they correspond equally. *Status.* Verified, with that hypothesis on $\equiv_{\mathcal{I}}$ stated.

C4. A total default for a partial term (Chapters 9 and 16)

Separates K , which holds uniformly, from C , which holds exactly on the set d . *Toy instance.* The real family needs $k \geq 9$ unknowns. The structure can be seen with one unknown. Let $B(n) \iff \forall x \ x^2 - 2 \neq 0$, which holds for every n over the natural numbers, so the least n is 0 and the term is well defined. Let instead $B(n) \iff \forall x \ x \neq n$, which fails for every n , so no least n exists, the specification’s term is undefined, and a library infimum on the empty set returns the default 0. In both cases the closing identity $(r+1)^2 = r^2 + 2r + 1$ is a ring identity and the proof is the same. *Status.* The toy instance is verified and is a toy. The hardness of the real family is Appendix B, with the source’s Theorem A.3.

C.2 Instances for the ledger**C5. A chain that preserves an open obligation (RV-018, part i; RV-009)**

Separates preservation from discharge. *Construction.* One obligation o_0 with content p and status open. At each step the single successor has content p and status open, and $\Delta_i = \Theta_i = \emptyset$. Then $\text{Ctx}_i(o_{i-1}) = \{p\} \models_{\mathcal{I}} p$ by reflexivity, so every step is preserving, and nothing is ever discharged. *Status.* Verified.

C6. A discharge that does not secure preservation elsewhere (RV-018, part ii)

Separates discharge of one obligation from preservation of another. *Construction.* Contents p, q, r with $\{p, q\} \not\models_{\mathcal{I}} r$. Obligations d with content q and o with content r . The transition discharges d , gives o the single successor of content p , and assumes nothing. Then $\text{Ctx}_1(o) = \{p, q\} \not\models_{\mathcal{I}} r$, so the transition is non-preserving at o , although d is discharged. The discharged d is preserved at its own step, since $q \in \text{Ctx}_1(d)$. *Status.* Verified, given a consequence relation with $\{p, q\} \not\models_{\mathcal{I}} r$, for instance derivability from three independent atoms.

C7. Premises that sum to the claim without any one implying it (RV-019)

Separates weaker successors from failure of preservation. *Construction.* A meet m of p and q with $\{p, q\} \models_{\mathcal{I}} m$, $m \models_{\mathcal{I}} p$, $m \models_{\mathcal{I}} q$, $\{p\} \not\models_{\mathcal{I}} m$ and $\{q\} \not\models_{\mathcal{I}} m$. A model is $\mathcal{L} = \{p, q, m\}$ with derivability from the rules $p, q \Rightarrow m$, $m \Rightarrow p$, $m \Rightarrow q$. The obligation o with content m has two successors with contents p and q , each strictly weaker, and the transition is nevertheless preserving. *Status.* Verified, with the existence of a meet as a hypothesis on the content language.

C.3 Instances for substitution and translation

C8. An answer-based check that cannot tell two questions apart (Chapter 7)

Illustrates an answer collision. *Construction.* Task T : does $\sum_{n \geq 1} n^{-2}$ converge? Task T' : does $\sum_{n \geq 1} n^{-3}$ converge? Both answers are yes. A certifier that maps an answer of yes to acceptance accepts a correct solution of T' as a solution of T and could not accept it otherwise. *Status.* Verified. The tasks are different questions with the same solution set over the answer space.

C9. A proof of a weaker theorem (RV-004)

Illustrates the enlarging direction. *Construction.* Take H' strictly stronger than H and $G := H'$. The implication $H' \rightarrow H'$ is derivable, while $H \rightarrow H'$ is derivable if and only if $H \models_{\mathcal{I}} H'$, which is excluded. *Status.* Verified, under the deduction property.

C10. A reward that cannot distinguish two trajectories (RV-010)

Separates the reward from the task predicate. *Construction.* The posed task is $x^2 = 9$ with $x > 0$. The trajectory h_1 solves it directly and ends at 3. The trajectory h_2 restates the problem as $x^3 = 27$, solves that over the reals, and also ends at 3. A reward that checks only the final answer gives $R(h_1) = R(h_2)$, while the task predicate gives $Q(h_1) = 1$ and $Q(h_2) = 0$, since h_2 solved a different problem. By Theorem 19.1 no classifier of the form $g \circ R$ is correct on both. *Status.* Toy: the trajectories and their task values are stipulated so that the answers coincide.

C11. A round trip that agrees and is wrong in both directions (RV-007)

Separates round-trip agreement from fidelity. *Construction.* $D = \{“n \text{ is even}”, “n \text{ is odd}”\}$. The forward translation exchanges the two formal predicates and the reverse translation exchanges them back. The round trip is the identity on D , and both translations are unfaithful at both inputs. *Status.* Verified. The real-world analogue, a shared library convention that both directions inherit, is the silent default of C4 and is not an instance of this construction.

C.4 Instances from the Navier–Stokes formalization

C12. A derivative shifted by one (Chapter 12)

Illustrates a weakening. *Construction.* The natural-language estimate (8.19) controls the derivative with $m + 4$ derivatives on the right, and the Lean statement uses $m + 5$. The Lean statement is the weaker one. *Status.* Source-reported and provisional. The classification as a weakening depends on the source’s claim that the Lean estimate does not entail the original. The monograph’s reconstruction of the exponents, $(1 + |k|)^{-3}$ against $(1 + |k_1| + |k_2|)^{-4}$ with a lattice-sum threshold of $s > 2$, is verified arithmetic and does not show that the proofs differ in strategy.

C13. A pressure-flux estimate with a changed quantifier (Chapter 13)

Illustrates a lateral change. *Construction.* The natural-language estimate (10.19) holds in B_R for almost every t . The Lean estimate also depends on a further quantity A_R and holds for all t . Neither estimate is reported to entail the other. *Status.* Source-reported and provisional, for the same reason as C12.

C.5 Summary

Entry	Separates	Chapter	Status
C1	Proof validity from kernel acceptance	1	Verified, translation reported
C2	$K \wedge C$ from A	16	Verified, route reported
C3	K from C	16	Verified, intensional $\equiv$
C4	K from C , uniformly	9, 16, App. B	Toy; family from source
C5	Preservation from discharge	App. A	Verified
C6	Discharge from preservation elsewhere	App. A	Verified
C7	Weaker successors from failure	App. A	Verified
C8	Questions with one answer	7	Verified
C9	Stronger from weaker theorem	16	Verified
C10	Reward from task	19	Toy
C11	Round trip from fidelity	8	Verified
C12	Statement strength	12	Source-reported
C13	Quantifier and dependence	13	Source-reported

Appendix D

A Lean 4 Prototype: Declarations and Their Status

Status of this appendix: not compiled. No Lean toolchain was available in the environment in which the manuscript was written (checked 8 October 2026: no `lean`, `lake` or `elan` on the path, and release metadata could not be retrieved). Every declaration below is unchecked. No declaration is a verified proved declaration, and the prototype is not a formal verification of any result of this monograph.

D.1 Three activities kept apart

Three different activities could be meant by an appendix of Lean 4 [14] code, and this one performs only the first.

- (1) *Documenting an implementation*: writing down, in the syntax of a proof assistant, definitions and statements that are intended to represent objects of Appendix A. This appendix does that.
- (2) *Verifying an implementation*: obtaining kernel acceptance for those declarations, with no `sorry` and a recorded list of axioms. This has *not* been done. Until it is, the word “proved” is not applied below to any declaration, and the tags in Table D.2 state intentions, not results.
- (3) *Auditing a source claim*: checking a claim of a cited source. Nothing in this appendix does that. The module `MathlibToy.lean` restates one toy case reported by the source and does not examine the source’s theorems.

The distinction applies to the prototype with the force it has everywhere in the book. If the files compile, that shows that the declarations are well typed, and that the declarations tagged as proofs are accepted by the kernel. It does not show that the definitions match Appendix A. That correspondence is the question the whole monograph is about, and it is open here, with the divergences listed in Section D.5 as the part of the open obligation that is already known.

Item	Record
Compilation status	Not compiled. No module has been checked by any Lean toolchain.
Lean toolchain	Not determined. The file <code>lean-toolchain</code> contains a deliberate placeholder (UNPINNED). The version used at first compile is to be recorded here, together with the output of <code>lean -version</code> .
Mathlib revision	Not determined. Needed only by <code>VWC/MathlibToy.lean</code> . The <code>lakefile.lean</code> dependency is deliberately unpinned. The revision is to be recorded from <code>lake-manifest.json</code> at first compile.
Version constraints assumed	Core modules use only core Lean 4 syntax and avoid the library functions that were renamed in recent releases (<code>List.bind</code> , <code>List.join</code> , <code>List.flatMap</code> , <code>List.flatten</code>). That the files elaborate on a given release is an untested assumption.
Mathlib lemma names assumed	<code>Nat.sInf_empty</code> , <code>Nat.find</code> , <code>Nat.find_spec</code> , <code>Set.mem_setOf_eq</code> , <code>Set.mem_empty_iff_false</code> . Not checked against any Mathlib revision.
Axioms declared	None. The Tarskian conditions and the soundness hypothesis are structure fields or theorem hypotheses.
Placeholders	Two sorry: <code>trace</code> and <code>compose_first</code> .
Files	<code>VWC / Ledger . lean</code> , <code>VWC / Consequence . lean</code> , <code>VWC / ToyExamples.lean</code> , <code>VWC/MathlibToy.lean</code> , <code>lakefile.lean</code> , <code>lean-toolchain</code> .

Table D.1: Reproducibility record for the Lean prototype. Entries marked “not determined” are open and must be filled at first compile.

D.2 Reproducibility record

The record is an obligation in the book’s sense. It is open, its resolution is the output of a toolchain run, and nothing in the text should be read as supplying it.

D.3 Classification of declarations

Each declaration carries one of the following tags in its docstring.

- [DEF] a definition or structure: it introduces a name and asserts nothing.
- [STMT] a theorem statement whose proof is the placeholder `sorry`. Elaboration of the statement is all that compilation would check.
- [PROOF-UNCHECKED] a theorem with a proof script or term written by the author and not yet run. It becomes a proved declaration only after compilation without `sorry` and with an acceptable `#print axioms` output.
- [DEF-CHECK] an `example` computing a value on a toy input, a sanity check of a definition.

No declaration is tagged as an axiom because none exists. Every `sorry` is counted in Table D.1. The command `#print axioms` is included after each theorem so that the first compile reports its dependencies.

Declaration	Tag	Intended counterpart in the book
<i>Ledger.lean</i>		
Status, Kind	DEF	statuses and obligation kinds (Def. A.9)
Obligation	DEF	obligation, reduced to id, kind, content, status
Transition, Stage	DEF	transition certificate $(Q_i, \Delta_i, \Theta_i)$ with its input list
succs, step, desc	DEF	$\text{succ}_i(o)$ and $S_n(o)$
Covers, WellFormed,	DEF	well-formedness of a ledger
Accepts, WFLedger		
DischargedIn	DEF	certified discharge in the ancestry of o
step_append	PROOF-UNCHECKED	auxiliary distributivity of step
trace	STMT, sorry	Theorem A.15 (RV-020)
<i>Consequence.lean</i>		
Consequence	DEF	Tarskian relation, C1–C3 as fields
stageCtx, pool, cumCtx	DEF	$\text{Ctx}_i(o)$, A_n , $\Gamma_n(o)$
LocallyPreserving	DEF	the hypothesis of the first statement of RV-008
compose_first	STMT, sorry	first statement of Theorem A.17
SoundFor	DEF	soundness of $\models_{\mathcal{I}}$ for the instance predicate $\text{Hold}_{\mathcal{I}}$
transport_core	PROOF-UNCHECKED	only the final step of the second statement of RV-008; see below
<i>ToyExamples.lean</i>		
t1, t2, toy	DEF	a two-stage toy ledger
two examples	DEF-CHECK	computation of desc on toy
<i>MathlibToy.lean</i> (requires Mathlib)		
B, nSilent, nChecked	DEF	B_e for $p_e = x_1 + x_2 - n$, the silent default, and a hypothesis-carrying alternative
B_false, B_set_empty,	PROOF-	the toy case of Chapter 9 and the source’s Section 4.2
nSilent_eq_zero,	UNCHECKED	
nChecked_spec,		
no_witness		

Table D.2: Every declaration with its tag. No tag records a verified result.

D.4 Source listings

The listings are the files of the companion directory, included verbatim. Docstrings repeat the status banner so that the file is not misread when separated from this appendix.

VWC/Ledger.lean

```

1  /-
2  VWC/Ledger.lean
3  Companion to Appendix D of "Verification Without Correspondence".
4
5  VERIFICATION STATUS: NOT COMPILED. No Lean toolchain was available when
6  this file was written. Every declaration below is unchecked. The tags
7  [DEF], [STMT], [PROOF-UNCHECKED] describe what the author INTENDS each
8  declaration to be; none is a verified proved declaration until this file
9  has been compiled and '#print axioms' has been inspected.
10
11 Imports: none (core Lean 4 only).
```

```

12 | Axioms declared by this file: none.
13 | Placeholders: every 'sorry' is listed in the Appendix D table.
14 | -/
15 |
16 | namespace VWC
17 |
18 | /-- [DEF] Obligation statuses (Definition def:status of Appendix A). -/
19 | inductive Status where
20 |   | open_
21 |   | supported
22 |   | discharged
23 |   | refuted
24 |   | defeated
25 |   deriving DecidableEq
26 |
27 | /-- [DEF] Kinds of obligation (a subset of the kinds used in the book). -/
28 | inductive Kind where
29 |   | formal
30 |   | statement
31 |   | argument
32 |   | dependency
33 |   | assumption
34 |   deriving DecidableEq
35 |
36 | /-- [DEF] An obligation. The book's tuple (tau, phi, rho, E, D, sigma) is
37 |   reduced here to an identifier, a kind, a content and a status. The
38 |   evidence, dependency and residue fields are NOT modelled. -/
39 | structure Obligation (L : Type) where
40 |   id : Nat
41 |   kind : Kind
42 |   content : L
43 |   status : Status
44 |
45 | /-- [DEF] A transition certificate  $T_i = (Q_i, \Delta_i, \Theta_i)$ . 'succ'
46 |   lists pairs (o, o') in  $Ob_{i-1} \times Ob_i$ ; 'disch' is  $\Delta_i$ ; 'assumed' is
47 |    $\Theta_i$ . Obligations are referred to by 'Nat' identifiers. -/
48 | structure Transition (L : Type) where
49 |   succ : List (Nat × Nat)
50 |   disch : List Nat
51 |   assumed : List L
52 |
53 | /-- [DEF] A stage pairs a transition with the list of identifiers of the
54 |   obligations of  $Ob_{i-1}$  that it must account for. -/
55 | structure Stage (L : Type) where
56 |   inputs : List Nat
57 |   tr : Transition L
58 |
59 | /-- [DEF] succ_i(o): the successors of 'o' under a transition. -/
60 | def succs {L : Type} (t : Transition L) (o : Nat) : List Nat :=
61 |   (t.succ.filter (fun p => p.1 == o)).map (fun p => p.2)
62 |
63 | /-- [DEF] One step of the descendant relation, extended to lists. -/
64 | def step {L : Type} (t : Transition L) : List Nat → List Nat
65 |   | [] => []
66 |   | o :: os => succs t o ++ step t os
67 |
68 | /-- [DEF] S_n(o), generalised to a list: the descendants after all stages. -/
69 | def desc {L : Type} : List (Stage L) → List Nat → List Nat
70 |   | [], S => S
71 |   | s :: ss, S => desc ss (step s.tr S)
72 |
73 | /-- [DEF] 'o' is discharged by the transition or is accounted for by a
74 |   successor (the well-formedness condition on one input). -/
75 | def Covers {L : Type} (t : Transition L) (o : Nat) : Prop :=
76 |   o ∈ t.disch ∨ ∃ p ∈ t.succ, p.1 = o
77 |

```

```

78  /-- [DEF] A stage is well-formed when it accounts for all its inputs. -/
79  def WellFormed {L : Type} (s : Stage L) : Prop :=
80    ∀ o ∈ s.inputs, Covers s.tr o
81
82  /-- [DEF] Is 'o' an input of the first stage of a ledger? (Vacuous for the
83    empty ledger.) -/
84  def Accepts {L : Type} : List (Stage L) → Nat → Prop
85    | [], _ => True
86    | s :: _, o => o ∈ s.inputs
87
88  /-- [DEF] A ledger is well-formed when each stage is, and each successor
89    produced by a stage is an input of the next stage. -/
90  def WFLedger {L : Type} : List (Stage L) → Prop
91    | [] => True
92    | s :: rest =>
93      WellFormed s ∧ (∀ p ∈ s.tr.succ, Accepts rest p.2) ∧ WFLedger rest
94
95  /-- [DEF] 'o' has a certified discharge in its ancestry: some descendant is
96    in Delta_k at some stage k. -/
97  def DischargedIn {L : Type} : List (Stage L) → Nat → Prop
98    | [], _ => False
99    | s :: rest, o =>
100      o ∈ s.tr.disch ∨ ∃ o' ∈ succs s.tr o, DischargedIn rest o'
101
102  /-- [PROOF-UNCHECKED] 'step' distributes over append. A short induction;
103    the 'simp' call is the step most likely to need adjustment. -/
104  theorem step_append {L : Type} (t : Transition L) (a b : List Nat) :
105    step t (a ++ b) = step t a ++ step t b := by
106    induction a with
107    | nil => rfl
108    | cons x xs ih => simp [step, ih, List.append_assoc]
109
110  /-- [STMT; proof is 'sorry'] Theorem thm:trace (RV-020), in the form: in a
111    well-formed ledger, an input obligation with no certified discharge in
112    its ancestry has a descendant after the last stage.
113    Plan: induction on the ledger, generalising 'o'. In the cons case,
114    'Covers' and 'not DischargedIn' give a successor p.2 of o with
115    'not DischargedIn rest p.2' and 'Accepts rest p.2'; the induction
116    hypothesis gives 'desc rest [p.2] ≠ []'; 'step_append' and a lemma
117    'desc ss (a ++ b) = desc ss a ++ desc ss b' lift this to
118    'desc rest (step s.tr [o])'. That distributivity lemma is not stated
119    above and is part of the unfinished work. -/
120  theorem trace {L : Type} (ledger : List (Stage L)) (o : Nat)
121    (hwf : WFLedger ledger) (hin : Accepts ledger o)
122    (hnd : ¬DischargedIn ledger o) :
123    desc ledger [o] ≠ [] := by
124    sorry
125
126  #print axioms step_append
127  #print axioms trace
128
129  end VWC

```

VWC/Consequence.lean

```

1  /-
2  VWC/Consequence.lean
3  Companion to Appendix D of "Verification Without Correspondence".
4
5  VERIFICATION STATUS: NOT COMPILED. Depends on VWC/Ledger.lean.
6  Imports: VWC.Ledger (core Lean 4 only; no Mathlib).
7  Axioms declared by this file: none. The Tarskian conditions C1-C3 and the
8  soundness hypotheses are carried as structure fields or theorem

```

```

9   hypotheses, not as global axioms.
10  -/
11  import VWC.Ledger
12
13  namespace VWC
14
15  /-- [DEF] A Tarskian consequence relation on contents of type 'L', over
16       finite contexts represented as lists. Fields 'refl', 'mono', 'cut' are
17       the conditions C1-C3 of Appendix A. Divergence from the book: contexts
18       are lists, not sets, and are finite. -/
19  structure Consequence (L : Type) where
20    ent : List L → L → Prop
21    refl : ∀(G : List L) (f : L), f ∈ G → ent G f
22    mono : ∀(G D : List L) (f : L), (∀ x, x ∈ G → x ∈ D) → ent G f → ent D f
23    cut : ∀(G : List L) (f g : L), ent G f → ent (f :: G) g → ent G g
24
25  /-- [DEF] The certified context  $Ctx_i(o)$  of an obligation 'o' at a stage,
26       with 'cont' giving the content of an obligation identifier. -/
27  def stageCtx {L : Type} (cont : Nat → L) (s : Stage L) (o : Nat) : List L :=
28    (succs s.tr o).map cont ++ s.tr.disch.map cont ++ s.tr.assumed
29
30  /-- [DEF] The cumulative pool  $A_n$  of discharged contents and assumptions. -/
31  def pool {L : Type} (cont : Nat → L) : List (Stage L) → List L
32  | [] => []
33  | s :: rest => s.tr.disch.map cont ++ s.tr.assumed ++ pool cont rest
34
35  /-- [DEF] The cumulative context  $\Gamma_n(S)$  of a list of obligations. -/
36  def cumCtx {L : Type} (cont : Nat → L) (ledger : List (Stage L))
37    (S : List Nat) : List L :=
38    (desc ledger S).map cont ++ pool cont ledger
39
40  /-- [DEF] Each stage is preserving at every obligation reached from 'S'
41       (the local hypothesis of the first statement of RV-008). -/
42  def LocallyPreserving {L : Type} (C : Consequence L) (cont : Nat → L) :
43    List (Stage L) → List Nat → Prop
44  | [], _ => True
45  | s :: rest, S =>
46    (∀ o ∈ S, C.ent (stageCtx cont s o) (cont o)) ∧
47    LocallyPreserving C cont rest (step s.tr S)
48
49  /-- [STMT; proof is 'sorry'] First statement of Theorem thm:compose
50       (RV-008): local preservation along the descendants gives
51        $\Gamma_n(o) \models cont(o)$ . Plan: a derived lemma
52       'ent_of_all : (forall x in D, ent G x) → ent D f → ent G f'
53       (from 'cut' and 'mono', by induction on 'D') together with 'mono' and
54       induction on the ledger. Not attempted. -/
55  theorem compose_first {L : Type} (C : Consequence L) (cont : Nat → L)
56    (ledger : List (Stage L)) (o : Nat)
57    (hp : LocallyPreserving C cont ledger [o]) :
58    C.ent (cumCtx cont ledger [o]) (cont o) := by
59    sorry
60
61  /-- [DEF] Soundness of a consequence relation for an interpretation, where
62       'hold' is the instance predicate Hold_I (instance data). -/
63  def SoundFor {L : Type} (C : Consequence L) (hold : L → Prop) : Prop :=
64    ∀(G : List L) (f : L), C.ent G f → (∀ x, x ∈ G → hold x) → hold f
65
66  /-- [PROOF-UNCHECKED] The core of the second statement of RV-008: if the
67       context entails 'f', the relation is sound, and every premise holds,
68       then 'f' holds. Immediate from the definition of 'SoundFor'. The
69       book's second statement also needs admissibility, warrant, and the
70       classification of premise sources; none of these is modelled here, so
71       this is NOT a formalisation of that statement. -/
72  theorem transport_core {L : Type} (C : Consequence L) (hold : L → Prop)
73    (sound : SoundFor C hold) (G : List L) (f : L) (h : C.ent G f)
74    (hG : ∀x, x ∈ G → hold x) : hold f :=

```

```

75 | sound G f h hG
76 |
77 | #print axioms compose_first
78 | #print axioms transport_core
79 |
80 | end VWC

```

VWC/ToyExamples.lean

```

1 | /-
2 |   VWC/ToyExamples.lean
3 |   Companion to Appendix D of "Verification Without Correspondence".
4 |
5 |   VERIFICATION STATUS: NOT COMPILED. Depends on VWC/Ledger.lean.
6 |   Imports: VWC.Ledger (core only).
7 |   These 'example's are sanity checks of the DEFINITIONS on a two-stage toy
8 |   ledger. They do not test any theorem of the book, and a successful
9 |   compile would not show that the definitions match Appendix A.
10 | -/
11 | import VWC.Ledger
12 |
13 | namespace VWC
14 |
15 | /- Toy transitions over contents of type 'String'. Obligation 0 is
16 |    carried to obligation 1 at stage one, and obligation 1 is discharged at
17 |    stage two. -/
18 | def t1 : Transition String := { succ := [(0, 1)], disch := [], assumed := [] }
19 | def t2 : Transition String := { succ := [], disch := [1], assumed := [] }
20 |
21 | def toy : List (Stage String) :=
22 |   [ { inputs := [0], tr := t1 }, { inputs := [1], tr := t2 } ]
23 |
24 | /- [DEF-CHECK; unchecked] Descendants of obligation 0 after both stages:
25 |    none, because obligation 1 is discharged at stage two. -/
26 | example : desc toy [0] = [] := by decide
27 |
28 | /- [DEF-CHECK; unchecked] After stage one alone, obligation 1 remains. -/
29 | example : desc [{ inputs := [0], tr := t1 } : Stage String] [0] = [1] := by
30 |   decide
31 |
32 | end VWC

```

VWC/MathlibToy.lean

```

1 | /-
2 |   VWC/MathlibToy.lean
3 |   Companion to Appendix D of "Verification Without Correspondence".
4 |
5 |   VERIFICATION STATUS: NOT COMPILED. REQUIRES MATHLIB (version not
6 |   determined; to be recorded at first compile). Not imported by the other
7 |   modules. Several lemma names below ('Nat.sInf_empty', 'Nat.find',
8 |   'Nat.find_spec', 'Set.mem_setOf_eq') are assumed from memory of Mathlib
9 |   and may have been renamed.
10 |
11 |   Purpose: illustrate, for the polynomial  $p_e(n, x_1, x_2) = x_1 + x_2 - n$ 
12 |   reported by Bastounis, Circelli and Hansen (Example 4.2 of their paper),
13 |   (a) the silent default of 'sInf' on an empty set of naturals and (b) a
14 |   hypothesis-carrying definition that cannot be applied unless existence
15 |   has been supplied. This restates a toy case; it formalises nothing of
16 |   Theorem A.3 or A.4.
17 | -/

```

```

18 import Mathlib
19
20 namespace VWC
21
22 /-- [DEF] B n: for all natural x1 x2, x1 + x2 - n is nonzero (over Int). -/
23 def B (n : Nat) : Prop :=
24   ∀x1 x2 : Nat, ((x1 : Int) + (x2 : Int) - (n : Int)) ≠ 0
25
26 /-- [PROOF-UNCHECKED] No 'n' satisfies 'B': take (x1, x2) = (n, 0). -/
27 theorem B_false (n : Nat) : ¬B n := by
28   intro h
29   exact h n 0 (by simp)
30
31 /-- [PROOF-UNCHECKED] The witness set is empty. -/
32 theorem B_set_empty : {n : Nat | B n} = ∅ := by
33   ext n
34   simp only [Set.mem_setOf_eq, Set.mem_empty_iff_false, iff_false]
35   exact B_false n
36
37 /-- [DEF] The silent-default definition: 'sInf' of an empty set is 0. -/
38 noncomputable def nSilent : Nat := sInf {n : Nat | B n}
39
40 /-- [PROOF-UNCHECKED] The silent default assigns 0 although no witness
41   exists. Relies on the assumed name 'Nat.sInf_empty'. -/
42 theorem nSilent_eq_zero : nSilent = 0 := by
43   unfold nSilent
44   rw [B_set_empty]
45   exact Nat.sInf_empty
46
47 /-- [DEF] Hypothesis-carrying definition: applicable only given a proof of
48   existence. Uses classical decidability to apply 'Nat.find'. -/
49 open Classical in
50 noncomputable def nChecked (h : ∃n, B n) : Nat := Nat.find h
51
52 /-- [PROOF-UNCHECKED] The checked definition satisfies the defining
53   property. -/
54 open Classical in
55 theorem nChecked_spec (h : ∃n, B n) : B (nChecked h) :=
56   Nat.find_spec h
57
58 /-- [PROOF-UNCHECKED] For this toy case existence fails, so 'nChecked'
59   can never be applied. -/
60 theorem no_witness : ¬∃n, B n := by
61   rintro ⟨n, hn⟩
62   exact B_false n hn
63
64 end VWC

```

D.5 Known divergences from Appendix A

The following divergences are known at the time of writing. They are modelling choices or omissions that make the Lean objects differ from those of Appendix A, and each is an open statement-correspondence obligation for the prototype.

- (i) Contexts are *lists*, not sets, and are finite. The book's consequence relation is defined on arbitrary sets of contents.
- (ii) Obligations are identified by **Nat**. The tuple $(\tau, \varphi, \rho, E, D, \sigma)$ is reduced to four fields, and the evidence, dependency and residue components are not modelled.

- (iii) The book gives each $\mathcal{O}_k$ as a set and $Q_i \subseteq \mathcal{O}_{i-1} \times \mathcal{O}_i$. The prototype stores for each stage an explicit list of inputs and imposes the chaining condition **Accepts** between successive stages. This condition is the author’s modelling choice and is not a clause of Appendix A.
- (iv) **DischargedIn** takes “certified discharge” to mean membership in Δ_k for some descendant at some stage. Warrant, defeat and the status function are not modelled.
- (v) The dependency graph, acyclicity, admissibility conditions (A1) and (A2), transition typing, and the status function σ_t are not modelled. Consequently the second statement of Theorem A.17, which depends on them, is not formalised. The lemma **transport_core** records only its last step, and calling it a formalisation of RV-008 would be an error of exactly the kind the book describes.
- (vi) RV-001, RV-002, RV-004, RV-005, RV-015, RV-018, RV-019 and RV-021 are not represented.
- (vii) The cumulative pool **pool** lists contents in the order of the stages, with repetitions. Under the list representation this is harmless for **mono**, but it is a representation choice.

D.6 What a successful compile would and would not show

If the four modules compiled with exactly the two declared placeholders, the evidence would be: the definitions are well typed; the statements of **trace** and **compose_first** elaborate; the declarations tagged **PROOF-UNCHECKED** are accepted by the kernel, with dependencies on axioms as reported by **#print axioms**. It would not establish that the statements are the theorems of Appendix A. A reviewer who wished to establish that would repeat for the prototype the procedure the book recommends for any formalization: read each definition against its counterpart, tabulate the divergences, and tag the transition from the appendix to the code as conservative, strengthening, weakening or lateral. The list in Section D.5 is the first line of that table. Several divergences make the Lean statements weaker or incomparable to the book’s, and no assertion is made here that they do not.

If **trace** or **compose_first** were completed, the result would be a machine-checked theorem about the prototype’s definitions. By the argument of Chapter 16 it would be a discharge of the formal obligation and a transition of the statement obligation whose type is to be determined.

D.7 Procedure for the first compile

- (1) Install a Lean toolchain; choose a release; write it into **lean-toolchain**; record **lean-version** in Table D.1.
- (2) For the Mathlib module, pin the revision in **lakefile.lean** and record it from **lake-manifest.json**.
- (3) Build the core modules first. Record every error verbatim and correct the declaration; do not weaken a statement to make it compile without recording the change in Section D.5.
- (4) Run the **#print axioms** commands and record their output.

- (5) Count `sorry`. Update the classification table: a declaration changes from PROOF-UNCHECKED to proved only after steps 3 to 5.
- (6) Only then revise the text of Section 21.4 so that the word “compiled” is applied to the declarations that were compiled.

D.8 Relation to the test suite of Chapter 21

Chapter 21 proposes a test suite with seeded unresolved obligations (the polynomial example, the symmetric-matrix example, the hidden-existence default, and the derivative and pressure-flux cases). None of these is encoded here, and the suite has not been run. The module `MathlibToy.lean` implements only the toy instance of the hidden-existence case. The remaining cases require the typing machinery of Appendix A that the prototype does not contain.

Appendix E

Navier–Stokes Source Audit

Status of this appendix: provisional, text-level audit. Prepared on 8 October 2026 by a single reviewer (an AI agent). Artifacts were read as text. No Lean code was compiled, no mathematics was independently re-derived, and no claim below is a refereed verification. The appendix separates what was confirmed by reading a primary artifact, what remains the report of the source paper, what the monograph interprets, and what is unverified together with the source that would resolve it.

E.1 Purpose, and what counts as primary

Chapters 12 to 14 were written from Bastounis, Circelli, and Hansen [3], without access to the artifacts it describes. Chapter 20 likewise reports material from its Section 5.1 at one remove. This appendix records what changed when some of those artifacts became readable.

Two levels of source must be kept apart. For the claims about OpenAI’s announced proof, the primary artifacts are OpenAI’s natural-language paper, its Lean repository, and the program outputs the repository contains. The paper of Bastounis, Circelli, and Hansen is a secondary account of those artifacts, and is primary only for its own claims: its theorems, its own arguments, and its forecasts. For the 2026 developments of Chapter 20, the primary artifacts are the cited paper, thread, declarations and blog post.

Each statement below carries one of four marks.

- P** Confirmed by reading the primary artifact during this audit. The reading is of text, by one reader.
- S** The report of the source paper [3], not confirmed here.
- I** An interpretation or classification made in the monograph, with its stated status.
- U** Unverified. The resolving source is named.

Two rules govern the register. An artifact that could not be reached is not evidence against the claims made about it: its claims stay marked S or U and are neither supported nor weakened by the failure of access. A secondary description is not promoted: when a source paper says a primary artifact contains something, the claim stays S until the artifact is read, and reading it changes the mark only for the passage actually read.

E.2 Artifacts consulted

Artifact	Identification and reading
OpenAI, <i>Finite time blowup for Navier–Stokes</i> [16]	PDF, 166 pages, linked from the repository README at <code>cdn.openai.com/pdf/32d9f210-8b73-45e0-91bc-82a30aef8a9a/navier-stokes.pdf</code> ; retrieved 8 October 2026; SHA-256 begins <code>0e779481c4da</code> . The PDF carries no version field. Read: Lemma 8.6 with (8.19) and its proof; Lemma 10.5 with (10.15) to (10.19) and the following energy estimate. Nothing else was read.
<code>openai</code> / <code>NavierStokesAndEuler</code> [17]	Cloned read-only; HEAD is commit <code>f9e8bc5b38b6e212696e8a30e3e91517af887bbd</code> , the commit cited by the source. Declared toolchain <code>leanprover/lean4:v4.34.0-rc2</code> ; Mathlib revision <code>85e3a25e006c35636f0e53b0e9296caca2685bc0</code> per <code>lake-manifest.json</code> . Read as text: the declarations named in Table E.2.
Rammal et al., <i>Formalizing mathematics at scale</i> [18]	arXiv:2605.29955, PDF retrieved 8 October 2026; SHA-256 begins <code>a2bcbf8cf4e4</code> . Abstract, Section 3, the table of per-book results and Appendix G were read; the remaining sections were not.
<code>facebookresearch</code> / <code>atlas-lean</code>	Cloned read-only; HEAD is commit <code>99cc25d8ba4a63e69844131ec508bb46ad526694</code> (7 October 2026). The root now describes a version 2 in preparation and keeps the original release in <code>v1/</code> . Only the README files and the review guidelines were read.
Leiden Declaration [12]	Zenodo record 20302944, PDF retrieved 8 October 2026; SHA-256 begins <code>f6c545ebfd69</code> . Searched for the quoted passage.
<i>A Severe Misalignment of AI in Mathematics</i> [1]	Zenodo record 22737750 (version 22737751), 2-page PDF retrieved 8 October 2026; SHA-256 begins <code>32e3afcefd9a</code> . Read in full.

Table E.1: Artifacts consulted in this audit, with retrieval details. The hash prefixes permit a later reader to check that the same files are being discussed.

Buzzard’s blog post *FLT: Anthropic has beaten me to it* [5] was read on 8 October 2026 through a page-fetching tool that returns summaries and quotations truncated to a fixed length, so only the passages quoted in item 32 were seen, and the full text was not read. The Anthropic announcement and the formalization’s own repository, to which the post links, were not read.

Not consulted: the Euler paper and the Euler directory of the repository; the Lean community discussion thread concerning the Meta project; and the experiment linked in Remark 3.4 of the source. An attempt to reach the thread failed. The community chat page loads only through a script and returned no messages, and the public archive of that chat lists no autoformalization stream and shows a last-update date of 28 February 2026, which precedes the thread. These failures say nothing about the thread’s contents. The figures of the source that reproduce chatbot output were not reproduced and carry no evidential weight here.

E.3 Audit of the two documented discrepancies

Table E.2: Claims about the Lean formalization and the natural-language paper. Line numbers are those of the repository at the pinned commit.

No.	Claim	Mark	Basis or resolving source
<i>Derivative case (Chapter 12)</i>			
1	Lemma 8.6 of the natural-language paper states $\ N^{-1}F\ _{C_y^m} \leq C_m \ F\ _{C_y^{m+4}}$ as (8.19) and its proof says that four more derivatives than the output leave a summable bound $(1 + k)^{-3}$.	P	Text of the PDF, Section 8.
2	The proof invokes the divisor bound (6.7) in the form $ v_t \cdot k ^{-1} \leq C(1 + k)$.	P	Text of the PDF.
3	Lean <code>norm _ derivativeWord _ inverse _ le</code> (line 1059 of <code>NavierStokes / SmoothFamilyTorusInverse .lean</code>) assumes sup-norm bounds on <code>xJet(w.length+5)</code> of the function and of its swap, and concludes a bound with constant <code>mixedLossConstantw.length*C</code> .	P	Text of the declaration.
4	That proof applies <code>coefficient_seminorm_bound</code> with order <code>w.length+1</code> ; that lemma (line 368 of <code>SmoothFourierData.lean</code>) takes jets of order $p+4$ and bounds by $\sum_k (\text{weight } k)^{-4}$, with weight $k = 1 + k_1 + k_2 $.	P	Text of both declarations and of <code>weight</code> in <code>TorusInverse.lean</code> .
5	Hence the count is $(m+1) + 4 = m+5$ for this declaration, against $m+4$ in (8.19).	P, I	The arithmetic is read off the text. Chapter 12 uses it as its explanation and marks it as the monograph's reconstruction.
6	<code>inverse_finiteJets</code> and <code>mixedJet_inverse_bound</code> also require order $n+5$, respectively <code>w.length+5</code> .	P	Lines 865 to 868 of <code>SmoothFamilyTorusInverse .lean</code> , and <code>ParametricTorusInverse .lean</code> line 654 onward. A second <code>mixedJet _ inverse _ bound</code> appears at line 1082 of <code>SmoothFamilyTorusInverse .lean</code> and was not read.
7	No declaration in the repository establishes (8.19) with $m+4$ derivatives.	U	A textual search of three files found no such variant; the search is not exhaustive. Resolving source: a search of the whole repository together with a compiled dependency analysis.

No.	Claim	Mark	Basis or resolving source
8	The Lean estimate is strictly weaker than (8.19), so the transition is a weakening.	S, I	The source asserts it after taking a supremum over multi-indices, an operation it notes is not done in the declaration. Chapter 12 types the transition as a weakening, provisional. What is confirmed is the difference in hypotheses (items 1 and 3). Non-entailment of (8.19) from the Lean estimate relative to the certified development is not confirmed.
9	The extra derivative is harmless, or harmful, for the rest of the argument.	U	Neither the source nor the monograph settles it. Resolving source: the places in Section 8 and following of the natural-language paper at which (8.19) is applied, read against the places at which the Lean declarations are applied.
<i>Pressure-flux case (Chapter 13)</i>			
10	Equation (10.19) of the natural-language paper bounds $ \int \pi w \cdot \nabla \chi_R $ by $C_T R^{-1}[(B_R+1)B_R^{1/2} + R^{-3/4}B_R^{3/4}]$, for almost every $t \in (0, T)$, in the proof of Lemma 10.5.	P	Text of the PDF.
11	$B_R = \ \varphi_R^4 w\ _6$, $A_R = (\int \chi_R \nabla w ^2)^{1/2}$, $\chi_R = \varphi_R^8$, and (10.17) bounds $B_R \leq C(A_R + R^{-1}\ w\ _2)$.	P	Text of the PDF.
12	Lean <code>exists_uniform_actual_pressure_flux_bound</code> (line 576 of <code>NavierStokes/R3/PressureFlux.lean</code>) gives, for $R \geq 1$ and all $t \in (0, T)$, a bound in terms of <code>cutoffL6</code> , <code>dissipationRoot</code> and $R^{-7/4}$, in the form displayed in Chapter 13.	P	Text of the declaration.
13	<code>cutoffL6</code> is $\ \varphi^4 w\ _6$ and <code>dissipationRoot</code> is $(\int \varphi^8 \nabla w ^2)^{1/2}$, so that they play the roles of B_R and A_R .	P	Definitions in <code>R3 / ComparisonSetup.lean</code> . That <code>ComparisonCutoffs.cutoffR</code> is φ_R and <code>ComparisonCutoffs.weightR</code> is χ_R was not checked (U, minor).
14	The Lean bound depends on A_R and holds for every t ; the natural-language bound is in B_R alone and holds almost everywhere.	P	Items 10 and 12.

No.	Claim	Mark	Basis or resolving source
15	The Lean statement uses $(p - q)$ and $(u - v)$, opposite to $\pi = P - p$, $w = v - u$ in the paper, and the product is unaffected.	P	Text of both. The sign claim about the surrounding definitions beyond the statement was not checked.
16	The Lean bound is not comparable with (10.19), so the transition is lateral.	S, I	Chapter 13 types it lateral, provisional. Items 10 to 14 confirm the textual difference. Non-entailment in both directions is not confirmed.
17	After (10.19) the natural-language text uses (10.17) to convert powers of B_R into powers of A_R , noting that every power of A_R in the flux bounds is at most $3/2 < 2$.	P	Text of the PDF following (10.19). This is a fact about the natural-language argument that Chapter 13 does not discuss. Whether a bound of the Lean form could take the place of (10.19) in that absorption step is unexamined (U).
18	The two proofs differ as the source describes: factoring φ_R^4 against φ_R^2 ; $L^{3/2}$ boundedness of Riesz transforms against Sobolev embedding with L^2 boundedness; Hölder exponents; an interpolation at $12/5$ in the Lean proof only.	P (NL side), S (Lean side)	The natural-language side is present in the text (the $L^{3/2}$ bound and the φ_R^4 commutator identity (10.18)). The Lean proof route was not traced lemma by lemma.
19	A related Lean bound <code>regularized_flux_bound</code> exists and is not used in the proof of the main theorem.	P (existence), S (non-use)	Declared at line 38 of <code>R3StressPressureEstimate.lean</code> . Non-use requires a dependency analysis of the compiled development.
<i>Statements about the repository as a whole</i>			
20	The repository states that it contains Lean 4 formalizations of the two papers.	P	Its README. This is the repository's own claim about itself, and is not evidence of fidelity.
21	The development compiles, with no <code>sorry</code> and no added axioms.	U	A textual search for <code>sorry</code> and <code>axiom</code> lines in the Navier–Stokes and Euler source directories returned no matches; this is not a kernel check. Resolving source: <code>lake build</code> with the declared toolchain and revision, and <code>#printaxioms</code> on the main theorems.

No.	Claim	Mark	Basis or resolving source
22	Other mistranslations exist beyond the two documented (Remark 3.4 of the source).	U	Resolving source: the experiment linked in that remark; a systematic comparison. The monograph’s Chapter 14 treats the remainder as unexamined, and that stands.
23	The Euler formalization is likely to contain substantial mistranslations.	S (forecast)	The source labels this a prediction from preliminary examination. The Euler artifacts were not read. Resolving source: the Euler paper and the Euler directory, compared as in items 1 to 19.
24	The natural-language proof is correct, or incorrect.	U	No part of this audit bears on it.

E.4 What the audit changes in the main text

The audit confirms the textual facts on which Chapters 12 and 13 rest: the statements compared there are the statements in the artifacts, and the exponents the source reports are present in the declarations. It does not confirm the classifications. The weakening and lateral typings remain interpretations (items 8 and 16) because each is a claim of non-entailment relative to a certified background, and no certificate of non-entailment has been produced by the source or by this audit. The change in epistemic standing is therefore the following. The rows of Table 14.1 that rest on the content of the cited declarations are now supported by a reading of those declarations, while the row recording kernel acceptance is not, and the rows recording the classification are not.

The audit also exposes two points that Chapters 12 and 13 did not state. First, the natural-language paper itself converts its flux bound into a bound in A_R immediately after (10.19) (item 17), so that the apparent difference in the variables used by the two estimates is less sharp as a difference in what the surrounding argument needs than as a difference in what each estimate says. Whether that matters is open. Second, the paper’s own lemma depends on a particular absorption (powers below 2), and a downstream check of the Lean bound should be made against that step and not against (10.19) as a free-standing inequality.

E.5 Audit of the claims of Chapter 20

Chapter 20 reports Section 5.1 of the source. The items are marked below.

Table E.3: Claims of Chapter 20 about the Meta project, the declarations and the Fermat’s Last Theorem formalization.

No.	Claim	Mark	Basis or resolving source
25	A paper by Rammal and coauthors (arXiv:2605.29955) reports translating 26 open-access textbooks into Lean 4, producing a library of over 45,000 declarations and about 500 thousand lines.	P	Abstract and conclusion of the paper. These are the paper’s own reports of its output.
26	The paper reports 2,855 of 4,007 target statements formalized (71.3%) over the 26 books, and 40 of 72 (55.6%) for <i>Geometry of Manifolds</i> .	P	Table of per-book results. The figure of 56% quoted in the community thread matches the second entry. Whether the quoted page number is correct was not checked.
27	The paper describes statement-level grading by three “independent” LLM judges, and states that the harness “should not a priori be fully trusted”.	P	Section 3 and Section 5.1 of the paper. The paper’s use of “independent” has the same limitation as the use in the Preface of this monograph.
28	The semantic verification was mainly performed by other AI systems.	P (as a description of the harness), I (as a characterization)	The harness also includes mechanical checks (structural tags, dependency analysis). “Mainly” is the source’s word; the paper’s own division of labour was not measured here.
29	The paper reports a human expert inspection of one example book (Stanley’s <i>Algebraic Combinatorics</i> , 37 of 39 targets) and states that the formalization is mostly solid, that two statements depend on explicitly added axioms, and that the hardest statements are not formalized faithfully.	P	Appendix G. This is a report by the project, on a single book, and bears on the claim of item 31 in both directions.

No.	Claim	Mark	Basis or resolving source
30	Three named members of the Lean community expressed the views quoted in the source.	U (secondary source quotation)	These are quotations as reproduced by the source paper, not quotations read from the original messages; their wording and context are unverified, and they are strong allegations that should not be treated as primary evidence until the original is read. The source cites the thread as “Atlas (Meta)” in the Autoformalization channel, accessed 18 September 2026. Searches on the title and on the quoted sentences found nothing, which is expected for a chat that search engines do not index. Resolving source: the discussion thread in the Autoformalization channel of the community chat, accessed by the source on 18 September 2026. Not reachable here: the chat page returned no messages and the public archive does not cover the period. The statement that the claims were “swiftly refuted” is the source’s characterization of three individual opinions; Chapter 20 reports it only as a dispute, and this appendix keeps that wording.

No.	Claim	Mark	Basis or resolving source
31	The project's claims were in fact refuted.	U	Not established by items 26 to 30. Item 29 reports a mixed outcome on the book inspected by the project itself. A judgment requires inspection of the library's statements by qualified reviewers. The project's repository preserves its original release under v1/, with a per-book <code>report.json</code> for each source book, which is an independent route to the mathematical criticism that does not depend on the thread. Only the README files of that repository were read, and the reports were not examined.

No.	Claim	Mark	Basis or resolving source
32	A formalization of Fermat’s Last Theorem attributed to Anthropic comprises about 13 million lines of Lean.	P (partly) U	The post of 4 September 2026 (modified 5 September) states that the proof is “over 13.4 million lines of code”, that the announcement was made by Anthropic (“officially announced by Anthropic”), and that its author compiled the code base and ran the <code>comparator</code> tool on it with a positive result (“it checks out”). These sentences were seen only as truncated quotations. The post also qualifies the scope of the result, in a formula given as an image that was not read. The blog byline shows the blog account and not a personal name; the identification of the author as K. Buzzard is the source paper’s. Not confirmed: the size as measured by anyone else, the compilation result, the scope qualification, and the Anthropic announcement itself. A separate, earlier human-led formalization project of the same theorem exists at Imperial College; it was not read and is not evidence for this claim.
33	The Leiden Declaration of June 2026 contains the quoted passage about plausible but unreliable arguments and about formalizations.	P	Zenodo record dated 2 June 2026; the passage occurs verbatim as the first numbered statement of the declaration text.
34	A September 2026 declaration titled <i>A Severe Misalignment of AI in Mathematics</i> contains the quoted passage on conceptual understanding and “true/false” statements.	P	Zenodo record dated 11 September 2026; the passage is present verbatim.

No.	Claim	Mark	Basis or resolving source
35	That declaration was initiated by 25 Fields medalists.	U (partly P)	The record lists 25 authors, and the text of the declaration marks each with a Fields Medal and a year. The word “initiated” does not occur in the declaration text. Whether each listed Fields Medal is correctly stated was not checked.

E.6 Register of unverified claims and their resolving sources

Table E.4: Unverified items with the source needed to resolve each. Items that were not accessible are not evidence against the claims.

Items	Precise unverified claim	Required source
7, 21	The cited declarations, and the whole development, are accepted by the Lean kernel without <code>sorry</code> or added axioms; no declaration proves (8.19) at order $m + 4$.	A build with the repository’s declared toolchain and Mathlib revision, <code>#printaxioms</code> output, and a whole-repository dependency analysis.
8, 16	Non-entailment relative to the certified development: (8.19) is not derivable from the Lean estimate, and the two pressure-flux bounds are mutually non-entailing.	A mathematical argument or a counter-model in a stated certified background; the typing of Chapters 12 and 13 cannot be upgraded without it.
9, 17	Whether the extra derivative, and the form of the Lean flux bound, can be absorbed by the downstream argument.	A trace of every use of (8.19) and (10.19) in the natural-language paper, read against the corresponding uses in the Lean development.
13	Identification of <code>ComparisonCutoffs.cutoffR</code> and <code>weightR</code> with φ_R and χ_R .	The definitions in the <code>R3</code> directory of the repository.
18, 19	The lemma-by-lemma route of the Lean pressure-flux proof; non-use of <code>regularized_flux_bound</code> .	A tracing of the proof terms and the compiled dependency graph.
22, 23	Further mistranslations in the Navier–Stokes case; the forecast for the Euler case.	The experiment cited in Remark 3.4 of the source; the Euler paper and Euler directory compared as in items 1 to 19.
24	Correctness of the natural-language proof.	Refereeing of OpenAI’s paper. Not addressed by this monograph.
30, 31	The community thread; whether the Meta project’s claims were refuted.	The thread itself; independent inspection of the library by qualified reviewers.

Items	Precise unverified claim	Required source
32	The scope qualification stated in the post, the compilation result, and the size, as measured independently of the post’s author.	The Anthropic announcement, the formalization’s own repository, and a build with the declared toolchain.
35	That the September declaration was initiated by 25 Fields medalists, and each medal.	The declaration’s provenance statement, if any, and the list of Fields Medal recipients.

E.7 Limits of this audit

The audit was made by one reviewer, once, from text. The text of a PDF produced by a text-extraction tool may differ from its typeset form in a symbol or a line, and line numbers cited are those of the repository as cloned. Declarations were read, not elaborated: a theorem whose statement reads as in Table E.2 may still differ in what its implicit arguments and notation mean, which is a statement-correspondence question of exactly the kind this monograph is about. The audit did not ask whether the natural-language mathematics is right, and no part of it should be read as support for or against OpenAI’s theorem. It did not cover anything in the repository beyond the declarations named. The reproduction details in Table E.2 and the artifact table are given so that a second reviewer can repeat the readings and mark any disagreement against the same item numbers.

Bibliography

- [1] A. Avila et al. (25 listed authors). *A severe misalignment of AI in mathematics*. Zenodo, 11 September 2026. doi:10.5281/zenodo.22737750 (concept record; version record 10.5281/zenodo.22737751).
- [2] H. Barendregt and F. Wiedijk. The challenge of computer mathematics. *Philosophical Transactions of the Royal Society A*, 363(1835):2351–2375, 2005. doi:10.1098/rsta.2005.1650.
- [3] A. Bastounis, F. Circelli, and A. C. Hansen. *Navier–Stokes lost in translation: why Lean verification of AI autoformalisation does not guarantee correct natural language proofs*. arXiv:2610.08144v1 [math.AP], 6 October 2026.
- [4] J. Chen, J. Zhang, Q. Ye, Y. Pruksachatkun, H. Zhang, J. Zhuo, Y. Zhang, Y. Dai, J. Tan, X. Peng, S. Savarese, Z. Chen, L. Qin, and C.-S. Wu. *CoTrace: data recipes for training terminal agents with harness–model co-evolution*. arXiv:2610.10426v1 [cs.CL], 7 October 2026.
- [5] K. Buzzard. FLT: Anthropic has beaten me to it. Xena Project (blog), 4 September 2026 (modified 5 September 2026), <https://xenaproject.wordpress.com/2026/09/04/flt-anthropic-has-beaten-me-to-it/>. Authorship as given by the source paper [3]; the blog byline shows the blog account.
- [6] X. Cheng, L. Wu, K. Wan, L. Hong, and Y. Dong. *Capability scaling-down laws for LLM compression*. arXiv:2610.02462v1 [cs.LG], 1 October 2026.
- [7] M. Davis, H. Putnam, and J. Robinson. The decision problem for exponential Diophantine equations. *Annals of Mathematics*, 74(3):425–436, 1961. doi:10.2307/1970289. (End page not returned by the database; taken from standard citation.)
- [8] B. Dushnik and E. W. Miller. Partially ordered sets. *American Journal of Mathematics*, 63(3):600–610, 1941. doi:10.2307/2371374. (End page not returned by the database; taken from standard citation.)
- [9] A. C. Hansen. On the solvability complexity index, the n -pseudospectrum and approximations of spectra of operators. *Journal of the American Mathematical Society*, 24(1):81–124, 2011 (published online 2010). doi:10.1090/S0894-0347-2010-00676-5.
- [10] J. P. Jones. Universal diophantine equation. *Journal of Symbolic Logic*, 47(3):549–571, 1982. doi:10.2307/2273588.

- [11] D. Ketterer. Untitled public post on Facebook beginning “I train modern LLM’s in mathematical reasoning,” 8 October 2026. <https://www.facebook.com/share/p/19WUisQYxe/>. Cited only for the four-axis distinction and the examples of problem substitution that Chapters 2 and 19 report.
- [12] J. Alper et al. (16 listed authors). *Leiden declaration on artificial intelligence and mathematics*. Zenodo, 2 June 2026. doi:10.5281/zenodo.20302944.
- [13] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381, 2020. doi:10.1145/3372885.3373824.
- [14] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction, CADE-28*, Lecture Notes in Computer Science 12699, pages 625–635, 2021. doi:10.1007/978-3-030-79876-5_37.
- [15] Yu. V. Matiyasevich. Enumerable sets are Diophantine. *Doklady Akademii Nauk SSSR*, 191:279–282, 1970. **[Not located in the databases searched; details from standard citation.]**
- [16] OpenAI. *Finite time blowup for Navier–Stokes*. PDF, 166 pages, undated, retrieved 8 October 2026 from the link in the repository README: <https://cdn.openai.com/pdf/32d9f210-8b73-45e0-91bc-82a30aef8a9a/navier-stokes.pdf>.
- [17] OpenAI. *NavierStokesAndEuler*. Lean 4 repository, commit f9e8bc5b38b6e212696e8a30e3e91517af887bbd, <https://github.com/openai/NavierStokesAndEuler>, read 8 October 2026.
- [18] A. Rammal, N. Patel, F. Gloeckle, A. Hayat, J. Kempe, R. Munos, C. Arnal, and V. Cabannes. *Formalizing mathematics at scale*. arXiv:2605.29955, 28 May 2026.
- [19] H. Rogers, Jr. *Theory of Recursive Functions and Effective Computability*. McGraw-Hill, 1967.
- [20] R. I. Soare. *Recursively Enumerable Sets and Degrees*. Springer, 1987.
- [21] Y. Tang and Y. Yang. *On-policy distillation teaches new skills but not new knowledge*. arXiv:2610.09639v1 [cs.CL], 7 October 2026.
- [22] A. Tarski. On the concept of logical consequence. Original 1936; English translation by J. H. Woodger in *Logic, Semantics, Metamathematics*, Oxford University Press (Clarendon), 1956. **[Original publication details not verified.]**
- [23] W. Wang, H. Li, Y. Gu, and H. Yang. *OnlineQAT: on-policy distillation for ultra-low-bit large language models*. arXiv:2610.09346v1 [cs.CL], 7 October 2026.

Note on the Status of the References

This note records what was checked, against what, and on 8 October 2026. A check of bibliographic metadata establishes that a work exists with the stated authors, title, venue, volume and pages. It does not establish that the work says what the text attributes to it.

Checked against Crossref. Barendregt05, DPR61, DM41, Hansen11, Jones82, LeanMathlib20, Lean4. Authors, title, venue, volume, year and DOI match. For DPR61 and DM41 the database returned the first page only, and the end page is from the standard citation (marked in the entry).

Checked against the arXiv API. BCH26, Chen26, Cheng26, Tang26, Wang26, Rammal26: title, complete author list, primary category and submission date. The author list of Wang26 is complete (four authors); the earlier abbreviated list was extended from this record.

Checked against Zenodo records. Leiden26 (16 listed creators, 2 June 2026) and Avila26 (25 listed creators, 11 September 2026).

Checked at the artifact. OpenAI26NS and OpenAI26Lean were retrieved and read in part, and Buzzard26 was read through a truncating fetch tool (Appendix E).

Checked for existence only (Open Library). Rogers67 (McGraw-Hill, 1967), Soare87 (Springer, 1987) and the book containing Tarski36 (Clarendon Press, 1956). The contents of the Tarski essay and its original 1936 publication details were not verified.

Not located. Matiyasevich70 was not found in Crossref and its details are from the standard citation. A related paper by the same author was found but is not the one cited.

Screenshot inspected, not independently verified. Ketterer26. The page could not be retrieved by automated tools. A screenshot supplied by the author of the monograph was inspected by the reviewer; it shows the post as public under the name Daniel Ketterer, with the opening text quoted in the entry, posted on 8 October 2026. It is a user-supplied image and not an independent capture of the live page, so it is weaker evidence than retrieval of the page. The post has no title. The text consulted for the monograph is a copy supplied by the author, and the post may have been edited since.

Cited at second hand. Results attributed in the text to works not in this list (for instance the sources that Bastounis, Circelli, and Hansen cite for the SCI hierarchy and the nine-unknown refinement) are cited only through the entries above.